

Onyx — Jet Browser, the Onyx web browser based on NetSurf

Onyx · multi-process OS on Raspberry Pi 4

Jet Browser is Onyx's web browser: the upstream NetSurf 3.x core, its framebuffer frontend and its libraries, vendored in `third_party/` and changed for Onyx. It is launched as `jet` (`SD:/apps/jet.app`), its window is titled "Jet", and its About box (Help ► About Jet Browser...) reads "Jet Browser -- the Onyx web browser, based on NetSurf" and credits NetSurf (GPL v2, © The NetSurf Developers) and the libraries' licences (§31). Until 2026-10-01 the app was called NetSurf (`netsurf.app`); the engine, the libraries and the source paths (`third_party/netsurf/`, `user/netsurf/`) keep NetSurf's names, so that they stay comparable with upstream, and so does the rest of this document where it means the engine.

This document lists what differs from upstream and why, as `05-CIRCLE-CHANGES.md` does for Circle. The aim of the changes: pages drawn as Chrome draws them (the user compares with Chrome on Windows).

NetSurf is ours to change: the patches are marked `Onyx:` in the sources (a comment on the line or the block), and each is listed here. The user's guide entry is in `04-USER-GUIDE.md` (Jet Browser), the build in `user/netsurf/README.md`.

1. Where things are

Path	What
<code>third_party/netsurf/</code>	NetSurf: <code>content/</code> (the core: HTML, CSS, layout, redraw), <code>desktop/</code> , <code>frontends/framebuffer/</code> (the frontend Onyx uses), <code>include/</code> , <code>utils/</code>
<code>third_party/libcss/</code>	CSS parser and selection engine (most of the CSS3 work)
<code>third_party/libnsfb/</code>	the framebuffer library (<code>user/nsfb/onyx_surface.c</code> : its Onyx surface)
<code>third_party/freetype-2.14.3/</code>	FreeType (options and modules: <code>user/netsurf/freetype/</code>)
<code>third_party/webref-css-8.7.5/</code>	the CSS specifications' value grammars (W3C webref, MIT): libcss's grammar tables are made from them (§14)
<code>third_party/brotli-1.1.0/</code>	Brotli's decoder only (FreeType's WOFF2), MIT
<code>third_party/wasm3-0.9.2/</code>	wasm3, the WebAssembly interpreter (MIT): the engine alone, its Onyx settings and patches in <code>README.onyx</code> (§27)

Path	What
<code>third_party/quickjs-ng-0.17.0/</code>	QuickJS-ng, the JavaScript engine (ES2023), MIT: the engine alone (<code>README.onyx</code> : its patches); <code>quickjs-jit.c</code> , the Onyx baseline JIT for AArch64 (§30)
<code>third_party/plutovg-1.3.3/</code> , <code>third_party/plutosvg-0.0.8/</code>	PlutoVG, the vector rasteriser (anti-aliased paths, strokes, gradients, clipping, compositing, TrueType text), and PlutoSVG, the SVG renderer on it, MIT: SVG images, inline <code><svg></code> , <code><canvas></code> (§12, §13; PlutoSVG's patches: its <code>README.onyx</code>)
<code>third_party/netsurf/content/handlers/javascript/quickjs/</code>	NetSurf's JavaScript on QuickJS: <code>qjs.c</code> (the engine's glue, the natives), <code>dom.js</code> (the DOM, in JavaScript), <code>canvas.js</code> + <code>qjs_canvas.c</code> (canvas, §13), <code>intl.js</code> + <code>qjs_intl.h</code> (Intl, §15), <code>html5.js</code> (the HTML5 DOM: §17), <code>net.js</code> + <code>qjs_net.c</code> (WebSocket, EventSource, the streamed fetch, Workers: §19), <code>wasm.js</code> + <code>qjs_wasm.c</code> (WebAssembly on wasm3) and <code>crypto.js</code> + <code>qjs_crypto.c</code> (Web Crypto on mbedTLS: §27), <code>qjs_codecache.c</code> (the scripts' bytecode kept on the card: §30), <code>qjs_frames.c</code> (the frames' windows, <code>postMessage</code> between them, <code>MessagePort</code> across realms: §29)
<code>third_party/libhubub/</code> , <code>third_party/libdom/</code> , <code>third_party/libparserutils/</code>	the HTML parser (the current standard's: §16), the DOM, the input decoding
<code>third_party/cldr-48/</code>	the locale data of Intl (CLDR 48 through ICU 78; Unicode License v3), made by <code>tools/tests/netsurf/intl/gendata.js</code>
<code>third_party/fonts/</code> , <code>third_party/dejavu-fonts-ttf-2.37/</code>	the fonts staged into <code>SD:/res/fonts</code>
<code>user/netsurf/</code>	the Onyx glue: <code>onyx_chrome.cpp</code> (the window, its wtk toolbar -- the zoom control, the downloads' button --, the status bar, the History dialog, the About box, the Save dialog: §38; the find bar, the context menu: §40), <code>onyx_fetch.c</code> (HTTP/HTTPS over the Onyx TCP kapis, mbedTLS; each download in a thread of its own), <code>onyx_ws.c</code> (WebSocket and

Path	What
	event streams, each in a thread: §19), <code>onyx_main.c</code> , the makefiles
<code>pc/Jet/</code>	Jet Browser for Windows (§34): <code>winkapi.cpp</code> (the kapi on Win32), <code>jet.mk</code> + <code>build.sh</code> (MinGW-w64) -> <code>pc/dist/Jet/</code> , <code>pc/dist/Jet.zip</code>
<code>tools/tests/netsurf/</code>	the PC test bench: NetSurf built for the PC on the desktop simulator (<code>host.mk</code>), a page to a PNG (<code>shot.sh</code>), the same page in Chromium (<code>chrome.sh</code>), copies of the two sites (<code>getsites.sh</code>), the JavaScript regression test (<code>jstest.sh</code> , <code>pages/js-*.html</code>), the HTTP test (<code>httptest.sh</code> : the fetcher over a local HTTP/1.1 server, <code>httpsrv.py</code> -- keep-alive, chunked, gzip, a redirect, cookies, the Referer, the page drawn as its file:// copy); <code>NS_JSDEBUG=1</code> prints the scripts' errors and <code>console.log</code> , <code>NS_BOXDUMP=<file></code> + F5 dumps the box tree, <code>NS_PERF=1</code> the timings (§9). <code>css3test.sh</code> : css3test.com's score in NetSurf and Chromium; <code>css-check</code> : what libcss keeps (<code>csscheck.c</code> , <code>css-values.txt</code>) (§14). <code>layouttest.sh</code> (+ <code>layoutdiff.sh</code> , <code>nsfonts-conf.sh</code> , <code>pages/layout/</code>): the layout against Chromium box by box (§5); <code>jstest.sh</code> : the DOM, the events, a recursion, <code>fetch</code> / XHR (file:// and data: URLs), the hover events, CSS <code>:hover</code> , <code>localStorage</code> kept, the HTML5 pages (<code>js-html5</code> , <code>js-forms</code> , <code>js-apis</code> , <code>js-ce</code>); <code>html5lib.sh</code> (the parser against the <code>html5lib-tests</code> , its speed: §16), <code>html5test.sh</code> (the <code>html5test.co</code> score: §17), <code>nettest.sh</code> (WebSocket, EventSource, the streamed fetch over a local server, <code>wssrv.py</code> : §19), <code>fxtest.sh</code> (the compositing layers against Chromium pixel by pixel, a hover's partial redraw against a full one: §21); <code>jstest.sh</code> 's <code>js-wasm.html</code> and <code>js-crypto.html</code> (§27: <code>wasm/bench.c</code> compiled by clang, <code>crypto/mkvectors.js</code> 's answers from Chromium's API in Node),

Path	What
	<code>iframetest.sh</code> (iframes, <code>postMessage</code>, <code>MessageChannel</code>, a reCAPTCHA mimic over two local origins: §29); <code>jit/</code>: QuickJS alone (<code>qjsrun</code>), Octane / React, test262, AArch64 instruction counts (§30); <code>quadtest.sh</code> (+ <code>pages/perf-quadratic.html</code>): what grows faster than the page, each case at $n = 1000..8000$ (§36); <code>dltest.sh</code> (+ <code>pages/jet-zoom.html</code>, <code>pages/jet-dl.html</code>): the page zoom, the status bar, the downloads (§38); <code>acidtest.sh</code> (+ <code>acidsrv.py</code>, <code>pages/acid2/</code>, <code>pages/acid3/</code>): Acid2 pixel by pixel, Acid3's score (§37); <code>findtest.sh</code> (+ <code>pages/jet-find.html</code>): find in page, copy and paste, the context menu, Paint pasting a copied image (§40); <code>typeaheadtest.sh</code> (+ <code>pages/stack-typeahead.html</code>, <code>pages/form-scripted.html</code>): Wikipedia's search box -- the stacking order, a menu's footer out of its scroller, <code>pointer-events</code>, Enter in a form a script made (§42); <code>otptest.sh</code> (+ <code>pages/otp-*.html</code>, <code>pages/stack-negz.html</code>, React in <code>pages/react/</code>): one-time-code fields -- Netflix's, six boxes, React-controlled boxes, an invisible field; negative z-indexes (§45)

Build for the Pi: `make -C user/netsurf` (the libraries, their `.a` are committed: `libquickjs.a` among them), then `make -f user/netsurf/netsurf-app.mk link stage`. A header change needs a clean rebuild of what includes it (the rules do not track headers): the libcss objects, `/tmp/nsbuild`. The codegen tools need a host compiler (`BUILD_CC`, `gcc` by default: `build-essential` in WSL; a WSL without one: `BUILD_CC=$PWD/tools/wsl-build-cc.sh`, the Windows MinGW `gcc`). GCC 14 turned some warnings NetSurf has into errors: the makefile keeps them warnings.

The network (`user/netsurf/onyx_fetch.c`): each download runs in a **thread** of its own (kernel v67) -- the DNS, the connect, the TLS handshake and every read block there, not the UI. **HTTP/1.1 with keep-alive**: a response is framed by its `Content-Length` or its chunks (read to the close only when it has neither), then its connection goes back to a **pool** (4, per host / port / scheme, kept 10 s idle) for the next request there -- no DNS, TCP or TLS handshake again; a pooled connection the server closed meanwhile is noticed at its first request (nothing back) and the request sent again on a new one. **Streaming**: the head goes to the core as soon as it is in (its headers, a redirect), then the body as it comes, inflated on the fly (gzip / deflate, brotli, zstd: §24): the HTML parser finds the style sheets, scripts and images while the page still downloads. The worker posts (`kapi_post`) when the head has come, every 32 KB, and at the

end: the UI thread, waiting in `kapi_pump_wait`, wakes at once. **Cookies:** the jar's (`urldb_get_cookie`, read on the UI thread when the fetch is set up) sent, and every `Set-Cookie` of a response -- a redirect's too (a login's session) -- given to NetSurf's jar (`fetch_set_cookie`). **Referer** as Chrome (strict-origin-when-cross-origin: the whole URL within its origin, its origin elsewhere, nothing from https to http) and an `Origin` on the methods other than GET / HEAD (`fetch_get_referer`, an Onyx addition to `content/fetch.c`). **User-Agent:** a current Chrome's on Android (`utils/useragent.c`; Choices' `user_agent` replaces it): the big sites send their light mobile pages -- their desktop script applications (google.com, yahoo.com's Next.js) are too heavy for QuickJS on the Pi; the same in `navigator.userAgent`; `Accept-Language`: Choices' `accept_language`, else French then English. No length limit on a URL's path (it was 1024). 6 download threads at once (and the streams of 4 HTTP/2 connections, §24); the connects one at a time (short now: the kernel caches the DNS answers). An aborted fetch whose thread still runs is freed by that thread. The request is a GET, a POST (a url-encoded or text body) or any method a script asks for (`X-Onyx-Method`, taken off), with the caller's headers; the response goes to the core with its status line and headers (the cache's ones too since §22, not those the decoding makes wrong). Without threads (an older kernel) it is the one-step-per-poll state machine (HTTP/1.1 with `Connection: close`, its chunks undone, the response delivered whole). mbedTLS (`user/tls/onyx_tls.hpp`): its session cache is locked (32 hosts, replaced in turn, kept across launches: §22), its receive buffer 16 KB (a whole record: fewer round trips to the network core); after the handshake its reads do not wait (`onyx_nstls.cpp`: the callers poll with their own idle rules -- a quiet connection was cut after 20 s). A script's request (fetch / XHR) waits 5 min for its answer (a long poll), the others 30 s. **WebSocket and event streams** (`onyx_ws.c`, §19): long-lived connections outside the fetch queue and the cache, each in a thread of its own, their connects in the downloads' turn. The cache (`content/l1cache.c`, Onyx changes): a request's own headers (`l1cache_handle_retrieve_ex`), its HTTP status kept (`l1cache_handle_get_http_code`), its headers in order (`l1cache_handle_get_header_at`).

2. The window and the frontend

- **A native Onyx window** (`user/netsurf/onyx_chrome.cpp`): the toolbar is wtk's (back, forward, reload/stop, home, History, the address field), the frame's close box closes NetSurf; no fltk toolbar (`frontends/framebuffer/gui.c`). Keys: F5, Esc, Alt+Left/Right, F6 / Ctrl+L, Ctrl+H.
- **A back buffer** (`user/nsfb/onyx_surface.c`, the `libnsfb` surface): NetSurf draws into an off-screen buffer of the page's size, and `update` copies the rectangle it redrew into the window's canvas -- the compositor, the apps being preempted, showed half-drawn redraws (a background cleared, then painted: the page flickered at each restyle). Made again at a resize, from what the canvas shows. **One present a main-loop iteration**, after all its redraws (`onyx_chrome_present_later` / `onyx_chrome_flush`, called at the end of `framebuffer_run`): a scroll's moved pixels and its new band are shown together (they were two presents, the first with the band not yet drawn).
- **The main loop waits for an event** (`onyx_input` -> `onyx_chrome_pump_wait` -> `kapi_pump_wait`), up to the next timer: the pointer, a key, a fetch thread's post or the close box wakes it at once -- it slept in blind 20 ms slices.
- **History** — a native dialog (most recent first, Find, Delete, Clear all); the pages visited kept in `SD:/apps/jet.app/History`, the cookies in `.../Cookies` (`ONYX_NS_DATAPATH`, written a few seconds after each page and on exit; never committed). The local history's thumbnails are off (`desktop/browser_history.c`).
- **Scrolling**: the mouse wheel; a scroll blits what stays on screen and redraws the strip that comes in. A new page opens at its top (`GW_EVENT_NEW_CONTENT`).
- **96 dpi, 16 px text** (`gui.c`): a CSS px is a pixel, `medium` is 12 pt = 16 px, the default font is serif (Chrome's Times New Roman) — NetSurf had 90 dpi and 12.8 pt.

3. Fonts (`frontends/framebuffer/font_freetype.c`)

- **Families by their CSS names.** NetSurf used only the generic family. The card's fonts (`SD:/res/fonts`) are found under their names and those of the fonts they stand in for, metric for metric (`fb_card_faces`, `fb_font_aliases`):

font on the card	stands in for
Liberation Sans 2.1.5	Arial, Helvetica, Arimo, Tahoma, Trebuchet MS
Liberation Serif 2.1.5	Times New Roman, Times, Tinos
Selawik 1.01 (Microsoft)	Segoe UI, system-ui
Gelasio	Georgia (and Georgia's ascent / descent)
DejaVu Sans / Serif / Sans Mono	Verdana, Courier New, Consolas, Menlo... — and the fallback

The generic families are Chrome's on Windows: serif → Liberation Serif, sans-serif → Liberation Sans, monospace → DejaVu Sans Mono. All are SIL OFL; a missing file is skipped (DejaVu Sans is the one required).

- **Matching** as CSS Fonts says: the style first (an italic with no italic face is its roman slanted, $x + = y/4$), then the nearest weight (600 and up is bold with two faces).
- **Per-character fallback:** the families of the list in turn, then DejaVu Sans.
- **Advances:** unhinted (fractional), at the exact size -- from the font's units (FreeType rounds the pixel size to a whole ppm for the TrueType fonts that ask for it: DejaVu's 10pt text was measured at 13 px, 2.5 % narrower than Chrome's) --, each glyph drawn at its pen position rounded; glyphs hinted lightly (`FT_LOAD_TARGET_LIGHT`, vertically only). `letter-spacing`, `word-spacing` (new `plot_font_style_t` fields) and `font-variant: small-caps` apply.
- **Vertical metrics** (`fb_font_metrics`, the layout table's `metrics`): as DirectWrite reads them — OS/2's typographic metrics when the font sets `USE_TYPO_METRICS`, else its Windows ascent / descent and `hhea`'s line gap; each rounded to whole pixels as Blink does.
- **Web fonts** (`@font-face`, `content/handlers/html/onyx_webfont.c`): once a document's style sheets are in, each face's first source FreeType reads (TrueType, OpenType/CFF, WOFF, WOFF2) is fetched, handed to the font code for that document only (`add_face` / `release_faces` / `set_scope` in the layout table) and the document laid out again (`font-display: swap`), its cached text widths forgotten first. Faces split by `unicode-range` are fetched when they cover some of Latin-1; a variable font is set to the weight asked (its `wght` axis). `onyx_fetch.c` asks `fonts.googleapis.com` with a current browser's user agent (WOFF2 subsets, ~50 KB a weight, rather than 300 KB TTFs).
- FreeType is built with the TrueType and CFF drivers, zlib (WOFF) and Brotli (WOFF2).
- **No minimum font size** by default (`font_min_size` 1pt, `desktop/options.h`): Chrome has none -- 8pt text was drawn at 8.5pt.

4. CSS (libcxx)

- **Values:** `calc()`, `min()`, `max()`, `clamp()` everywhere a length goes; custom properties and `var()` (resolved at cascade time); CSS Color 4/5 (`rgb()`/`hsl()` space syntax, `hwb()`, `lab()`/`lch()`/`oklab()`/`oklch()`, `color-mix()`); gradients kept as a canonical text (`onyx-gradient: URLs`) for the redraw.
- **Properties added:** `row-gap/gap`, the four `border-*-radius`, `box-shadow`, `text-shadow`, `background-size`, `background-clip` (with text), `text-overflow`, `justify-items/justify-self`, `aspect-ratio`, `object-fit/object-position`, `transform/translate/scale/rotate`, `transform-origin`, `filter`, `backdrop-filter`, `mix-blend-mode` (§21), the grid properties, `mask-image / mask-size / mask-position / mask-repeat` and the `mask` shorthand (the first layer, kept as canonical texts: `onyx_css3b.c`; `-webkit-mask-*` by the prefix rule below); logical properties and the CSS3 shorthands mapped to their physical longhands.
- **Rules and selectors:** `@supports`, `@layer`, `@container` (their content kept), `:is()`, `:where()`, `:focus-within`, `:focus-visible`, `:any-link`.
- **Vendor prefixes:** `-webkit-`, `-moz-`, `-ms-`, `-o-` properties whose standard name is known are that property; `-webkit-text-fill-color` is `color`.
- **@font-face:** `format(woff2)`, `unicode-range`, `font-weight` ranges and any weight 1–1000; `css_stylesheet_font_faces()` lists a sheet's rules.
- **Units:** a length is its unit's exact size times its value (upstream rounded the unit to whole pixels first: 1em of a 12.48 px font was 12 px, 1pt was 1px).
- **The specifications' grammars** (§14): every property and descriptor libcxx does not compute is checked against its official grammar and kept without effect; `@keyframes`, `@counter-style`, `@property`, `@scope...` kept; Selectors 4; the newer units and math functions computed; SVG's fill / stroke... computed.
- **flex shorthand:** `<grow> <shrink>? || <basis> --` the shrink factor right after the grow one (`flex: 1 0 auto` read its 0 as the basis, then failed on `auto`: the whole declaration was dropped; google.com's header).
- **min-width / min-height:** `auto` stay `auto` in the computed style (upstream made them 0 unless the box itself was a flex container): the layout reads them as 0 (`ns_computed_min_*`) except for a flex / grid item's automatic minimum size.
- libcxx's own selection tests still pass: `make -f tools/tests/netsurf/host.mk libcxx-test` (its dump prints the old reading of `min-*: auto; flex: 0 0`'s expected shrink factor corrected to 0).

5. Layout (content/handlers/html/layout*.c)

Measured against Chromium box by box: `tools/tests/netsurf/layouttest.sh` runs `layoutdiff.sh` over the reduced pages of `pages/layout/` (one feature each: flex rows and columns, grid, heights, abs-pos, inline, lines, forms, tables / floats, text, misc, google.com's home and bar) and prints, per page, the boxes that differ (the page's coordinates, and "local": relative to the previous sibling or the parent, so that one misplaced box does not count all that follow). `layoutdiff` gives Chromium NetSurf's fonts and substitutions (`nsfonts-conf.sh`: Verdana is DejaVu Sans, Segoe UI Selawik... as `fb_font_aliases`), unhinted at fractional sizes as Chrome on Windows, and a viewport as high as NetSurf's page: the boxes differ by their layout, not by the fonts a Linux Chromium has. At the start of this work: 380 of 491 boxes differed (238 local); now 38 of 608 (28 local, most of them text rounding and things NetSurf does not draw: `<video>`, the scroll bar's width); news.ycombinator.com 797 of 799 -> 52.

5.1 Flexbox (`layout_flex.c`, rewritten)

The upstream flex layout (a partial CSS Flexbox 9.7) is replaced by the specification's algorithm, section 9, as Blink, Taffy and Yoga implement it:

1. the items: the in-flow children in `order` (a stable sort); their margins, borders, paddings, specified sizes and min / max sizes (percentages of the container);
2. each item's **flex base size** -- `flex-basis` (a length, a percentage of a definite main size), `auto` (its width / height), `content` (its max-content width in a row; in a column its height laid out at its cross size) -- or the size transferred through a ratio (an image's natural ratio, `aspect-ratio`) from a definite or stretched cross size (an image with no size in a 100 px high row is 100 px high, its width from its ratio); its **automatic minimum size** (4.5: its min-content size, no larger than its specified or transferred size, for an item that is not a scroll container); its hypothetical main size;
3. a column with an auto height: its items' hypothetical sizes, within `min-height` / `max-height` -- and the items flexed in that height (a `min-height: 100%` page column whose `flex: 1` middle pushes the footer down: google.com);
4. the lines (`flex-wrap`), then the flexible lengths resolved line by line (9.7: the inflexible items frozen, the free space shared by `flex-grow` or by `flex-shrink` scaled by the base size, min / max violations frozen) in floating point;
5. the cross sizes: each item laid out at its main size; the lines' cross sizes (baseline alignment: the largest ascent plus the largest descent), a single line as tall as a definite container, `align-content`; the items with `align-self: stretch` and an auto cross size stretched to their line -- their height is then **definite** (`DEF_HEIGHT`, below) and they are laid out again only when their content depends on it (percentage heights; a flex, grid or table box, a form control when the height changed): the layouts stay few;
6. main-axis alignment (auto margins, `justify-content`, the gaps, `row-reverse` / `column-reverse`), cross-axis alignment (auto margins, `align-self`, `wrap-reverse`).

Sizes and positions are floating point until stored, each box's left edge and size rounded (as Chrome's rectangles are). The container's own width, height, margins, paddings and borders are its caller's (block context, flex / grid item, float, absolute box); `layout_flex` applies its `min-height` / `max-height`. A **flex container's intrinsic widths** are its items' contributions (`layout_minmax_flex`, 9.9.1: the larger of an item's content and its specified width, clamped by its flex base size -- a maximum if it does not grow, a minimum if it does not shrink -- then by its min / max-width; summed in a row, the widest in a column); a flex item's own `min_width` / `max_width` stay its content's (its automatic minimum, its content basis). Each item's layout goes through a per-pass memo (§26): an item is laid out once per set of inputs, however deep the nesting (it was 2^{depth}).

5.2 Heights and blocks

- **Percentage heights** (`layout_pct_height_base`, `layout_internal.h`): against a containing block whose height is specified, or given by its flex / grid container (`DEF_HEIGHT`, a new box flag: a stretched or flexed item's height, definite for its children -- CSS Flexbox 9.8), an absolute box's always; `min-height` and `max-height` in percentages too (they were ignored), with `box-sizing`; a specified height is kept within `min-height` / `max-height`.
- **aspect-ratio**: an auto width from a definite height, an auto height from the width (blocks, floats, inline-blocks, flex items, intrinsic widths; the ratio of the border box with `box-sizing: border-box`).

- **display: contents** (`box_construct.c`): the element's box is kept off the tree (its style, which its children inherit), its children are its parent's (flex items of the parent's flex container).
- A flex / grid container never collapses margins through it, and gives its ancestors `HAS_HEIGHT` (the margins before it go above them, not inside); it avoids the floats beside it (a formatting context root); a floated flex container is laid out as one.
- **Compressible replaced elements** (CSS Sizing 3 5.2.2): an image with a percentage width or max-width contributes nothing to a min-content width (a `max-width: 100%` image in a grid or flex track, a table cell); an image with a definite height and an auto width contributes its height times its ratio; an image with an auto width and a `max-height` (or `min-height`) is as wide as that height times its ratio (Facebook's logo, `max-height: 60px`, was 180 px wide).
- A **replaced element with display: flex / grid** (an `<img>`, an `<object>`) is a block (an inline-block when inline-flex): Facebook's icons were empty flex containers, 0 px high.
- **Absolute boxes**: the containing block of an absolute child of a positioned flex / grid container is its padding box; the available width is taken after the padding; the static position of an absolute child of a flex container follows `justify-content` / `align-items` / `align-self` (`layout_abs_flex_static`).

5.3 Lines and inline boxes

- **Lines on their baseline** (CSS 2.1 10.8): the block's strut, text (its font's ascent below the half leading), inline-blocks and buttons (their last / first line's baseline), images (their bottom edge), `vertical-align` (sub, super, middle, text-top, text-bottom, top, bottom, lengths) -- NetSurf put each box at the top of the line and a text's baseline three quarters down. `line-height: normal` is the font's spacing (was 1.3 em); a run of lines keeps its fractions of a pixel. A block's own `vertical-align` (a table cell's `middle`) is not its text's.
- **Text and inline boxes are their font's content area** (ascent + descent) around the baseline, not their line-height box: their background and their rectangle (`getBoundingClientRect`) are Chrome's; the text is drawn on the same baseline.
- **A line of floats only** between two blocks (a floated figure between paragraphs) no longer stops the margins collapsing through it (`layout_floats_only`): the paragraphs were apart by both margins. Its floats are placed below the pending margin all the same; a clearance ends the pending margins.
- **Zero-height lines** (9.4.2): a line with no text, no atomic inline, no `<br>` and no inline with a horizontal margin, border or padding has no height (the empty start of an inline holding a block made a line of its own); a form control's value line always has.
- **Line breaks only at break opportunities**: a space, an atomic inline -- (`<a>x.com</a>`) was broken between (and the link; a run without one stays on its line (overflowing).
- `transform`'s translation moves a box as a relative offset (paint and hit-testing); a transform that is more than a translation is painted through a layer (§21).
- The space after an inline-block, inline-flex, image or control is kept.

5.4 Form controls

- The UA sheet (`resources/default.css`) has Chrome's controls: 13.333px Arial, no margins; text fields `padding: 1px 2px`; `border: 2px inset`; buttons `padding: 1px 6px`; `border: 2px outset`, `border-box`; textareas `padding: 2px`; `border: 1px solid`, monospace; selects `border-box` with a 1 px border; check boxes and radios 13 px with Chrome's margins.

- **Intrinsic sizes** (`layout_text_control_size`): a text field's width from its `size` (20) -- that many average characters of its font (Arial's 0.574 em, rounded) plus the widest one's excess (1.26 em); a textarea's from `cols` (20) monospace advances plus a 15 px scroll bar and its height from `rows` (2) lines. They are no longer CSS width / height hints (`css/hints.c`): a style's width or a flex stretch overrides them, as in Chrome. A block-level control keeps its intrinsic width.
- **Baselines**: a text field's is its text line's, centred in its content box; a check box's is its border box's bottom; a button's content is centred in its height and its baseline follows.
- `<button>` keeps a flex / grid / block box; inputs and selects stay flex / grid items (`box_special.c`: they were made inline-blocks again).

5.5 Tables

- A cell's `width` is its content box's: the column holds its padding and border (`table.c`); a table's `width` follows `box-sizing` (the UA sheet sets `border-box` on tables, as Chrome's).
- A table wider than its columns' max-content widths gives the spare width to the auto columns in proportion to their max-content widths (it was shared equally: Hacker News' header put its "login" column at 212 px instead of 84).
- `table-layout: fixed`: the columns from the first row's cells, the rest shared equally.
- Row groups and rows span the cells, not the border spacing around them (their boxes as Chrome's; the cells are moved to match, nothing moves on screen).
- **Captions** (`display: table-caption`, CSS 2.1 17.4): a caption was an inline, put in an anonymous cell beside the table's content (Wikipedia's figures -- `display: table` with a `table-caption` figcaption -- showed their caption beside the picture, not floated). A caption is now a block (`TABLE_CAPTION`, a new box flag) that `box_normalise_table` keeps out of the grid: the table's first children (`caption-side: top`) or last (`bottom`). `layout_table` lays out the grid with them detached (`layout_captions_*`), then each caption as wide as the table's border box, above or below it; the table's box holds them (as Chrome's table wrapper box: `getBoundingClientRect` agrees), and the redraw paints its background and borders around the grid only. A caption's minimum width widens the table (its min / max widths, and its columns: the extra shared equally). Test page: `pages/layout/captions.html`.

5.6 Grid

`layout_grid.c` (new): templates, `repeat()`, `fr`, `minmax()`, auto tracks, placement, areas, `auto-fill/auto-fit`, gaps, alignment. A grid container is a `BOX_FLEX` / `BOX_INLINE_FLEX` box whose `display` is `grid`. Its items' automatic minimum is their content's (libcss now keeps `min-width: auto`, below).

5.7 UA sheet (HTML5)

[`hidden`], `template`, `datalist`, `dialog: not([open])` and the other non-rendered elements are hidden (google.com's screen-reader text showed in its search box); an open `dialog` is centred; the headings', paragraphs', lists', `dl / dd`, `figure`, `blockquote`, `fieldset` / `legend` margins and paddings are the HTML standard's; `body` has no `line-height` (it was 1.33).

6. Painting (`content/handlers/html/redraw.c`, `onyx_paint.c`)

- New plotter operations (`include/netsurf/onyx_paint.h`, `plotters.h`): `onyx_shape` (rounded boxes, rings, blurred shadows, gradients), `onyx_round_clip`, `onyx_text_paint` (text in a gradient); the framebuffer's in `frontends/framebuffer/onyx_paint.c` — anti-aliased by signed distance, Gaussian (`erfc`) shadows, linear / radial / conic / repeating gradients. Made fast (a full redraw of

kotonviolins.com: 12.6 ms -> 4.9 ms on the PC, the same pixels): the rows' fully covered spans filled without a distance a pixel (a shadow's too: 3 sigma inside), the insides of a ring's hole and of a shadow's caster skipped, the blur's coverage from a table (`blur_lut`: no `expf` a pixel), a gradient's colours from a table of 1025 made once per gradient and kept (`glut_get`, keyed by its stops), a linear gradient's position stepped along the row, the blend without a division (`div255`).

- **background-size** (`contain`, `cover`, lengths, percentages, one side `auto` from the image's ratio; `onyx_background_size` in `redraw.c`): libcss parsed it, the redraw drew every background image at its own size -- the SVG backgrounds of the big sites (icons sized by CSS) came out wrong.
- `border-radius`, `box-shadow` (outer and inset), gradient backgrounds, rounded clips of a box's content; `background-clip: text` paints the descendants' text with the box's background.
- Translucent colours are blended; a translucent fill no longer hides what is under it (`desktop/knockout.c`); `color: transparent` draws nothing.
- **The painting order of positioned boxes** (CSS 2.1 appendix E, simplified): NetSurf painted the boxes in the tree's order, so a menu panel positioned over the page (a header's absolute dropdown, a sticky bar with a `z-index`) was drawn under what follows it. A positioned box (and a flex / grid item with a `z-index`) is now put off while its layer — the page, or the positioned box it is in — is painted, then painted after that layer's in-flow content, sorted by `z-index` (`auto`: 0), each a layer of its own; a negative `z-index` is painted in place (`html_redraw_layer_z`, `onyx_layer_*`).
- **Floats inside a positioned layer** are painted with that layer, in the tree's order (`onyx_float_layered`, `onyx_float_container`): they were painted with the page's floats, under the layer's content or not at all when the layer was put off (Wikipedia's article images).
- **A clip outside the surface** (`framebuffer_plot_clip`): a box entirely above or left of the screen gave a clip the framebuffer refused -- the plot failed and the rest of the redraw was dropped (nothing inside Wikipedia's `#content` was painted: a mask element at `y = -99821`). Such a clip is now an empty one-pixel clip.
- **mask-image** (`html/onyx_mask.c`): pages draw icons as a box with a background colour and a mask image (Facebook's `<img>`s, their own picture pushed away with `object-position: 10000px 10000px`; Wikipedia's icons, `-webkit-mask-image` SVGs): what shows is the mask's shape in that colour. The mask image is fetched as the box's `box->mask` (`html_fetch_mask`, `object.c`); the redraw paints, over the border box, the image at its `mask-size` (`cover`, `contain`, lengths, `auto`), `mask-position` and `mask-repeat`, through a plotter table whose bitmap plot draws a copy of the bitmap in the background colour with the bitmap's alpha (an SVG mask is a bitmap too: PlutoSVG rasterises it). The box's own background and image are not painted; nothing is while the mask loads or if it failed (as in Chrome). The children are painted unmasked. Test page: <pages/css-mask.html>.
- **The box under the pointer** follows the same order (`interaction.c`, `onyx_hit_*`): the last box painted there, then its ancestors' links, controls and titles, root first — a click on an open menu reaches its link, not the page under it. A click on a link's text is the `<a>`'s (NetSurf gave the block's node: a text box has none).
- **Fixed boxes** (google.com's search: a click on its box opens a full-viewport overlay, `position: fixed`; `width: 100%`; `height: 100%`, inside a fixed `<body>` and zero-height `overflow: hidden` blocks — on the Pi the box "took no focus": the overlay holding the focused textarea was invisible):
 - their containing block is the viewport (`layout_position_absolute`): they were laid out against their positioned ancestor (the overlay was the fixed body's shrunk width); `height: 100%` is the viewport's height;

- their ancestors get `HAS_FIXED` (a new box flag): the descendant boxes' extents keep them through an `overflow: hidden` ancestor, the redraw does not skip such an ancestor (outside the clip, empty, clipped to nothing: its children painted in an empty clip), a fixed box put off to its layer gets the redraw's own clip, and the hit test (`box_contains_point`) reaches them.
- **Scrolling inner scrollers** (a consent screen: a fixed overlay, its panel `overflow: auto`, the accept button at its end): the wheel follows the box a click reaches (`html_hit_path`, the deepest first; a scroller at its end hands the rest to its ancestors) — it was the first box in the tree's order, which missed overlays; the keys (arrows, Page Up / Down, space, Home / End) scroll the scroller under the pointer before the window; an element's scroller fires its `scroll` event, and `scrollTop` / `scrollLeft` / `scrollHeight` / `scrollWidth` / `scrollTo()` / `scrollBy()` are its (`N.boxScroll`, `N.boxScrollTo`; they were 0 / no-ops). Also in an `iframe`. Tests: `pages/js-scrollers.html`, `js-scrollframe.html` (jstest.sh).
- **The wheel event:** the scripts get a `wheel` event (a `WheelEvent`, `deltaX` / `deltaY` in px) at the element under the pointer before anything scrolls; one they prevent scrolls nothing (a page's own scroller: its content moved by the script). The wheel reached no script before. A click also gives `pointerdown` / `pointerup` before `mousedown` / `mouseup` (as Chrome). Tests: `pages/js-wheel.html`, `js-consent.html` (a flex column's `overflow: auto` middle), `js-consent-body.html` (the body its own scroller).
- **A dialog capped to the viewport** (Facebook's cookie dialog: a fixed backdrop, `inset: 0`, centring a flex column with `max-height: 90%`, its middle `overflow: auto`, the buttons pinned): an absolute / fixed box whose height is given by its insets (`top` and `bottom` set, `height: auto`) now knows it before its content is laid out (`layout_absolute`, `DEF_HEIGHT`): a flex container centres and flexes its items in it, its children's percentages resolve against it. The dialog took its whole content's height (the buttons below the screen, nothing to scroll). A stretched grid item that is a scroll container is its area's height even when its content is taller (`layout_grid.c`: a `minmax(0, 1fr)` middle row). Tests: `pages/layout/dialogs.html` (layouttest: 0 of 49 differ), `pages/js-dialog.html` (jstest).
- **No touch screen:** `ontouchstart` / `ontouchend` / `ontouchmove` are gone from the elements and the window ('`ontouchstart`' in `window` told the pages there was one).
- `element.focus()` on a text field or a textarea puts the browser's caret in it (what is typed goes there, as after a click); in a handler that has just shown it (no box yet), after the next rebox (`html_script_focus_control`). Test: `pages/js-focus.html`.

7. JavaScript (QuickJS)

NetSurf's JavaScript was Duktape (ES5) with bindings generated by `nsgenbind`, and the page was never laid out again after a script changed it. It is now **QuickJS** (QuickJS-ng 0.17.0, ES2023: classes, `let/const`, arrow functions, promises, `async/await`, optional chaining...) with the DOM written in JavaScript:

- `quickjs/qjs.c` implements `javascript/js.h` on QuickJS: one runtime per window, one context per document, `js_exec` for the scripts NetSurf runs as it parses. `libdom`'s nodes are wrapped (one JavaScript object per node, for the document's life) and a small set of **natives** is given to the prelude: the tree (`parent`, `children`, `insert`, `remove`, `clone`, `attributes`, `text`), `setHTML` (hubbub's fragment parser: `innerHTML`), the boxes (a node's rectangle, whether it is displayed, a few computed properties), the window (its scroll, its size), timers, navigation, history, cookies (NetSurf's `urldb`), `document.write` (while parsing), the form controls (`formValue`, `formChecked`:

NetSurf's controls, what the user typed) and `submit` (NetSurf sends the form: its method, its encoding).

- **quickjs/dom.js** (compiled in as a C string, `qjs_dom_js.h`, made by the makefiles) is the DOM on those natives: `Node`, `Element`, `HTMLElement` and ~60 element classes, `Document`, events (capture, target, bubble; `on*` properties and attributes; `preventDefault`), `querySelector/matches/closest` (its own selector engine: `:is`, `:not`, `:has`, `:nth-*`...), `classList`, `dataset`, `style`, `innerHTML/outerHTML`, `MutationObserver`, `IntersectionObserver`, `ResizeObserver`, `matchMedia`, `getComputedStyle`, `localStorage` (kept: a file per origin next to the cookie file, `ls-<origin>.json`, written a turn after a change; `sessionStorage` in memory), `URL`, `URLSearchParams`, `FormData`, `TextEncoder` / `TextDecoder` (UTF-8 in C: `N.utf8`), timers, `requestAnimationFrame` (paced by the content's frames: §22), `location`, `history`, `navigator`.
- **fetch and XMLHttpRequest**: `fetch` (a promise of a `Response`: `ok`, `status`, `statusText`, `url`, `redirected`, `headers`; `text()`, `json()`, `arrayBuffer()`, `bytes()`, `blob()`, `formData()`, `clone()`; an `AbortSignal` cancels it), `Headers`, `Request`, `Response` (`Response.json`, `.error`, `.redirect`), and `XMLHttpRequest` (`readyState` 1 → 4 with `readystatechange`, `loadstart` / `progress` / `load` / `error` / `abort` / `timeout` / `loadend`, `responseType` '' / `text` / `json` / `arraybuffer` / `blob`, `setRequestHeader`, `getResponseHeader`, `getAllResponseHeaders`, `timeout`, `abort`). Bodies: a string, `URLSearchParams`, `FormData` (url-encoded), a `Blob`, `bytes` (as text: no NUL). On one native, `request(method, url, body, headers, binary, cb)` (`qjs.c`): NetSurf's low-level cache with `LLCACHE_RETRIEVE_FORCE_FETCH`, the callback on the page's thread (the promises' jobs run after it, a changed DOM laid out again); the requests in flight are cancelled when the document goes. The response as it comes -- `Response.body` a stream fed as the bytes arrive, XHR's `LOADING` and `progress` --, `WebSocket`, `EventSource` and `Workers`: §19. Not yet: synchronous XHR (it runs async), multipart bodies, CORS checks (every origin answers); `responseXML` and `responseType` = `'document'`: `html5.js` (§17).
- **A changed DOM is laid out again** (`html.c`, `html_script_dom_changed`): once a script is done, NetSurf builds the document's boxes again (`dom_to_box_now`, synchronously) and lays it out — a menu a script opens, a class toggled, nodes added. The old boxes' objects (images) are taken over by the new ones (`html_fetch_object`: no fetch, no flicker), the form controls kept (their text areas, their typed text; a select's options read again), the focus and the iframes moved to the new boxes. A script asking for a rectangle gets the new layout at once (`html_script_layout_now`).
- **The browser's events** reach the scripts (`html_script_event`): `mousedown`, `mouseup`, `click` at the element under the pointer — a click a script prevents neither follows its link nor sends its form —; the pointer's moves (`onyx:hover` from `interaction.c`, every move: `dom.js` makes `mouseover` / `mouseout` with their `relatedTarget`, `mouseenter` / `mouseleave` for each ancestor entered or left, and `mousemove`); **CSS :hover** -- the node under the pointer is kept (`html_content.hover_node`); `libcss` asks `node_is_hover` (`css/select.c`: that node and its ancestors), and when it changes the styles are made again (`html_script_dom_changed`) -- only when the style sheets have `:hover` rules (a `:hover` selector was tried: `uses_hover`); `keydown`/`keypress`/`keyup` at the focused control or the document (a key prevented is not typed); `input` (typing; deferred a turn: the text area is not changed under its feet), `change` (checkboxes, radios, a select's menu), `submit` (a submit button, Enter; prevented: not sent); the window's `scroll` (the frontend tells the core: `browser_window_scrolled`), `resize`, `DOMContentLoaded` (the document parsed), `load` (laid out, its images in; `<body onload>`).
- **WebAssembly** (`wasm3`) and **Web Crypto** (`mbedtls`): §27.
- **Speed**: the interpreter's measured wins, the code cache on the card, the window kept responsive, the JIT: §30.

- A script runs at most `script_timeout` seconds (NetSurf's option); its recursion is stopped past **4 MB** of stack (a `RangeError`). The Onyx app has an 8 MB stack for it: `stack = 8M` in `SD:/apps/jet.app/app.txt`, read by the kernel (`AppStackSize`, `kernel/kernel.cpp`) — the default is 256 KB. `NS_JSDEBUG=1` (the PC bench) prints the scripts' errors and `console.log` on stderr.
- **libdom**: a changed `class` attribute updates the element's classes (`element.c`: they were cached when the attribute was made, so a `classList` change did not restyle).
- The Duktape backend (`javascript/duktape`, `user/netsurf/gen/duktape`) is no longer built (and, since §39, no longer in the repository).

9. Performance

- **Timings** (`user/netsurf/onyx_perf.h`): with the file `SD:/apps/jet.app/perf` (the PC bench: `NS_PERF=1`), each step longer than 1 ms is printed -- `ONYX-PERF rebox:boxes, rebox:reformat, layout, redraw WxH, hover:restyle`, and why a hover built the boxes again (`hover:rebox (<reason>)`), each script run (`js:exec <url> (<size>)`) and each call into the scripts (`js:call <event>`) -- on stderr, the kernel log on the Pi. The file `SD:/apps/jet.app/jsdebug` (the PC: `NS_JSDEBUG=1`) prints the scripts' errors and `console.log` there too.
- **The scripts do not keep the window from responding** (`frontends/framebuffer/schedule.c`): the scheduler runs its due callbacks (the page's timers among them) for 40 ms at most, then hands the main loop back to the events (it goes on at once after). A single script is still cut off after Choices' `script_timeout` (10 s). Seen on google.com and yahoo.com with the Chrome User-Agent (their full script applications): the window stopped responding.
- **CSS :hover without the whole page** (`content/handlers/html/onyx_hover.c`): the selection notes each node a `:hover` selector is tried on (`nscss_hover_note`, `select.c`); when the node under the pointer changes, only the topmost element that left (entered) the hover chain among those has its subtree styled again (`box_style_select`, the construction's own), without building the boxes. When the new styles differ only in how boxes are painted (`libcss's css_computed_style_paint_only_change`: colours, backgrounds, border colours and radii, outline, shadows, text decoration, visibility, opacity, z-index, cursor), the boxes take them in place -- an element's own boxes and those without a node (text, `::before`, markers, an inline's end) matched by owning element *and* style, `libcss` interning styles -- the borders' colours copied again, the anonymous boxes' styles derived again, and only their rectangles are redrawn. A change of `transform / translate` (a card lifted on hover) moves the box (its descendants follow) as the layout would, its ancestors' descendant bounds grown; a change of its effects (opacity, a transform beyond a translation, filters: §21) redraws all it holds where it was painted and where it is. Anything else (a layout property, a pseudo-element appearing, a background image to fetch, a `:hover` tried on a sibling, a table's borders) builds the boxes again as before (`html_script_dom_changed`). On the two sites every hover is now a restyle: no rebox, the pixels those of a rebox (`NS_HOVER_FULL=1`: always the rebox, to compare). The replaced style results are kept until the next rebox (a box the walk missed would still point at them).
- **The rebox after a script's change** keeps the elements' style selections and selects again only what the change can have restyled; attribute-only changes are restyled in the boxes as a hover is (`onyx_hover_restyle_nodes`); reboxes are coalesced and throttled: §26.
- **libcss**: `css_computed_style_paint_only_change` (`a`, `b`, &moved) (`src/select/arena.c`) copies the paint properties' bits and values of `b` over a copy of `a`'s, then compares the rest as the interning does; the bits' positions are in `src/select/onyx_propbits.h`, copied from `autogenerated_propget.h` (which `#undefs` them) by `onyx_propbits.py`: run it (after `select_generator.py`) whenever `select_config.py` changes the layout.

- The windows' `dom.js` checkout: the makefiles strip the CRs before embedding it (a Windows checkout, `core.autocrlf`, broke `qjs_dom_js.h`).

10. Scripts of the big sites (yahoo.com's Next.js, 2026-09-30)

Found by running a saved copy of yahoo.com on the PC bench (`NS_PERF=1 NS_JSDEBUG=1`):

- **<script nomodule> is not run** (`html/script.c`), as in Chrome: it is for the browsers without ES modules. Run, yahoo.com's polyfills replaced `Promise`, and `queueMicrotask` (built on `Promise`) and the polyfill called each other for ever: the window froze.
- `queueMicrotask` uses the engine's own `Promise`, kept when `dom.js` starts (`NativePromise`), whatever a page's polyfill does to `Promise`.
- `document.currentScript`: the `<script>` element whose code runs (`js_set_current_script`, called around each run by `html/script.c`; `struct html_script` keeps its element) -- webpack finds its chunks' path with it.
- The file fetcher types `.js` / `.mjs` (JavaScript), `.json`, `.webp`, `.ico`, `.woff` / `.woff2` / `.ttf` / `.otf`, `.txt` (`frontends/framebuffer/fetch.c`): a page saved with its scripts runs from the card (they were typed `text/html`: never run).
- `tools/tests/netsurf/pages/js-reactreveal.html`: React 18's streaming reveal (`$RC` / `$RV`), its comments, `insertBefore` (`x`, `null`).

11. ES modules, CSS feature detection, a CSSOM (css3test.com: 0% -> 23%)

- **ES modules** (`<script type="module">`, inline or `src`): `html/script.c` hands the element to the scripts (`js_module_script` -> the event `onnx:module`). `dom.js` fetches the module and, in parallel, every module its imports name (`import` / `export ... from` / `import(...)` found in the source: `modGraph`), gives the sources to `qjs.c` (`moduleSource`), then `moduleRun` compiles the root module; QuickJS's loader (`qjs_mod_loader`, `qjs_mod_normalize`: the specifier resolved against the importing module's URL) compiles the imports from the sources -- a loader cannot wait for the network -- and a source still missing (an import `dom.js` did not see) comes back in `moduleRun`'s result: fetched, then again. `import.meta.url` is set. Deferred as in a browser: after parsing, in the document's order, before `DOMContentLoaded` (which waits for them), `load` after them; an `async` module as soon as it is ready. No import maps (a bare specifier does not load).
- **CSS.supports and element.style answer from libcss** (they said yes to everything): `cssKept` (`text`, `inline`) (`qjs.c`) -> `nscss_text_kept` (`css/select.c`) parses the text as an inline style or a sheet and `css_stylesheet_onyx_kept` (`libcss`, `src/stylesheet.c`) says what was kept (rules, declaration bytecode). A property is known when `prop: inherit` is kept; a value valid when `prop: value` is; a selector when `sel { color: red }` keeps a rule. `'prop'` in `element.style` is true for the known properties only; an invalid value is not set (the old one stays); `CSS.supports (prop, value)` and `CSS.supports ("(a: b) and (not (c: d))", "selector(...))"` are real.
- **A read-only CSSOM**: `<style>.sheet` (`cssRules`: the style rules with their valid declarations, `@font-face` with the descriptors `libcss` reads -- `font-family`, `src`, `font-style`, `font-weight`, `unicode-range` --, `@page`, `@media` / `@supports` with their rules, `@import`, `@namespace`; what `libcss` drops is not there), `document.styleSheets` (the `<style>s`). Since made a real, writable CSSOM with the linked sheets (§14); `matchMedia` answered by `libcss`, and the answers' honesty checked against Chromium (`js-cssdetect.html`): §23.

- `css3test.com` (Lea Verou's) now runs -- its engine is a module graph of 156 modules -- and scores **23%** (1154 of 6419 tests), honestly (what libcss parses; Chrome ~ 75%). It found a double free in libcss's Onyx gradient parser (a prefixed legacy `radial-gradient` such as `-webkit-radial-gradient(center, circle, ...)`: its buffers freed, then used and freed again -- `src/parse/properties/onyx_background.c`), which could crash on real pages too.
- `tools/tests/netsurf/pages/js-module.html` (+ `js-mod-a/b/c.js`), in `jstest.sh`: a module graph, `import.meta.url`, `import()`, an inline module, the order, `CSS.supports`, `element.style`.

12. SVG (images and inline `<svg>`, on PlutoSVG)

NetSurf's own SVG handler (`image/svg.c`) needs libsvgtiny, never vendored: SVG images were not drawn, inline `<svg>` neither (the logos and icons of `google.com`, `bbc.co.uk`, Facebook).

- **The image handler** (`content/handlers/image/onyx_svg.c`, registered in `image.c` for `image/svg+xml` and `image/svg`; the file fetcher types `.svg`): the document is parsed once by PlutoSVG; its intrinsic size is its `width` / `height`, else its `viewBox`'s, else 300 x 150 (a width alone: 150 high, as Chrome). It is rasterised -- anti-aliased, into a NetSurf bitmap -- at the size a redraw asks for and kept for the next redraws (six sizes per image, the least recently used replaced): a redraw is a bitmap plot, never a rasterisation. An SVG with a `viewBox` is drawn in a viewport of the drawn size (its `preserveAspectRatio` applies), one without is scaled, as browsers do for images; a drawing over 2048 x 2048 pixels is rasterised smaller and scaled up. `currentColor` is black in an image. `<img>`, backgrounds (tiled too), `list-style-image`, `<object>` and `<embed>` use it; a canvas' `drawImage` and favicons get its last size (`get_internal`). A bitmap replaced during a redraw is destroyed once the main loop turns: the knockout (`desktop/knockout.c`) may still have its plot queued.
- **Inline `<svg>`** (`content/handlers/html/onyx_svg_inline.c`, one call in `box_special.c`'s `convert_special_elements`): hubbub builds an `<svg>` and its descendants in the SVG namespace (their camelCase names); an outer `<svg>` becomes a replaced box -- its children are not converted -- whose object is the subtree written out as SVG text: the elements (their names lower-cased: libdom upper-cases an HTML document's; the camelCase ones restored), their attributes, a `<style>`'s text, the element's CSS `color` as the root's `color` (its `currentColor`), a `var(--x[, fallback])` in a value replaced by the `<svg>`'s custom property (libcss: `css_onyx_node_var` reads it from the node data its selection keeps -- `google.com`'s menu icon is `fill: var(--IXoxUe)`), the root's `width` / `height` in `em` / `rem` / `ex` written in `px` with the element's font size (`bbc.co.uk`'s logo is `width="7em"`). The elements it names by `href="#id" / url(#id)` that are elsewhere in the page (an icon sprite's `<symbol>`s, a shared gradient) are copied into a `<defs>` at its end. The text is the box's object as a `data:image/svg+xml;base64,...` URL through the usual `html_fetch_object`: the low-level cache shares one content between identical icons, and the SVG image handler draws it at the box's size (CSS `width` / `height` size it as an image). A script's change builds the boxes again (`html_script_dom_changed`): a new text, a new URL; so does a `:hover` that changes the colour of an inline `<svg>` (`onyx_hover.c` asks `onyx_svg_box_is_inline`: its `currentColor` is in its text, a restyle in place would keep the old one -- `pages/svg-hover.html`). Costs, kept low for the reboxes the scripts cause: an element named by id is looked up (libdom walks the tree) and written once per main-loop turn, the URLs are kept by their text (256, 512 KB) -- a page of 300 sprite icons: its rebox 4.4-6.9 ms -> 3.0-5.0 ms on the PC (2.2 ms without the icons). `NS_SVGDEBUG=1` prints each SVG text on stderr (the PC bench).
- **PlutoSVG's Onyx patches** (`third_party/plutosvg-0.0.8/README.onyx`): the `<style>` sheets (type, class, id selectors, descendant and child combinators, specificity; below `style=""`, above

the attributes), `clip-path` (`clipPathUnits`, `transforms`), `<a>` and `<switch>`, quoted `url('#id')`, `em` / `ex` / `rem` lengths, `set_size` / `has_view_box`, and `<text>` / `<tspan>` (a chart's labels): glyph outlines of the card's fonts (`image/onyx_vgfont.c`: a CSS family list to Liberation, Gelasio, Selawik or DejaVu, as `font_freetype.c` maps Chrome's; shared with the canvas' text), `font-size`, `font-weight`, `font-style`, `x` / `y` / `dx` / `dy`, `text-anchor`, filled and stroked as shapes.

- Built for the Pi by `user/netsurf/Makefile` (`libplutovg.a`, `libplutosvg.a`, committed: PlutoVG without the font directory scan, `stb_image` for PNG / JPEG only) and on the PC by `host.mk`. Pages: `tools/tests/netsurf/pages/svg-img.html` (`<img>` at three sizes, a `viewBox`-only icon, tiled and sized backgrounds, a `data`: SVG, list markers, `<object>`, `<embed>`), `svg-chart.svg` (`<text>`: anchors, a `tspan`, entities), `svg-inline.html` (a sprite's symbols through `<use>`, `currentColor` from the link's colour, a gradient from the sprite, a `<style>`, `clip-path`, `var()`, an `<svg>` made by `innerHTML`), each against Chromium (`chrome.sh`).

13. Canvas 2D (on PlutoVG)

`<canvas>` was a replaced box drawing nothing and `getContext` answered null.

- `quickjs/canvas.js` (compiled in as `qjs_canvas.js.h` by both makefiles, as `dom.js`; run by `qjs_canvas_setup` right after `dom.js`) is the API and keeps its state (the styles, the font, the save stack): `HTMLCanvasElement.getContext('2d')`, `width` / `height` (a change clears the canvas and its state), `toDataURL` / `toBlob` (PNG); `CanvasRenderingContext2D` -- `fillRect` / `strokeRect` / `clearRect`, `paths` (`moveTo`, `lineTo`, `quadraticCurveTo`, `bezierCurveTo`, `arc`, `arcTo`, `ellipse`, `rect`, `roundRect` (one radius), `closePath`), `fill` (non-zero, even-odd), `stroke`, `clip`, `isPointInPath` / `isPointInStroke`, `fillStyle` / `strokeStyle` (CSS colours given back as Chrome does: `#rrggbb` or `rgba()`), `createLinearGradient` / `createRadialGradient` / `createConicGradient` (its middle colour: PlutoVG has no conic gradient), `createPattern`, `lineWidth`, `lineCap`, `lineJoin`, `miterLimit`, `setLineDash` / `getLineDash` / `lineDashOffset`, `globalAlpha`, `globalCompositeOperation` (the Porter-Duff ones and `copy`; the blend modes are kept but drawn source-over), `save` / `restore`, `translate` / `rotate` / `scale` / `transform` / `setTransform` / `resetTransform` / `getTransform` (a 2D `DOMMatrix`, defined if the page has none), `drawImage` (3, 5 and 9 arguments), `createImageData` / `getImageData` / `putImageData`, `fillText` / `strokeText` (`textAlign`, `textBaseline`, `maxWidth`) and `measureText` (a `TextMetrics` with its bounding boxes); shadows, filters and `imageSmoothing*` are kept, not drawn. `Path2D` (the path methods, `addPath`, SVG path data), `ImageData`, `OffscreenCanvas` (`getContext`, `convertToBlob`, `transferToImageBitmap`), `createImageBitmap`.
- `quickjs/qjs_canvas.c`, the natives (`N.cv*`, added to `dom.js`' natives): each draws on the canvas' PlutoVG surface (premultiplied ARGB, anti-aliased) -- thin, one native per call. The path is kept in device space, each point through the transform of the moment it is added (the HTML canvas' rule); a fill draws it with the identity (the paint in the current user space), a stroke maps it back through the current transform (the line width and dashes scale with it). Fonts: the card's (`SD:/res/fonts`: a CSS family to Liberation, DejaVu, Gelasio or Selawik, as `font_freetype.c` maps them: `image/onyx_vgfont.c`, shared with SVG `<text>`), read once by PlutoVG's `stb_truetype`. `drawImage` / `createPattern` take a canvas (its surface), an `<img>` NetSurf fetched (its content's bitmap, converted once and kept: 8 images), an SVG image, or an `Image` a script made and never put in the page: `canvas.js` completes `HTMLImageElement` -- its `src` fetches it through the high-level cache (`cvLoadImage`), then `load` / `error`, `complete`, `naturalWidth`, `width`.
- **The picture**: the element's node carries a NetSurf bitmap as its user data (`__ns_key_canvas_node_data`, which `redraw.c` already plots into the canvas' box, scaled to its CSS

size); after drawing, the surface is copied into it (straight alpha, the client's layout) once a main-loop turn -- a scheduled flush -- and the box redrawn (`html__redraw_a_box`): a script drawing a thousand shapes costs one copy and one redraw.

- `qjs.c` (Onyx blocks): the setup call, `qjs_node_of` / `qjs_html_of` / `qjs_invoke` for `qjs_canvas.c`, and `qjs_canvas_context_gone` before a document's context is freed (its canvases stop flushing, its image loads are released). A canvas is at most 4096 x 4096.
- `tools/tests/netsurf/jstest.sh` runs `pages/canvas-api.html` (40 checks: the state, the colours' text, pixels after fills / clears / alpha / transforms / gradients, paths, Path2D, image data, text metrics, `toDataURL`, an OffscreenCanvas, a pattern, the reset on `width`, `drawImage` of the page's SVG `<img>` and of an `Image` loaded by the script); `pages/canvas-draw.html` is drawn against Chromium.

14. CSS by the specifications' grammars, a real CSSOM, SVG's properties (css3test.com: 23% -> 81%)

css3test.com asks, for 6419 tests over 1262 features, whether the browser *recognizes* a property, a value, a selector, an at-rule, a descriptor, a media query or a CSSOM interface. NetSurf now recognizes what the specifications define -- and checks it as a browser does: an invalid value is still dropped.

- **The value grammars** (`src/parse/onyx_grammar*.c`, `onyx_grammar_gen.py`): the grammar of every CSS property and at-rule descriptor ("Value Definition Syntax"), from the W3C's webref consolidation of the specifications (`third_party/webref-css-8.7.5`, MIT; `css-grammar.json` trimmed from the npm package by `trim.py`), is compiled by `onyx_grammar_gen.py` into one node graph (`onyx_grammar_tables.c`, committed: 816 properties, 48 descriptors, ~70 KB of tables). The types are shared nodes; the numeric and token types (`<length>`, `<number>`, `<custom-ident>`, `<url>`...) are the matcher's primitives. The matcher (`onyx_grammar.c`) is a backtracking matcher in continuation-passing style: juxtaposition, `&&`, `||`, `[ ]`, `*`, `+`, `?`, `{A,B}`, `#`, `!` exactly as the syntax defines them, the omissible commas of CSS Values 2.6, a step bound (a pathological value is refused, not a hang). The numeric primitives take the math functions, type-checked (a length is not an angle; `px * px` is not a value). A function left open at the end of a value is closed there (CSS Syntax), a relative colour's channel keywords are numbers (`rgb(from x r g b)`), `calc-size()`'s `size` a length. The generator's `FUNC_FIXES` / `PROP_FIXES` fill webref's gaps (`circle()` takes a percentage, `anchor()` in the logical insets, `anchor-center` in the `*-items`, the engines' `x-self-start` / `anchors-visible` names, `url-set`).
- **How libcss uses them** (`parseProperty`, `language.c`): a property libcss computes is parsed by its own parser as before; if that parser refuses the value, or the property is one libcss does not compute (`anchor-name`, `scroll-snap-type`, `mask-mode`, `text-wrap`, the `transition-*` / `animation-*` longhands...), the value is checked against the property's grammar: valid, it is kept as `CSS_ONYX_OP_GENERIC` (one bytecode word, the grammar's index; its cascade does nothing -- an earlier declaration libcss computes keeps applying), invalid, it is dropped. A CSS-wide keyword is valid everywhere, a value with `var()`, `env()`, `attr()` or `if()` too (substitution functions). `@supports` asks the same thing (`css__onyx_declaration_valid`), `selector()` really parses. So `CSS.supports`, `element.style` and `@supports` answer as a browser does -- a page's feature detection now takes its modern branch where the property exists in the specifications (NetSurf may not draw it: that is the price of an honest "recognized").
- **At-rules kept without effect** (`onyx_atrules.c`; `@keyframes` now animates: §22): `@keyframes`, `@counter-style`, `@property`, `@font-feature-values` (and its `@styleset`...), `@font-palette-values`, `@position-try`, `@view-transition`, `@scope`, `@starting-style`: their prelude checked, kept as a

media rule that never matches (`css_rule_media.onyx_kind`), their descriptors checked against the descriptor grammars and counted (`css_stylesheet.onyx_desc_words`, read by `css_stylesheet.onyx_kept`); `@font-face`'s and `@page`'s descriptors libcss does not read (`font-display`, `size-adjust`, `size`, `marks`...) checked the same way.

- **Selectors 4** (`parsePseudo`, `onyx_parse_extra_pseudo`; matching in `select.c`): the pseudo-classes and pseudo-elements of Selectors 4, CSS Pseudo 4 and the other specifications are parsed and checked, their arguments too (`:has()`'s relative selectors, `::part()`, `::view-transition-*`'s `<pt-name-selector>`...). Matched when libcss can say: `:nth-child(A+B of S)`, `:read-only / :read-write`, `:required / :optional`, `:placeholder-shown`, `:defined`, `:scope`, `:dir(ltr)`; the others (`:has()`, `:host()`, `:invalid`..., every pseudo-element) never match -- a list `a:has(b)`, `c` keeps its `c` (the whole rule was dropped). Also `*` namespaces and the `[a=b i] / s` flags; the common `::-webkit-scrollbar`... pseudo-elements (pages hide scrollbars in lists with them).
- **Units and math functions computed** (`parse/properties/utils.c`): `svh / lvh / dvh` (and `w`, `i`, `b`, `min`, `max`) are the viewport's units; `cqw`... the small viewport's (the spec's fallback: containers are not modelled); `cap`, `ic`, `rex`, `rch`, `rcap`, `ric`, `rlh` from `em / rem` with libcss's ratios; `x` is `dppx`. `round()`, `mod()`, `rem()`, `abs()`, `sign()`, `sin()`... `atan2()`, `pow()`, `sqrt()`, `hypot()`, `log()`, `exp()`, `e`, `pi`, `infinity` are folded when parsed (numbers, angles, or lengths of one unit), alone or inside `calc()`. `height: 100dvh` now sizes a box.
- **SVG's presentation properties** (`parse/properties/onyx_svg.c`, `select/properties/onyx_svg.c`): `fill`, `stroke` (`none`, a colour, `currentColor` kept as such, `url()` and its fallback, `context-fill / -stroke`), `stroke-width`, `stroke-dashoffset`, `stroke-miterlimit`, `stroke-dasharray` (a canonical text), `fill-rule`, `stroke-linecap`, `stroke-linejoin` (inherited), `stop-color`, `stop-opacity` are computed (`select_config.py`; the autogenerated headers made again; `onyx_propbits.py` now makes `onyx_propbits.h`) and read by `css_computed_fill()`... (`computed.h`), so a page's `.icon { fill: currentColor }` reaches an inline `<svg>`. An SVG element's presentation attributes (`fill="#f00"`, `stroke-width="3"`...) are NetSurf hints (`css/hints.c`, `css_hint_svg`): the lowest author precedence, as in browsers (a colour attribute in `rgb()` is not read: `nscss_parse_colour`'s forms only). `getComputedStyle` answers `fill`, `stroke`, `stroke-width`.
- **A real CSSOM** (`dom.js`, "the CSSOM" block): `CSSRule` (its type constants) and its classes as globals -- `CSSStyleRule`, `CSSMediaRule`, `CSSSupportsRule`, `CSSContainerRule`, `CSSLayerBlockRule` / `StatementRule`, `CSSScopeRule`, `CSSStartingStyleRule`, `CSSImportRule`, `CSSNamespaceRule`, `CSSFontFaceRule`, `CSSPageRule` / `MarginRule`, `CSSKeyframesRule` / `KeyframeRule`, `CSSCounterStyleRule`, `CSSPropertyRule`, `CSSFontFeatureValuesRule`, `CSSFontPaletteValuesRule`, `CSSPositionTryRule`, `CSSViewTransitionRule`, `CSSNestedDeclarations`, `CSSGroupingRule` / `CSSConditionRule` -- `StyleSheet` / `CSSStyleSheet` (constructible; `replace / replaceSync`, `insertRule / deleteRule / addRule / removeRule`), `StyleSheetList`, `CSSRuleList`, `MediaList`; `rule.style` a `CSSStyleDeclaration` over the rule's valid declarations (or descriptors). A sheet's rules are read from its text when first asked for, as libcss keeps them (each rule, declaration and descriptor asked of `N.cssKept`); a change is written back to the text -- a `<style>`'s: NetSurf styles the page again. `document.styleSheets` holds the `<link>` sheets too (`qjs.c` `sheetText`: the loaded sheet's text and URL; `html/css.c` keeps a `<link>`'s node with its sheet); `document.adoptedStyleSheets` realizes each constructed sheet as a `<style data-onyx-adopted>` at the end of the `<head>`. (google.com's script stopped at `x instanceof CSSStyleRule`.)
- **Two bugs found on the way**: `libparserutils`' own charset filter lost the characters its codec had decoded but the pivot had no room for when the input was all read -- the last character of a 65-byte inline style (a `:`: the declaration dropped), possibly a sheet's last bytes (`src/input/filter.c`); and the input stream reported the end while the filter still held converted data (`inputstream.c`,

`parserutils__filter_pending`). libcss's parser now closes a function or bracket left open at the end of the input (`parse.c`: the declaration is kept, as CSS Syntax says).

- **The score** (`tools/tests/netsurf/css3test.sh [netsurf|chrome|both]`): fetches the site's sources -- git branch v1 of `github.com/LeaVerou/css3test`, what `css3test.com` serves -- into `$OUT/css3test`, runs them in NetSurf and in Chromium, prints the score and a line per spec and per failed test): **81%** (5150 of 6419 tests; 23% before); Chromium 139 headless 71% on the same copy. NetSurf passes 1245 tests Chromium fails -- features of drafts no browser ships (css-speech, css-borders-4's corner shapes, fill-stroke-3...) whose grammar is in the specifications -- and fails 471 Chromium passes (the Typed OM's interfaces, Web Animations, CSSOM View: scripts' APIs, not parsing). Counting only what Chromium also passes, NetSurf scores 64% (Chromium 71%).
- **Cost**: parsing a big sheet takes 0.3% (bulma, 763 KB), 2.7% (bootstrap, 281 KB) to 8% (material-components-web, 624 KB: many properties libcss does not compute, now checked and kept) more instructions (`csscheck -b`, `callgrind`); libcss.a's code +114 KB on the Pi (the tables ~70 KB). `kotonviolins.com` and `kotonstudio.com` draw pixel-identical.
- **Tests**: `make -f tools/tests/netsurf/host.mk css-check` (`csscheck.c` on `css-values.txt`: 157 declarations, sheets and descriptors kept or dropped; `csscheck -b N sheet.css`: a parse-speed bench); `jstest.sh: js-cssom.html` (26 CSSOM checks), `css-svgprops.html`, `css-selectors.html`, `css-math.html`.
- **Since** (\$23): 83 % on the live site; the Pi's run no longer cut off by the script time limit; libcss evaluates most media features; `:popover-open / :modal`.
- **Not done**: the Typed OM (`CSSStyleValue`, `attributeStyleMap`...), Web Animations, CSSOM View's interfaces; `:has()` matching (libcss's handler has no child walk); `@container` against a real container; the pseudo-elements' drawing (`::marker`, `::placeholder`...); a math function over relative lengths (`round(1em, 3px)`) computed.

15. Intl (ECMA-402) and the locale built-ins

QuickJS-ng is built without `Intl`: `bbc.co.uk`'s and `bbc.com`'s Next.js applications stopped on "Intl is not defined" (their React tree crashed, the page went blank), `youtube.com` too. NetSurf now has its own implementation, in JavaScript:

- **quickjs/intl.js**: `Intl.DateTimeFormat` (`dateStyle` / `timeStyle`, every component option, `hour12` / `hourCycle`, `timeZoneName` in its six forms, `format`, `formatToParts`, `formatRange`, `formatRangeToParts`, `resolvedOptions`), `Intl.NumberFormat` (decimal, percent, currency with symbol / narrow symbol / code / name and accounting, unit with the simple units and their `-per-` compounds; integer, fraction and significant digits, the nine rounding modes, rounding increments and priorities, `trailingZeroDisplay`; grouping; standard, scientific, engineering, compact short and long notations; `signDisplay`; strings and `BigInt`s exactly; `formatRange` with CLDR's collapsing), `Intl.PluralRules` (cardinal and ordinal, `selectRange`), `Intl.RelativeTimeFormat`, `Intl.ListFormat`, `Intl.Collator` (a simplified Unicode collation: base letters with accents folded at the primary level, accents, case; `numeric`, `caseFirst`, `sensitivity`, `ignorePunctuation`; Spanish ñ and the Nordic letters after z), `Intl.Segmenter` (graphemes with the Unicode properties of QuickJS's regular expressions; words and sentences approximated, Chinese / Japanese / Thai a character at a time), `Intl.DisplayNames` (languages, regions, scripts, currencies, calendars, date fields), `Intl.Locale` (with `maximize` / `minimize` from CLDR's likely subtags), `Intl.getCanonicalLocales`, `Intl.supportedValuesOf`. Language tags are parsed and

canonicalised as UTS 35 says (aliases, extensions); the legacy constructor behaviour (`Intl.NumberFormat.call (obj)`) is there.

- **The built-ins** take their ECMA-402 versions: `Number.prototype.toLocaleString`, `BigInt.prototype.toLocaleString`, `Date.prototype.toLocaleString` / `toLocaleDateString` / `toLocaleTimeString`, `Array.prototype.toLocaleString` and the typed arrays', `String.prototype.localeCompare`, `toLocaleLowerCase` / `toLocaleUpperCase` (Turkish, Azeri, Lithuanian). The formatter of the calls without arguments is kept (a table of numbers formats fast).
- **The locales:** English (US, GB, AU, CA, IN, IE, NZ, ZA; the other English regions as GB), French (FR, CA, BE, CH), German (DE, AT, CH), Spanish (ES, MX, US, 419, AR; the other Latin-American regions as 419), Italian, Dutch (NL, BE), Portuguese (BR, PT; the other regions as PT), and, for formatting only (their units, display names and time zone names are English's): Japanese, Chinese (simplified, traditional), Korean, Russian, Polish, Swedish, Danish, Norwegian Bokmål, Finnish, Turkish, Czech. Any region of these languages is accepted (`fr-LU`). Another language resolves to the default locale, as the specification says.
- **The default locale is `navigator.language`** (dom.js: `fr-FR`), as in Chrome for a French user. A page that formats without a locale gets French, as it would in Chrome in France.
- **The data** (`third_party/cldr-48/intl-data.txt`, 450 KB, Unicode License v3) is CLDR 48 as ICU 78 has it: `tools/tests/netsurf/intl/gendata.js` reads every pattern and name back from Node.js's own `Intl` (full ICU, Chrome's data) by formatting probe values -- the date patterns of ~160 option combinations per locale, the number templates, the compact and unit patterns per plural category, the relative-time and list patterns, the display names, the time zone names; a regional locale keeps only what differs from its language. CLDR's narrow no-break space in times is a plain space, as Chrome shows it. The plural rules are code (`PLURAL`).
- **Time zones:** every IANA zone (418, and the common links) with its standard offset and its current daylight-saving rule (EU, US, Australian, New Zealand, Chilean, or the zone's own: Cairo, Jerusalem, Havana...) found by the generator from the 2026-2028 transitions -- historical changes are not kept (a 1990 date gets today's rule). Offset time zones (`+05:30`), `Etc/GMT±N`; a zone name is kept as written (case-normalised). **The default time zone:** the host's name (the PC: `TZ`, `/etc/localtime`) when the `Date` agrees with it, else the zone of the locale's region whose offset matches `Date`'s (`fr-FR`: Europe/Paris), else a popular one, else `Etc/GMT±N`: `resolvedOptions().timeZone` always agrees with `getTimezoneOffset ()`.
- **Onyx's clock:** `gettimeofday` gives the local time (the kernel's clock is UTC + the `timezone=` of `SD:/etc/system.ini`), so `Date.now ()` was off by that offset and `getTimezoneOffset ()` was 0. `quickjs.c` (under `__ONYX__`, `README.onyx`) now takes `js_onyx_utc_offset_min` off `Date.now` and answers it in `getTimezoneOffset`; `qjs_intl.h` reads `timezone=` once. Onyx has no daylight-saving rule of its own: the offset is the one the Setup wrote (a date in the other season is still shown at today's offset by `Date`; `Intl` shows it with the zone's rule).
- **Lazily loaded, compiled once:** `intl.js` has two parts, split at the line `///<@@INTL-IMPL@@` by `qjs_intl.h` (included by `qjs.c`; `qjs_intl_init (ctx)` before dom.js). The boot part runs in every document's context: the `Intl` object, whose members are accessors that load the implementation at their first use and then become ordinary data properties, and the built-ins. Both parts are compiled once per process and kept as bytecode (`JS_WriteObject`, the process's heap), read back in each context. The locale data stays a C string in the binary: a record (`"en-GB/d"`) is found and `JSON.parsed` when a formatter first needs it. Cost on the PC (`tools/tests/netsurf/intl/qjsintl-m`): a context 0.12 ms, + 0.09 ms for the boot; the first `Intl` use in a context 2 ms (12 ms the very first time: compiling). The binary grows by ~575 KB (the data and `intl.js` as strings).

- **Tests:** `tools/tests/netsurf/pages/js-intl.html` (made by `intl/mkpage.js` from `intl/cases.js`: 233 expressions with Node's answers as the expected values; in `jstest.sh`), `intl/compare.sh` (the same cases in Node and in `qjsintl`, QuickJS with the Intl on the PC: `intl/build.sh`), `intl/test262.js` (test262's intl402 in `qjsintl`: 1061 of the 1146 tests run pass, 92.6%; Temporal, DurationFormat and a few features are skipped). Live: `bbc.co.uk`, `bbc.com` and `youtube.com` no longer stop on Intl.

16. The HTML parser (libhubbub, libdom's binding, libparserutils)

NetSurf's parser, hubbub, followed the HTML5 drafts of about 2008: no `<template>`, a different `<select>` and `<table>` handling, the character references of that time, no fragment parsing in a context element. It now follows the current standard (the WHATWG tokenizer and tree construction), measured by the **html5lib-tests** (the test suite the browsers' parsers share; `tools/tests/netsurf/html5lib.sh`):

	Before	After
Tokenizer (tokenizer/*.test)	6739 / 7032 (95.8%)	7024 / 7028 (99.9%)
Tree construction (tree-construction/*.dat, documents and fragments, scripting on and off)	930 / 1716 (54.2%)	1792 / 1792 (100%)

(The old count missed the tests with a NUL in them: 1716 of the 1792. The 4 tokenizer tests left start with a U+FEFF that libparserutils' decoder takes off as a byte order mark, as a browser does for a document; `xmlViolation.test` is for XML-style parsers and is skipped. The tree tests also pass fed in chunks of 1, 2, 3, 7 and 64 bytes, and under valgrind.)

- **The tokenizer** (`libhubbub/src/tokeniser/tokeniser.c`) is written again from the standard's state machine: every state, the character reference states with the standard's **2231 named references** (`onyx_entities.inc`, generated by `build/make-onyx-entities.py` from the standard's `entities.json`; searched by bisection -- the old trie, `entities.c`, and its Perl generator are gone), the numeric references' replacement table, CR / CRLF made LF, the script data escape states, CDATA sections in foreign content. The text runs between markup are handed on without a copy (a table of the bytes that stop each text state). New options: `HUBBUB_TOKENISER_LAST_START_TAG` (an appropriate end tag in RCDATA / RAWTEXT / script data when a fragment starts in such an element) and the content models `SCRIPTDATA` and `CDATA_SECTION` (`include/hubbub/types.h`).
- **The tree builder** (`src/treebuilder/treebuilder.c`, one file: the old per-mode files, `element-type.*`, `internal.h`, `modes.h` are gone) has every insertion mode of the standard: the stack of open elements and the list of active formatting elements (with the Noah's ark clause), the adoption agency algorithm, foster parenting, `<template>` (its own insertion modes; its children go to the template's contents fragment), foreign content (SVG and MathML, their attribute and tag name adjustments, the integration points), the "in select" handling of the current standard (the relaxed `<select>` parser: other elements allowed in a select) with `<selectedcontent>` (a copy of the selected option's contents), frameset, quirks mode from the doctype, and **fragment parsing** in a context element (`innerHTML`, `createContextualFragment...`). Consecutive characters are gathered and inserted as one text node (`insert_text`, not one call per character).

- **The API** (`include/hubbub/tree.h`, `parser.h`; appended, the old fields kept): the tree handler's `template_content` (a template's contents fragment) and `insert_text` (append to the preceding text node); the parser options `HUBBUB_PARSER_LAST_START_TAG` and `HUBBUB_PARSER_FRAGMENT_CONTEXT` (the context element: namespace, name, node, form element, whether it is an HTML integration point, the quirks mode); `hubbub_treebuilder_flush` (the text gathered so far inserted -- `parser.c` calls it at the end of each chunk and when a script pauses the parser).
- **libdom's binding** (`libdom/bindings/hubbub/parser.c`, its copy in `libdom/include/dom/bindings/hubbub/`): `add_attributes` adds only the missing ones (the standard's `<html>` / `<body>` merge); the templates' contents are a document fragment held as the element's user data (`dom_hubbub_template_content`); `insert_text`; `dom_hubbub_fragment_parser_create_ctx` (a fragment parser in a context element: `qjs.c`'s `setHTML`); the elements made as the parser names them (`_dom_html_document_create_element_parser`: no prefix split of `xlink:href`-like names; attribute names that are not XML names -- the standard allows them -- are kept, `_dom_element_parser_attrs`).
- **libdom**: an element in another namespace than HTML keeps its name's case (`viewBox`, `foreignObject`: they were made upper case) and has no HTML type (only `<style>` keeps its type: its sheet); attribute names are lower-cased on HTML elements only; the document element and the doctype can be removed (`node.c`).
- **libparserutils**: the input filter emptied its decoder after a full output buffer (the last character of a 65-byte input was lost: `filter.c`, `inputstream.c`); `U+FFFE` / `U+FFFF` are valid UTF-8 (`utf8impl.h`).
- **Speed** (the PC, `html5lib.sh time <page>`, best of 20, fed in 32 KB chunks as the fetcher gives them): `bbc.co.uk` (668 KB) 15.7 -> 10.7 ms, `bbc.com/news` 8.1 -> 6.2 ms, a Wikipedia article 24.1 -> 17.5 ms, `google.com` 3.0 -> 2.2 ms.
- `html/box_construct.c`: the text of a closed `<details>` other than its `<summary>` makes no boxes (`onyx_in_closed_details`).
- The PC tests (`tools/tests/netsurf/`): `html5lib.sh [tree|tok|time <page>] [-v] [files]` (the tests are cloned once, at the last `html5lib-tests` commit that has the tree-construction tests); the drivers `html5lib_tree.c` (-c N: fed N bytes at a time), `html5lib_tok.c` + `html5lib_tok.py`, `html5lib_time.c`; `html5lib.mk` builds them.

17. The HTML5 DOM (`quickjs/html5.js`)

`html5.js` is a second prelude, run after `dom.js` (compiled in the same way: `qjs_html5_js.h`); `qjs.c` calls it with the natives and `dom.js`'s table of element classes (`TAGS`). It extends and corrects `dom.js`'s classes, and lists what it does in its header. Measured by **html5test.co** (Niels Leenheer's test, `tools/tests/netsurf/html5test.sh`: its engine run in the PC NetSurf, the score and every missing feature printed): **252 -> 314 of 588** (Chrome about 530). Every feature it counts works; nothing answers "supported" without doing it. `dom.js` changed in two places only: `N.internals` (its mutation queue, observers, dispatch, for `html5.js`) and its selector engine asking `N.internals.pseudo` for the pseudo-classes it does not know.

- **Namespaces**: `namespaceURI`, `localName`, `tagName`, `prefix`, the `*NS` methods; SVG and MathML elements get their own classes (`qjs.c`'s `qjs_proto_for` looks up `svg:<name>`, `svg:*`, `math:*`), an unknown HTML name is an `HTMLUnknownElement`, a valid custom element name an `HTMLElement` (`*custom`); attribute names lower-cased on HTML elements only.

- **Fragments on the new parser:** `innerHTML` (qjs.c's `setHTML`: parsed in its context element -- a `<tr>` into a `<tbody>`, text into a `<textarea>` or `<title>` decoded, a `<script>` or `<style>` raw), `outerHTML`, `insertAdjacentHTML`, `setHTMLUnsafe`, `getHTML`, `Range.createContextualFragment`; the standard's serialization (escaping, void elements, raw text); `<template>`'s content (cloned and imported with the template); `DOMParser` (HTML, and XML with a small parser: `parsererror` on bad XML), `XMLSerializer`, `Document.parseHTMLUnsafe`, `document.implementation.createHTMLDocument / createDocument`; `ownerDocument`, `importNode`, `adoptNode`, `cloneNode` across documents (qjs.c: `createDocument`, `createIn`, `importTo`, `parseDocument`).
- **Messaging:** `MessageChannel / MessagePort` (`port1 / port2`, `postMessage`, `onmessage`, `start`, `close`), `window.postMessage` with a `MessageEvent` (`origin`, `source`), `BroadcastChannel` -- delivered in a task, not a microtask -- and `structuredClone` (the structured clone algorithm: cycles, `Map`, `Set`, `Date`, `RegExp`, typed arrays, errors; a `DataCloneError` for functions and nodes).
- **Forms:** the input types (email, url, tel, search, number, range, date, month, week, time, datetime-local, color) with the standard's value sanitization, `valueAsNumber`, `valueAsDate`, `stepUp / stepDown`; the constraint validation API (`validity` with every flag, `willValidate`, `checkValidity`, `reportValidity`, `setCustomValidity`, `validationMessage`, the `invalid` event); an invalid form is not sent (a capture listener on `submit`; `novalidate`, `formnovalidate`); the `form` attribute; `<output>`, `<fieldset>` elements, `<datalist>`, `<meter>`, `<progress>`; the pseudo-classes `:valid`, `:invalid`, `:in-range`, `:out-of-range`, `:read-write`, `:read-only`, `:indeterminate`, `:default`, `:open`, `:modal`, `:defined`.
- **<dialog>, <details>, hidden:** `show`, `showModal`, `close`, `returnValue`, a `<form method="dialog">` closing its dialog, the `close / cancel / toggle` events; `<details>` open with its `toggle` event; the global attributes (`hidden`, `translate`, `accessKey`, `contentEditable`, `isContentEditable`, `draggable`, `spellcheck`; the `popover` attribute reflected, no `popover` behaviour). The drawing: `default.css`'s block marked "Onyx HTML5" (`[hidden]`, `<template>`, `<datalist>` not drawn; a closed `<dialog>` not drawn, an open one centred; a closed `<details>` shows its summary only); `getComputedStyle` of an element without a box (`display: none`, a hidden subtree, a script's reading before the first layout) is its selected style (qjs.c: `qjs_unboxed_style`, with the style sheets already loaded -- `html_css_new_selection_context` skips those still coming); a script's reading of a size lays out its changes first (`qjs_layout_now`, 16 times a turn at most).
- **Custom elements:** `customElements.define / get / getName / whenDefined / upgrade`; the element's constructor runs on `createElement` and `new`, and on the elements already there once they are in the document (an upgrade); `connectedCallback / disconnectedCallback` on insertion and removal, `attributeChangedCallback` for the observed attributes; `attachInternals` (`ElementInternals`: a value, a validity). The parser's elements: libdom's `DOMNodeInserted` calls `js_handle_new_element`, which keeps the element (the DOM is read-only during a mutation event) for `html5.js`' hook (`N.ceHook`), run before the next script or from the scheduler; the scripts' insertions are seen by `html5.js` itself (it wraps the natives `insert`, `remove`, `setHTML`, `setText`). Customized built-in elements (`{ extends }`) are recorded but not upgraded, as in Safari.
- **The session history:** `history.pushState / replaceState` change the document's URL (qjs.c's `setURL`: `location` and the address bar show it; no navigation), `back / forward / go` between the entries the page made fire `popstate` (and `hashchange`), past them NetSurf's own history; `location.hash` set makes an entry and scrolls.
- **Streams:** `ReadableStream` (its reader, a BYOB reader -- a byte stream reads as a default one and the BYOB reader copies --, `tee`, `pipeTo`, `pipeThrough`, async iteration, `ReadableStream.from`),

WritableStream, TransformStream, the queuing strategies, TextEncoderStream / TextDecoderStream, Response.body and Blob.stream().

- **Files:** Blob holds bytes (size, type, slice, text, arrayBuffer, bytes, stream), File, FileReader (text, data URL, array buffer, binary string, its events), FileList, URL.createObjectURL / revokeObjectURL (fetch reads blob: URLs).
- **Smaller APIs:** microdata (itemScope, itemProp, itemValue, properties, document.getItems), <ol reversed> (NetSurf's layout already numbers it backwards), performance.mark / measure / getEntries* and PerformanceObserver, XMLHttpRequest's responseType = 'document' and responseXML.
- tools/tests/netsurf/pages/js-html5.html, js-forms.html, js-apis.html, js-ce.html (in jstest.sh): 44 + 45 + 26 + 17 checks. html5test.sh starts the run at the load, as html5test.co does (it waits for its browser detection script).
- Not done (html5test counts them): IndexedDB, the editing APIs (designMode, execCommand) (shadow DOM: §20), <input type="image">'s sizes, and what is out of this work: canvas, SVG, audio and video, WebRTC, WebGL.

18. The big sites: memory, requests, events, early layout (bbc.co.uk, google.com, m.facebook.com)

Found on the live sites with the PC bench, which now reaches the network (https through the PC's OpenSSL, an HTTPS proxy's CONNECT, JPEG and WebP decoded: site.sh draws a live site in NetSurf and in Chromium, layoutdiff.sh compares their boxes, sitesweep.sh loads a list of sites and counts crashes and script errors, NS_MEMSTAT prints the heap in use, NS_JSDUMP writes the scripts that failed, NS_INJECT + F5 runs a script in the page).

- **bbc.co.uk's "out of memory":** its consent script bundles core-js, which replaced the engine's Promise because PromiseRejectionEvent was missing, and its polyfill queued microtasks for ever -- 1.5 GB in a minute on the PC. PromiseRejectionEvent exists; the promises' jobs run 200 ms at most per turn and the rest a turn later (a runaway chain no longer holds the window, qjs_leave); the scripts' heap is limited to 384 MB (an allocation past it throws in the script instead of stopping the app). bbc.com now takes ~170 MB on the PC, steady.
- **The requests' Fetch Metadata** (onyx_fetch.c, onyx_add_fetch_metadata): Chrome's Sec-Fetch-Site / -Mode / -Dest / -User, Upgrade-Insecure-Requests, an Accept by destination and, on https, the User-Agent client hints (sec-ch-ua, -mobile, -platform). m.facebook.com answered a navigation without Sec-Fetch-Mode with a 400 ("Sorry, something went wrong"). The destination comes from what the caller accepts (hlcache.c: an X-Onyx-Dest header the fetcher takes off; web fonts and script requests say theirs).
- **Load and error events** of <script src> (run or failed, inserted ones too), <link> / <style> sheets and (NetSurf's picture); the window's load waits for the fetches still going (an inserted script delays it) and an element's load does not reach the window (duckduckgo.com's capturing window listener re-added itself at each image: a loop). .complete / naturalWidth from NetSurf's picture (N.image); a script's own images (new Image()) are canvas.js's (fetched, decoded, load / error). Facebook's bootloader waits for its modules' load events.
- **A script inserted by a script** runs outside libdom's mutation-event guard (libdom refused every change it made: yahoo.com's loaders' "bad attribute"; dom_document_onyx_mutation_guard). innerHTML of a raw text element (script, style...) is its text, of textarea / title its decoded text (a consent stub inserted as script.innerHTML lost every <...>).

- **Form controls outside any form** are kept for the document's life (`orphan_controls`): a rebox found them no more -- what the user typed in a React app's input (Facebook's login) was lost at each script change.
- **Early layout** (`html_early_layout`): a script that asks for a geometry while the document is still parsed gets boxes built and laid out from the nodes parsed so far (no object fetched), made again when the DOM changed, given up at the conversion. google.com's footer script set `#gb-main`'s min-height from its `scrollHeight` (0 without boxes): the footer covered the search box.
Script-blocking style sheets: a parser-inserted inline script waits (the parser paused) while a style sheet -- a `<style>` still to be converted too -- is fetched, as a browser's `DOMContentLoaded` (`html_script_sheets_arrived`).
- **A style attribute** is the author's with the specificity (1,0,0,0) (`libcss_select.c`): it kept the last sheet's origin and selector's specificity, and lost to an `<img width>`'s presentational hint after the user sheet (bbc's images kept their attribute size).
- **URL / URLSearchParams** after the WHATWG URL Standard (`dom.js`: the basic URL parser's state machine, IPv4 / IPv6 / IDN hosts, the percent-encode sets, every setter; the WPT's `urltestdata` 896/896, setters 278/278, toascii 72/87 -- `urltest.sh`).
- **Dynamic import()** of a URL computed at run time: rewritten at compile time to `__onyxImport(base, x)` (`qjs_rewrite_import`: strings, comments and methods named `import` skipped), which fetches the module graph then calls the engine's `import` (QuickJS's loader cannot wait for the network: developer.mozilla.org's chunks).
- **SVG made by scripts** (React): `createElementNS` creates the element in its namespace, the prototypes are chosen by namespace, the names keep their case (`N.name`).
- **Web fonts** of the sheets added after the conversion (Facebook's bootloader adds its sheets: Optimistic 95 was never fetched), `format("woff2-variations")` read as WOFF2 (`libcss`), and the **CSS Font Loading API**: `FontFace` (a URL or bytes) and `document.fonts` (a `FontFaceSet`: `add`, `ready`, `load`, `loading` / `loadingdone`), a script's faces given to NetSurf's font code (`N.addFontFace` -> `onyx_webfont_add_script_face`) and the page laid out again.
- **Style sheets after the layout:** a `<style>` or `<link>` a script adds, changes or takes out once the page is laid out now restyles it -- NetSurf ignored them ("NS layout is static"): the selection context is made again from the sheets there now and the boxes built again (`html_css_restyle`, `css.c`; `html_css_node_removed` from `DOMNodeRemoved`, `dom_event.c`). Facebook's Bloks screens (`.wbloks_1 { display: flex }`, a `<style>` added by the script) were laid out as blocks. The UA sheet hides `link`, `meta`, `base`, `area`, `param` too (a `<link>` in the body made a line). Test: `js-latesheets`.
- **The body's overflow is the viewport's** (CSS Overflow 3, 3.3; `libcss_select.c`): when the root's overflow is visible, the body's computed overflow is made visible (NetSurf's window scrolls) -- `body { height: 100%; overflow-y: scroll }` (en.wikipedia.org's Minerva) was a scroller of the window's height inside the window, `body { overflow: hidden }` cut the page. (`getComputedStyle` shows the used value, visible.) Test: `css-bodyoverflow`.
- **The box tree is built in slices of 15 ms**, not of 10 elements (`box_construct.c`): the scheduler runs the next slice at the main loop's next turn only, and a 5000-element page (Wikipedia) took hundreds of turns -- the throbber turned for seconds, the early boxes shown.
- **The connects wait asleep** (`onyx_fetch.c`): the downloads' connects are made one at a time and the other workers spun on `kapi_lock` meanwhile (a connect: DNS + TCP, 100 ms and more) -- on bbc.com 60 % of the process's CPU; they sleep 5 ms at a time now (the same run: 4080 -> 1335 ms of CPU). Found with the bench's sampling profiler: `NS_PROF=<file>` (`host_stubs.c`, `SIGPROF` + `backtrace`) then `sh tools/tests/netsurf/prof.sh <file>` (self and total per function).

- **A style sheet given in parts** (libparserutils `inputstream.c`): the HTML parser work's "the filter still holds data" refill ran in the middle of a stream too, left the cursor past the buffer's end and the CSS lexer read and wrote beyond it -- a sheet that arrived in several network reads (more of them on the Pi's slower link) could come out broken. Only at the end of the input now (found by the network work, on github.com's Brotli-decoded sheets).
- **"Desktop Site / Mobile Site"** (the Navigate menu): the User-Agent is a mobile Chrome's by default (the light pages of the big sites); a site the user switches gets a desktop Chrome's (Windows) -- in its requests, its client hints (`sec-ch-ua-mobile`, `sec-ch-ua-platform`) and its scripts' `navigator.userAgent / platform` -- and the page is loaded again. The sites are kept on the card (SD:/apps/jet.app/desktop-sites, one registrable domain a line: m.facebook.com and www.facebook.com are one site; `user_agent_for_host`, `utils/useragent.c`). The disk cache keys a desktop site's objects apart (`D|<url>`, `onyx_cache.c`): a site switched back to mobile found its desktop copies. (Since §35: the toolbar's pill, three versions, `site-modes`.)
- **Media queries' range syntax** (libcss `src/parse/mq.c`, an upstream bug): with the name first (`((width >= 1012px))`) the stored value was the name itself and the operator was negated instead of having its sides swapped -- the query never matched: GitHub's Primer showed its mobile header (a hamburger and the menu open, with a scroll bar) on a 1080p screen; a ratio after the operator was read from the operator's token. Test: `css-mqrange`.
- **content_broadcast told each user once per broadcast with the list of users told kept by that broadcast** (`content.c`): the shadow DOM work's mark in each user was a global generation, overwritten by a broadcast a callback makes (`DONE -> a reformat -> ...`): the outer broadcast told everyone again, for ever -- an `<iframe src="about:blank">` hung the page. Test: `js-iframeblank`.
- **Two crashes of the sweep fixed**: a subtree a script takes out of the document forgets its boxes (`html_box_unlink_subtree` from `DOMNodeRemoved`) -- the rebox unlinked only the nodes in the document, and `getBoundingClientRect` on a removed element walked a freed box (`lemonde.fr`); a canvas `Image`'s callback is taken off before it is freed at the page's teardown (`qjs_canvas.c`) -- the closure held its image, whose finalizer freed it again (`yahoo.com`). `performance.measure` takes `PerformanceTiming` names (`"navigationStart"`: `bbc`).
- **A fragment let go of once the user scrolls** (`desktop/browser_window.c` `frag_scroll`, NetSurf's own "@todo don't do this if the user has scrolled"): each reformat scrolled back to the URL's `#fragment`, so a page still reflowing (late images, scripts, animations) could not be scrolled up from it (`kotonstudio.com`'s menu links). The offset the fragment scroll left is kept; a different one later (the user, a script) ends it, until the next navigation. Test: `gputest.sh` 3b (`nav-fragment.html#target` scrolled up while it reflows).
- **A context's prelude compiled once**: `dom.js`, `html5.js` and `canvas.js` are compiled by the first context of the process, their bytecode kept and read back in the next ones (`qjs_eval_cached`, as Intl's): on the PC a context's prelude went from ~37 ms to ~5 ms (a page's iframes, each navigation). Timed as `js:prelude` (NS_PERF).
- **google.com's mobile search** (the page the Android User-Agent gets): a tap in the search box opens a full-screen overlay (the URL's `#sbfbu=1&pi=`: the field's container made `position: fixed; inset: 0; overflow: auto`, `html / body` fixed). NetSurf showed the back arrow, the mic and the lens over a blank panel with a scroll bar of its own: typing went nowhere, no suggestions, and the back arrow opened the page's menu under the overlay. Four causes, each general:
 - **The focus moved on the release** (`interaction.c`, `default_mouse_action` and `default_mouse_action_focus`): any mouse action outside a text field -- the button's release (`CLICK_1`), a drag, a hold -- took the caret out of the field. The field takes the focus at the press; Google's focus handler moves it to the top of the overlay, so the release

landed on the overlay's blank area and the keys went to the page. Only a press moves the focus now, as in the browsers (the focus is mousedown's default action). In `dom.js` the focus moves after the mousedown handlers, not before, and not when one prevents it (an autocomplete list's items keep the field focused), and a press on nothing focusable blurs the focused element (Chrome's default action).

- **Nodes moved between documents** (`libdom dom_document_onyx_adopt`, `qjs.c n_insert / N.adopt`, `html5.js adoptNode`): Closure's HTML sanitizer builds the suggestions' markup with the page's document, appends it to a document of its own (`createHTMLDocument`) and serializes that; libdom refused a node of another document (`WRONG_DOCUMENT_ERR`, "cannot insert that node here (4)"); every suggestion's text came out empty. An insertion into another document's tree adopts the node first, as the DOM says -- in place: the node, its subtree and their attributes change owner, the document's list of nodes pending deletion follows (libdom's own `adopt_node` makes copies: a script would have kept the old node). `document.adoptNode` does the same (it copied).
- **The scroll extent left the fixed boxes in** (`box.h scroll_ext_x1 / _y1`, `layout.c layout_calculate_scroll_extent`): a box's scroll bars, its `scrollWidth / scrollHeight` and the page's size came from its descendants' bounds, `position: fixed` ones included -- hidden fixed panels Google keeps below the viewport gave the overlay a scroll bar. The descendants' bounds less the fixed subtrees, computed only for the boxes that hold a fixed box (`HAS_FIXED`; the others' are their bounds).
- **The hit test missed a fixed box in an overflow: hidden ancestor** (`layout.c layout_calculate_descendant_bboxes`): the overlay sits in `form#tsf { overflow: hidden }`, whose descendants' bounds left out its children -- the fixed overlay too, so the hit test (which follows those bounds to the fixed boxes) never reached it and the click went to the header beneath. A fixed child, or one holding one, is kept in the bounds. An anonymous box (an inline container) of a `visibility: hidden` element no longer takes the click (`onyx_hit_visible`), and `document.elementFromPoint` asks the same hit test (`N.hitNode`; it took the last element in document order whose box held the point, hidden or covered). Also `getComputedStyle`'s z-index (it gave libcss's fixed-point value: 989 read 1012736) and `overflow / overflow-x / overflow-y` (always `visible`). With them the field takes the typing, `/complete/s` is asked at each key and its suggestions listed (with their pictures), the arrow closes the overlay and Enter runs the search (`/search?q=`; this container's IP gets Google's captcha). Test: `pages/js-searchoverlay.html` (jstest). A field a script focuses while an attribute-only change is pending (restyled in place, no rebox: \$26) takes the caret after the restyle too (`html_rebox_scheduled`).
- **A document a few bytes past 4 KB failed to load: "BadParameter"** (`libparserutils inputstream.c`, `parserutils_inputstream_refill_buffer`): the input filter decodes the raw bytes 64 characters at a time into its pivot and writes them to the 4096-byte UTF-8 buffer; a document of 4097 to ~4160 bytes left its last characters in the pivot with no raw bytes left. The refill at the end (the earlier "filter still holds data" fix) wrote them, then discarded zero bytes from the empty raw buffer -- which libparserutils answers with BADPARM: the parse failed, the page was an error page (a style sheet of that size too). Nothing is discarded when nothing was read. Test: `pages/js-size4k.html` (4142 bytes).
- Smaller: `DOMStringMap`; `addEventListener` & co called unbound are the window's; `localStorage`'s Proxy keeps the Proxy invariants (`Object.keys(localStorage)` threw); inline scripts are named by their first characters in errors and timings.

- Tests: `jstest.sh` gained `js-microloop`, `js-rawtext`, `js-loadevents`, `js-url`, `js-dynimport`, `js-svgns`, `js-fontface`, `js-searchoverlay`, `js-size4k`.

19. The scripts' real-time and background APIs (WebSocket, EventSource, streams, Workers)

What chats, notifications and live pages need (Facebook's chat is MQTT over a WebSocket): **WebSocket**, **EventSource**, the **streamed fetch / XHR** and **Web Workers**. The C side is `user/netsurf/onyx_ws.c` (the connections) and `quickjs/qjs_net.c` (the natives); the API is `quickjs/net.js`, a third prelude run after `html5.js` (compiled in as `qjs_net_js.h`, as `canvas.js` is). `html5test.co`: +31 points (338 -> 369 of 588: `eventSource`, `websocket.basic`, `websocket.binary`, `worker`, `sharedWorker`).

- **The connections** (`onyx_ws.c`): a WebSocket or an event stream is a long-lived socket, outside the fetch queue and the low-level cache -- it takes no download slot (8 at once), keeps no body in memory (llcache keeps every byte of a response), and has no idle timeout. Each runs in a **thread** of its own (kernel v67), as the downloads do: the DNS and the connect (in the downloads' turn: `onyx_fetch_connect`, "the connects one at a time"), the TLS handshake (`onyx_nstls`), the HTTP handshake, then the frames. The thread posts (`kapi_post`) when an event is ready; on the UI thread a scheduler callback gives the events to the scripts. Messages to send wait in a queue; the thread sleeps between reads -- 1 ms after some traffic, doubling to 50 ms (a socket's `recv` does not block on Onyx) -- and `kapi_wake_word` wakes it at once when a message is queued. No NetSurf call in the thread: the UI thread builds the request's headers (the Origin, the jar's cookies -- `urldb` is the UI thread's --, the User-Agent, Accept-Language, the subprotocols, Last-Event-ID) and gives the handshake's `Set-Cookie` to the jar.
- **WebSocket** (RFC 6455, `ws`: and `wss`): the opening handshake (`Sec-WebSocket-Key`, the `Sec-WebSocket-Accept` checked with SHA-1, the subprotocol chosen among those asked, `permessage-deflate` offered); client frames masked (the key from `kapi_random`); the messages reassembled from their fragments, pings answered with pongs, control frames between fragments; **permessage-deflate** inflated with `zlib` (the context kept between messages unless `server_no_context_takeover`; the client's messages are sent uncompressed, which the extension allows); text checked as UTF-8 (1007), a masked server frame, reserved bits, an unknown opcode, a bad close code fail the connection (1002), a message past 64 MB closes it (1009); the closing handshake both ways (the peer's close answered with its code; ours waits 5 s for the answer). `net.js`: `WebSocket` (`url`, `readyState`, `protocol`, `extensions`, `binaryType` `blob / arraybuffer`, `bufferedAmount` -- the bytes queued in C and not yet sent --, `send` of a string, an `ArrayBuffer`, a view or a `Blob`, `close`(`code`, `reason`) with its checks, the `open / message / error / close` events, `CloseEvent` with `code`, `reason`, `wasClean`); a failure before the open is `error` then `close` 1006.
- **EventSource**: the stream (`Accept: text/event-stream`, redirects followed, chunked undone) is given in parts cut after a line's end; `net.js` parses it as the standard says (`data` lines joined, `event`, `id` -- the last event ID kept and sent back as `Last-Event-ID` on a reconnection --, `retry`, comments, a CRLF cut between two parts, the BOM), dispatches `message` and named events (`MessageEvent` with `lastEventId`, `origin`), reconnects after `retry` ms (default 3 s) when the stream ends or fails, and fails for good (CLOSED, no reconnection) on a status other than 200 or another content type; `withCredentials` (cross-origin: the cookies only with it).
- **The streamed response** (`qjs.c`'s `request`): a request may give its head and body as they come -- `pcb(0, head)` at `LLCACHE_EVENT_HAD_HEADERS`, `pcb(1, ArrayBuffer)` for each part once the script reads the body as a stream (`N.requestMode`: the bytes come so far first), `pcb(2, bytes)` for XHR's progress; an abort from inside the callback waits until it returns (the llcache handle is in its

callback). `fetch` resolves its `Response` when the head is in; `Response.body` is a real `ReadableStream` fed as the bytes arrive (`getReader`, `tee`, `pipeTo...`); `text()` / `json()` / `arrayBuffer()` / `blob()` wait for the whole body (nothing crosses to JavaScript before, unless the stream is read); `clone()` tees; an `AbortSignal` errors the stream; a `ReadableStream` request body is read whole, then sent. `blob()` keeps a binary body's bytes. `XMLHttpRequest: HEADERS_RECEIVED` (its status and headers) when the head is in, `LOADING` and a `progress` event (the bytes come so far, `total` from `Content-Length` when there is no `Content-Encoding`) at each part, `responseText` while it loads (`N.requestSoFar`), the `upload` events. A script's request (`fetch` / `XHR`) waits 5 min for its answer, not 30 s: a long poll holds its request till something happens.

- **TLS reads no longer wait** (`onyx_nstls.cpp`): after the handshake, a read that finds nothing returns at once (mbedtls goes on with a record read in part at the next call). `onyx_tls.hpp`'s BIO waited up to 20 s and then reported the connection reset: a `WebSocket` or a long poll quiet for 20 s was cut, and its thread could not send while it waited. The callers poll with their own idle rules (the fetcher's 30 s / 5 min).
- **Workers:** a `Worker` is a QuickJS **context of its own** in the window's runtime, run on the UI thread by NetSurf's scheduler (its timers and messages are tasks, cooperative as the page's): a `jsthread` without a document (`qjs.c`'s `qjs_worker_create`: its URL is its location and the base of its requests), with the same preludes (`Intl`, `dom.js`, `html5.js` -- a stand-in document while `html5.js` sets up), then `net.js` makes its global a worker's: the DOM, `window`, `document`, the storages and the element classes removed; `self`, `WorkerGlobalScope` / `DedicatedWorkerGlobalScope` (the global's prototype), `name`, `postMessage`, `close`, `onmessage` / `onmessageerror` / `onerror`, `importScripts`, and `fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, timers, `Intl`, `crypto`, `TextEncoder`, `Blob`, streams from the page's preludes. The page fetches the script (`blob:` and `data:` URLs too), then the context is made and the script runs a task later; the `importScripts` with literal URLs are fetched first, in parallel; another is read when it runs (`file:`, or a blocking `http(s) GET`: `onyx_http_get_sync`). **Module workers** (`{ type: 'module' }`): the module and what it imports, fetched as QuickJS asks (`N.moduleRun`). A message is written by QuickJS's object serializer (`JS_WriteObject2`: objects, arrays, typed arrays and buffers, `Map`, `Set`, `Date`, `RegExp`, `BigInt`, cycles) in the sender's context and read into the receiver's (`JS_ReadObject`) -- the structured clone between realms; `net.js` wraps what the serializer does not know (`Blob`, `File`, `Error`, `ImageData`) and refuses a function, a symbol or a node (`DataCloneError`); transferred `ArrayBuffers` are detached. An exception in the worker's script or message listeners is the worker's `error` event, then the `Worker`'s (`ErrorEvent` with its message, file, line). `terminate()` ends it at once (its messages dropped); `close()` from inside ends it once the task is done (the messages it sent still arrive).
- **SharedWorker:** shared by the `SharedWorkers` of the document with the same URL and name (one window: no other documents to share with); each gets a `port` (`postMessage`, `onmessage`, `start`, `close`); the worker's `onconnect` gets the port in `ev.ports[0]`.
- **BroadcastChannel:** `html5.js`' channels (within a context) also reach the page's other contexts -- the page and all its workers, and theirs (`N.broadcast`).
- **The end:** when the document goes (`js_closethread`, the context's free: `qjs_net_stop`), its sockets and streams are dropped (their threads stop, they free what is left), its workers ended (and theirs), the messages still queued freed; a callback running then finishes first (the records are freed after it). The JS values are freed with the runtime once a context may be gone.
- Tests: `tools/tests/netsurf/nettest.sh` (a local server, `wssrv.py`, Python's standard library: `WebSocket` with permessage-deflate and fragments, an event stream with a reconnection, a chunked body; `pages/net-ws.html`, `net-stream.html`, `net-teardown.html`: 54 checks), `pages/net-worker.html` in `jstest.sh` (20 checks: the worker's scope, `importScripts` fetched first and read

when asked, timers, fetch, the structured clone of `Map` / `Set` / `Date` / `RegExp` / typed arrays / `Blob` / errors / `BigInt` / cycles, errors, `BroadcastChannel`, `close`, a `blob: worker`, a module worker, a `SharedWorker`); `pages/net-live.html` against public servers (`wss://echo.websocket.org`, Wikimedia's event stream: its events and a streamed fetch of it). Valgrind: no invalid access in these pages.

20. Shadow DOM (web components: Lit, Stencil, reddit's shreddit)

`attachShadow` returned the host, whose light children were drawn; `ShadowRoot` did not exist (reddit.com's scripts stopped on "ShadowRoot is not defined"). NetSurf now has real shadow trees: the DOM of them, the parser's declarative ones, the boxes built from the flat tree, and the styles scoped to each tree.

- **Where a shadow root lives** (libdom's hubbub binding, `bindings/hubbub/parser.c`, its copy in `include/dom/bindings/hubbub/`): a document fragment the host keeps as user data (a reference; the host kept on the fragment, its mode and options too), made by `dom_onyx_attach_shadow` -- the DOM standard's rules for the host (a valid custom element name or one of `article ... span`; one root per host). It is no child of the host, so the host's `children`, `innerHTML`, the document's `querySelector`, `getElementById` and the document's own walks never see it; `dom_onyx_shadow_root` / `dom_onyx_shadow_host` / `dom_onyx_shadow_flags` link them, `dom_onyx_has_shadow(doc)` says whether a document has any (NetSurf does nothing of this otherwise).
- **Declarative shadow roots** (libhubbub `treebuilder.c`, `insert_shadow_template`; the tree handler's new `attach_shadow`): a `<template shadowrootmode="open|closed">` in the document parser (not in `innerHTML`) is pushed but not inserted, its contents are a shadow root attached to the element it is in (`shadowrootdelegatesfocus`, `-clonable`, `-serializable` kept); a host that cannot take one gets the template as any other. The `html5lib-tests` stay at 100%.
- **The DOM** (`quickjs/html5.js`, "shadow DOM"; `qjs.c`'s natives `attachShadow`, `shadowRoot`, `shadowHost`, `shadowFlags`, `hasShadow`, `shadowProto`, `shadowSheets`): `Element.attachShadow` (open / closed, `delegatesFocus`, `clonable`, `serializable`, `slotAssignment`; a declarative root handed to its custom element emptied; the errors of the standard), `element.shadowRoot` (open ones), `ShadowRoot` (a `DocumentFragment`: `mode`, `host`, `innerHTML` parsed in the host's context, `setHTMLUnsafe`, `getHTML`, `activeElement`, `styleSheets`, `adoptedStyleSheets`, `getElementById`, `querySelector...`), `getHTML({ serializableShadowRoots, shadowRoots })` writing declarative roots, `getRootNode({ composed })`, `isConnected` through the hosts, `ElementInternals.shadowRoot`; `<slot>` (`HTMLSlotElement`: `name`, `assignedNodes` / `assignedElements` with `flatten`, `assign()` for the manual mode), `element.slot`, `assignedSlot`, `part` (a `DOMTokenList`), `slotchange` (the slots whose assigned nodes changed, at the next microtask); custom elements in a shadow tree upgraded and connected through their host.
- **Events** (`dom.js`' dispatch asks `html5.js` once the document has shadow roots: `N.internals.shadowHook`): the path goes from a slotted node to its slot, from a shadow root to its host (a non-composed event stops at its target's shadow root); the target and `relatedTarget` are retargeted at each node (a listener outside sees the host); `composedPath()` hides the closed trees from outside and is empty after the dispatch; the browser's own events are composed (clicks, keys, focus, input, the pointer's); `document.activeElement` is the host of a focused shadow element; `delegatesFocus`.
- **The boxes from the flat tree** (`html/onyx_shadow.c`, `box_construct.c`): when the document has shadow roots, the box tree's walk asks for a node's first child, next sibling and parent in the flat tree -- a host's children are its shadow root's, a slot's are the nodes assigned to it (else its own:

the fallback), a light child assigned to no slot is not drawn. Slot assignment is the standard's named one, worked out per host once per box tree. The parent box, the containing block and the inherited style follow the flat tree (`b_parent`), so do `getComputedStyle` of an unboxed element and the CSS `:hover` chain (through the hosts). Hit testing and the events need nothing more: the boxes point at the shadow tree's nodes. `html_rebox_unlink` clears the shadow trees' box links too.

- **display: contents** (all documents; it was a block): the element makes no box, its children are its parent's; its style is kept in a box out of the tree (freed with the box tree) for its children to inherit (`onyx_contents_keep` / `onyx_contents_style`, `box_extract_properties`). A `<slot>` is `display: contents` by default (its style's inline, in a document with shadow roots). Not for replaced elements and form controls; `::before` / `::after` of such an element are not drawn.
- **Style scoping** (`libcss select.c`, `css_select_style_onyx`; NetSurf `css/select.c`, `onyx_shadow.c`): each shadow tree has its own selection context -- the user agent's and the user's sheets, then its `<style>` elements' (in tree order, their `media`) and its adopted sheets' (`adoptedStyleSheets`: the constructed sheets' texts given to C, again at each `replaceSync` / `insertRule`), each text parsed once for the document's life. The document's author sheets never match inside a shadow tree, and a shadow tree's `<style>` is not the document's (`html_css_new_selection_context` leaves it out; `document.styleSheets` does not list it). An element's style is selected in its own tree's context, with:
 - `:host`, `:host(<compound>)`, `:host-context(<compound>)` -- the host is featureless in its shadow tree (only these match it; its parent is none there): a second handler (`onyx_scoped_handler`: the document's own pages keep the plain one) answers no for its name, classes, attributes and states and stops the ancestor walks at it, and gives the host's own tree's context for the arguments (`onyx_node_is_scope_host`, `onyx_host_pw`: handler functions appended to `css_select_handler`); the host's style takes its shadow tree's `:host` rules;
 - `::slotted(<compound>)` -- an element assigned to a slot takes the slot's tree's `::slotted()` rules whose selector matches the slot (and the argument the element), and those of the slots that slot is assigned to (flattened);
 - `::part(<ident>+)` -- an element of a shadow tree with a `part` attribute takes the `::part()` rules of the tree around its host whose selector matches the host;
 - the cascade's **encapsulation contexts** (CSS Cascade 4): each of these other trees' rules has a context level (`prop_state.level`); between two declarations of the same origin and importance, the outer context's normal one wins and the inner one's `!important` (a document rule beats a `:host` rule whatever their specificities; presentational hints stay under every author rule). The other tree's rules are matched on their node (the host, the slot) with a bloom filter that rejects nothing, without the reject cache;
 - a host, an element assigned to a slot or with a part takes no other element's style (`CSS_NODE_FLAGS_ONYX_NO_SHARE`), and inherits its custom properties from its flat tree parent (the slot).
- **Cost**: a document without shadow roots takes none of these paths (a flag per box tree, the selection's plain handler; `css_select_style` is `css_select_style_onyx` without scopes); the flat tree's lookups (a hash of the hosts' assignments, the trees' contexts) are made once per box tree. Measured with callgrind on a saved `en.wikipedia.org` article (no shadow roots, 3400 elements styled): box building costs ~1-2% more instructions per styled element (libcss's selection behind the new entry point, the contexts' comparison in the cascade); a page with shadow roots pays for

its trees' selection contexts (made per box tree), the other trees' passes of the hosts and slotted elements, and no style sharing for them.

- **Tests:** `jstest.sh -- js-shadow.html` (58 checks: the API, the encapsulation, slots, declarative roots, events, custom elements in shadow trees, `:host` / `:host()` / `:host-context()` / `::slotted()` / `::part()` / adopted sheets / the contexts' cascade, the boxes of the flat tree), `js-shadow-click.html` (clicks on a shadow tree's element and on a slotted one through the simulator, `:hover` inside a shadow tree); `pages/js-shadow-render.html`, a Lit-like page (one constructed sheet per component class, named / default slots with fallback, `::slotted`, `::part`, a flex host, nested components, a declarative root, `display: contents`) against Chromium with `layoutdiff.sh` (which now walks the open shadow roots too): the boxes agree but for NetSurf's general line-height and whitespace differences. **web-platform-tests:** `wpt.sh` (new) runs WPT's testharness pages in the PC NetSurf (a sparse clone, served on 127.0.0.1, a `testharnessreport.js` that logs each subtest; the testdriver ones and the reftests skipped): `shadow-dom/` 999 of 1666 subtests pass (60.0%; before: `ShadowRoot` undefined, next to none), with the build before the `TreeWalker` and `DOMException` fixes. The misses: the testdriver-less focus and selection tests, manual slot assignment's rendering, `elementFromPoint` across trees, the imperative slot API's corner cases, `:host` in `matches()`. One page crashed: a shadow root's `innerHTML` whose markup a fragment could not take (`n_set_html` unreferenced a node twice when an append failed) -- fixed.
- **reddit.com** (shreddit, Lit components): "ShadowRoot is not defined" gone; its JS challenge now passes and its components render (their shadow trees, slots, adopted sheets). Found on the way: `TreeWalker` was a snapshot of the root's nodes and ignored a `currentNode` set outside the root -- Lit's template preparation does exactly that (the template's contents walked from a walker on the document): its slots kept their `name$lit$` marker names. `TreeWalker` and `NodeIterator` now walk live as the DOM standard says. With its feed now built, three crashes of the core showed and are fixed: a script navigating while the page was converted (the JS challenge) -- the old content's READY laid out the new, contentless one (`browser_window_callback` ignores a message from a content no longer loading); an image's `load` event whose script asked for a layout, the rebox releasing other users of the same image while `content_broadcast` held the next one (each user now told once, the list walked again from its head); a node the new box tree no longer boxes (removed, unslotted) keeping its freed box (`html_rebox_unlink_boxes` clears the links of every node the old tree boxed). `sitesweep.sh`: no crash, reddit's 3 errors gone. Left for reddit: its feed is laid out at `x = -2147483648` -- the children of its `display: grid` container (`.grid-container.grid-full`, named grid lines) get no `x` from `layout_grid.c` (not a shadow DOM matter: the grid layout).
- Also: `DOMException` has its legacy `code` and constants (`NOT_SUPPORTED_ERR...`).
- Not done: the manual slot assignment (`assign()`) is the DOM's only (the boxes use the named assignment); `exportparts`; a clonable shadow root copied by `cloneNode`; `<link rel=stylesheet>` and `@import` in a shadow tree; `@font-face` in a shadow tree's sheets; `:host` in `querySelector` / `matches`; the focus navigation order of shadow trees; `::before` / `::after` of a `display: contents` element.

21. Compositing layers: `opacity`, `transform`, `filter`, `backdrop-filter`, `mix-blend-mode`

NetSurf drew a translucent colour, and a transform's translation (the layout moved the box); a subtree's `opacity`, a rotation, a scale, a skew, the filters and the blend modes were parsed and not drawn. A box with one of them is now painted as a **group** -- a stacking context, with all it holds -- through a layer of the framebuffer.

- **libcss** (src/parse/properties/onyx_css3b.c, the compositing block): `filter` and `backdrop-filter` (-webkit-backdrop-filter by the prefix rule), `transform-origin` and `mix-blend-mode` are computed, kept as canonical texts (`select_config.py`, the autogenerated headers and `onyx_propbits.h` made again; `css_computed_filter()`... in `computed.h`): a filter's functions (`blur(4px)` `drop-shadow(2px,2px,3px, #ff000000)`, a percentage as a number, `currentColor` kept as such, a `url()` refused -- the declaration then kept by its grammar only, §14), the origin as two lengths / percentages (the keywords in either order), the blend mode's keyword. They are paint properties for the hover's restyle (`css_computed_style_paint_only_change`). The 3D functions are **flattened** (no perspective, as Chrome draws a 3D transform without `perspective`): `rotateX(a)` / `rotateY(a)` a scale by `cos a` across the axis, `rotate3d()` and `matrix3d()` their 2D part, `perspective()` and the z parts dropped; `rotate: x 30deg` likewise.
- **The effects of a box** (`content/handlers/html/onyx_fx.c`): its opacity, its matrix -- `translate`, `rotate`, `scale` then `transform`, composed, about its `transform-origin` (the border box's; 50% 50%) -- its filters, its backdrop filters, its blend mode. A transform that composes to a translation only stays **the layout's** (`onyx_box_translate`: the box moved as a relative offset is, no layer: today's fast path); any other is painted.
- **The stacking context** (`html_redraw_layer_z`): a static box with an effect (a translation too, as in CSS) is put off and painted as a positioned box with `z-index: 0`, and its own positioned descendants are painted inside it (`onyx_fx_paint_context`). The hit test follows (it uses the same function).
- **The group** (`redraw.c`, `onyx_fx_redraw`; the plotter's `onyx_layer_begin` / `onyx_layer_end`, `struct onyx_layer` in `netsurf/onyx_paint.h`; the knockout plotter flushes around them):
 - *an opacity alone* is drawn **in place**: what is under the group's rectangle (its bounds -- border box, descendants, shadow -- within the redraw's clip) is copied, the group is painted where it is, then the copy is blended back at `1 - opacity`. This is exact (the group over the page, mixed with the page at `1 - a`, is the group at `a` over the page) and costs one pass, a copy and a blend of its rectangle; nested opacities nest.
 - *a transform, a filter, a blend mode* paint the group **apart**, into a RAM surface the size of the part of it that lands in the clip (the clip mapped back through the inverse matrix, grown by what the filters read), twice: over black, then over white. No plotter keeps an alpha channel, but every one blends linearly: a pixel's coverage is `'1 - (white black)'`, its premultiplied colour the black pass -- exact. The layer is then filtered and drawn through its matrix (each target pixel's centre mapped back, bilinear; a scale's columns and rows tabulated), at its opacity, with its blend mode.
 - **One pass** when the box is known opaque in its (rounded) border box and clear outside it: an opaque background colour under its borders, nothing past the border box, no shadow, no mask (`onyx_fx_opaque`) -- its coverage is the rounded box's (a rotated card).
 - **A large blur** (a standard deviation of 8 px and more) paints the layer at 1/2, 1/4 or 1/8 of its size (the core's scaled redraw, as a thumbnail's), its filters scaled, drawn back through the matrix: what it leaves is smooth. kotonstudio.com's hero glow (a 900x700 radial gradient under `blur(20px)`): 30 ms -> 7 ms a full redraw on the PC.
 - The layer's surfaces and work buffers are kept, a set for each nesting level (8 levels; an isolated group within 3 others, or in a scaled redraw, is drawn without its transform and filters). A layer larger than 8 M pixels is painted without its effects.
 - **Opacity 0** paints nothing; the box is still hit (as in CSS).

- A **fixed** descendant of a group painted apart is clipped to the layer, not to the viewport (the transformed or filtered box is its containing block's clip, as in CSS).
 - **An inline** with an opacity (a `<span>`: its pieces are its line's siblings) is a group from its box to its end box, in place.
- **The filters** (`frontends/framebuffer/onyx_layer.c`), on the premultiplied layer: `brightness`, `contrast`, `invert` (a table), `grayscale`, `sepia`, `saturate`, `hue-rotate` (Filter Effects' matrices, in sRGB as Chrome), `opacity`; `blur` as three box blurs (the specification's approximation, its box sizes and offsets; a wide one on a reduced copy: a power of 2 that leaves at least 1.5 px, brought back bilinearly); `drop-shadow` the layer's alpha blurred at half its radius, coloured, moved, under it. **backdrop-filter**: what is under the box (and what its blur reads around it, the page's edge repeated) copied, filtered, and put back inside the rounded border box, before the box is painted (in place: faded with the group's opacity, as in CSS). **mix-blend-mode**: the separable modes (`multiply`, `screen`, `overlay`, `darken`, `lighten`, `color-dodge`, `color-burn`, `hard-light`, `soft-light`, `difference`, `exclusion`, `plus-lighter`, `plus-darker`) against what is under the group.
- **Where a transformed box is**: the layout grows its ancestors' descendant bounds with the transformed, filtered bounds (`onyx_fx_child_bounds`: a rotated card is not culled); the hit test (`interaction.c`, `onyx_hit_box`) takes the point back through the inverse matrix for the box and all it holds; `getBoundingClientRect()`, `elementFromPoint` and the `IntersectionObserver` see the transformed box's bounding box, in fractions of a px (`N.rect(n, true)`; `offsetWidth...` stay the layout's).
- **The hover's restyle** (§9): a change of a box's effects (`onyx_fx_style_differs`) redraws all it holds where it was painted and where it is (`hv_request`: the rectangles through the transforms and filters of the box and its ancestors); its ancestors' bounds grow as the layout's. `NS_HOVER_FULL=1` gives the same pixels (`fxtest.sh`).
- **Timings**: `NS_PERF=1` prints each layer of 1 ms and more -- `layer WxH [transform] [filter] [backdrop] [(one pass)], layer WxH in place [backdrop]`.
- **Tests** (`tools/tests/netsurf/`): `fxtest.sh` shoots `pages/css-opacity.html`, `css-transform.html` and `css-filter.html` in NetSurf and in Chromium and compares them cell by cell (38 cells: all within a mean of 0.3 per channel but the drop-shadow's 1.9 and the backdrop blur's 3.5), then hovers `css-fxhover.html` (a card scaled, rotated, faded and shadowed on `:hover`) and compares the partial redraw with the full one (identical); `jstest.sh` checks an opacity-0 box's click, a rotated box's rectangle (23, 23, 113, 113 as Chrome) and clicks through the rotation (a corner outside the diamond reaches the box under it).
- **Cost** (the PC bench, a full redraw of the first screen, 1262x792; noisy): `kotonviolins.com` ~9 -> ~12 ms (its header's `backdrop-filter: blur(14px) saturate(140%)`: 3-5 ms); `kotonstudio.com` ~8 -> ~15 ms (the hero glow's blur 7 ms, the app mock-up's flattened `perspective()` `rotateY()` `rotateX()` with a shadow 5 ms, two small backdrops 1-2 ms). A page without effects pays nothing (a few style reads a box). On the Pi count ~5x.
- **Not done**: a real perspective (the 3D functions flattened); (a layer kept between redraws: §25, GPU compositing -- opacity and transforms); `clip-path` and `mask` on the layer; the non-separable blend modes (`hue`, `saturation`, `color`, `luminosity`: drawn normal); `isolation`; `filter: url()`; `backdrop-filter` of a transformed box (not drawn) or its fade with the group's opacity when the group is apart; a blend mode inside an isolated group (its black / white passes then differ non-linearly); the text caret and a form field's click position inside a transformed box; `will-change`.

22. Transitions, animations, the Web Animations API, animation frames

CSS transitions, CSS animations (@keyframes), the Web Animations API and a paced requestAnimationFrame, on one timeline per HTML content (content/handlers/html/onyx_anim.c).

- **libcss computes the lists** (select_config.py, parse/properties/onyx_css3b.c): the `transition-` and `animation-` longhands -- property, duration, timing function, delay; name, duration, timing function, delay, iteration count, direction, fill mode, play state -- and the `transition` / `animation` shorthands (one item per layer, the omitted ones at their initial values, a keyword taken by the first slot it fits, `none` by the name first) are kept as canonical texts, comma separated: times in seconds (`0.3s, 1s`), timing functions spelled `ease`, `cubic-bezier(a,b,c,d)`, `steps(n, jump-end)`, `linear(0, 0.5 50%, 1)`, the properties' names lowercase, the animations' names as written. `css_computed_transition_duration()`... read them; they count as paint properties (onyx_propbits.py: a hover changing only a transition is still a restyle). `var()` in them works (transition: opacity .6s var(--ease)).
- **@keyframes kept** (language.c, stylesheet.h): the at-rule was already kept without effect (§14); now its name, each keyframe's selectors (from, to, percentages: offsets 0..1) and its declarations (parsed by the properties' own parsers, !important ones dropped as the spec says) are stored in the rule (css_rule_media.onyx_name / onyx_offsets / onyx_style). `css_select_ctx_onyx_keyframes()` gives the last @keyframes <name> of a selection context's sheets (their @imports, @media / @supports / @layer groups), sorted by offset. The lexer takes an at-keyword starting with - (@-webkit-keyframes: it was a stray @ and the rule was lost).
- **The animation API of libcss** (src/select/onyx_anim.c): `css_computed_style_onyx_apply` cascades a keyframe's declarations over a copy of an element's style (as a selection would: `inherit` from the parent, `initial`, the values made absolute), noting which properties it sets; `css_computed_style_onyx_clone` / `_blend` / `_intern` make an animated style -- a copy of the base with each animated property set between two styles' values at a progress: colours in premultiplied RGBA, lengths of one unit (px with px, % with %), numbers (opacity, flex factors), `z-index` rounded, `visibility` (visible while either end is), `box-shadow` (a `none` end a transparent shadow of no size), the SVG paints and opacities, and the transform texts (`transform`, `translate`, `scale`, `rotate`): function by function when both lists have the same kinds (the shorter completed with identity functions, a `none` end too), else through their 2D matrices (decomposed and recomposed as CSS Transforms 1 says), and the filters' lists (`filter`, `backdrop-filter`: the same functions, a `none` end their identities -- `blur(0px)`, `brightness(1)...`); any other value flips at the middle (discrete). `css_computed_style_onyx_same` compares one property of two styles. `css_stylesheet_onyx_inline_decls` gives an inline sheet's declarations (a script's keyframes).
- **The timeline** (onyx_anim.c): the style selection (`box_get_style`: the construction's, a `:hover` restyle's) hands every element's new style to `onyx_anim_styled`. A page with no transition nor animation never creates the timeline (one check of two computed values per element). An element with one gets a record: its base style (the cascade's) and its current one (what its boxes show). When the base changes, each animatable property the `transition-property` list names (a longhand, a shorthand -- `border`, `margin`, `inset`, `border-radius...` -- `all`, a `-webkit-` name) whose value changed starts a transition from the current value (the before-change style: the record's, else the element's box's, else -- the boxes built again -- the style the old boxes had, noted by `onyx_anim_rebox_begin`) to the new one, unless the values cannot be interpolated (then the change is immediate) or the combined duration is not positive; a transition still going to the same value goes on; one going elsewhere is cancelled. A new name in `animation-name` starts a CSS

animation (its keyframes applied over the base; a keyframe's own `animation-timing-function` eases its interval), a name gone cancels it, the others take the new durations, counts, directions, fills and play states (`paused` holds its time). An element no longer rendered (`display: none`, removed: the rebox's sweep) has its animations cancelled. Each animation is an effect with the Web Animations model's timing (delay, duration, iterations, iteration start, direction, fill, end delay, an easing) and player (start time, hold time, playback rate), and its keyframes per property; the implicit 0% / 100% keyframes are the base's values. The composite order: transitions, then CSS animations in the list's order, then the scripts'.

- **Easing functions** (CSS Easing 2): the keywords, `cubic-bezier()` (Newton then bisection, the tangent lines beyond the ends), `steps()` with the four jump positions (the before flag), `linear()` with its stops (the missing inputs spread evenly).
- **The frames:** a frame is scheduled only while an effect runs or a script asked for one (`requestAnimationFrame`), 16 ms after the last (~60 Hz), longer when a frame's work took more than 10 ms (1.5 times it: the events and timers keep a third of the time); nothing runs on a page whose animations are over. (Since §32: ~30 Hz, 15 when the change is a few px or the window is not focused, none while the window is hidden.) A frame makes each record's animated style again (a copy of the base, each animated property blended, interned) and, when it changed, gives it to the element's boxes through the `:hover` restyle's machinery (`onyx_hover_restyle_elements`, `onyx_hover.c`): all the animated elements at once, their pseudo-elements' styles kept, their subtrees styled again only when an inherited property changed (`color`, `font-size`, `visibility`...). When only painting changed (colours, opacity, shadows, radii, a translation -- the box moved as the layout would) the boxes' rectangles are redrawn; else the styles are swapped and the page laid out again (no box tree built); an element this cannot restyle (the root, a pseudo-element appearing) has the boxes built again, 5 times a second at most. The replaced style results no box points at any more are freed every 120 frames (`onyx_hover_collect`, which the hovers' limit of kept results uses too). An element out of the window's view whose animations change only painting (not a transform) is updated 4 times a second, and the frames slow to that when only such ones run. Then the queued events go to the scripts, then the `requestAnimationFrame` callbacks run with the frame's time.
- **The events:** `transitionrun` (when it is made), `transitionstart` (its delay over), `transitionend`, `transitioncancel`; `animationstart`, `animationiteration`, `animationend`, `animationcancel` (`TransitionEvent` / `AnimationEvent`: `propertyName` / `animationName`, `elapsedTime`), bubbling, with their `on*` properties; an `Animation`'s `finish` and `cancel` (`AnimationPlaybackEvent`, `onfinish`, `oncancel`, its `finished` promise resolved / rejected with an `AbortError`). They are dispatched after the frame's styles are given (`js_dispatch_anim_event`, "onyx:anim" in `dom.js`), never while the records are walked.
- **The Web Animations API** (`dom.js`; the natives `N.animate`, `N.animCtl`, `N.animInfo`, `N.animList`): `element.animate(keyframes, options)` -- keyframes as an array or property-indexed (`{ opacity: [0, 1] }`), their offsets spread as the spec says, each keyframe's easing; options as a duration or an object (`duration`, `delay`, `endDelay`, `iterations` including `Infinity`, `iterationStart`, `direction`, `fill`, `easing`, `id`, `playbackRate`) -- makes a script's animation on the same engine (its keyframes parsed as an inline style, applied as a CSS animation's). `Animation`: `play`, `pause`, `cancel`, `finish`, `reverse`, `updatePlaybackRate`, `currentTime`, `startTime`, `playbackRate`, `playState`, `pending`, `finished`, `ready`, `onfinish` / `oncancel`, `commitStyles` (the computed values written into the style attribute), `persist`, `effect` (`KeyframeEffect`: `getKeyframes`, `setKeyframes`, `getTiming`, `getComputedTiming`, `updateTiming`, `target`), `timeline` (`document.timeline`, a `DocumentTimeline`). `document.getAnimations()`, `element.getAnimations()` and a shadow root's

list the transitions and CSS animations too, as `CSSTransition` / `CSSAnimation` (`transitionProperty`, `animationName`), controllable the same way. A script's change is shown before its next read of a style or a geometry (`onyx_anim_flush` in `qjs_layout_now`): `finish()` then `getComputedStyle()` sees the end value. A script's animation that has finished without filling leaves the engine (its `Animation` keeps its state; `play()` makes it again); one that fills forwards replaces the earlier finished filling ones whose properties it covers (their `remove` event, `replaceState` "removed"): a page calling `animate()` at each hover does not pile them up.

- **requestAnimationFrame** (`dom.js`): the callbacks run at the content's frame (`js_animation_frame`, "onyx:frame"), with the frame's time on `performance.now()`'s clock, all those asked for before the frame, once; `cancelAnimationFrame`, `webkitRequestAnimationFrame`. It was a 16 ms timer: now the animations' frames and the scripts' are the same, and none run when nothing asks.
- **getComputedStyle** answers the animated values (the boxes' styles) and more properties: `transform` as a `matrix(...)` (the lengths against the box), `translate` / `scale` / `rotate`, `filter` / `backdrop-filter`, `left` / `top` / `right` / `bottom`, the margins, paddings, border colours and widths, `outline-color`, `letter-spacing`, `z-index`, `box-shadow`, and the transition / animation lists (`transitionDuration`: "0.4s, 0.4s").
- **Measured** (the PC bench, `NS_PERF=1`: `ONYX-PERF anim:frames 120 in <ms>`, `<n> elements: <us>` a frame on average, `<us>` at most; `<n>` laid out again, `<n>` reboxed every 120 frames; `NS_NO_ANIM=1` switches the engine off, to compare): `pages/anim-perf.html` (a spinner, a pulse, a colour cycle, a sliding bar: paint only) runs at 62 frames a second, the animations' work 55 us a frame, the whole frame (with the redraw of the ~600 x 180 rectangle they cover, the spinner's rotation drawn through its layer (§21), and the window's update) ~1.1 ms of CPU (0.72 s of CPU over 9.6 s, 0.07 s without the animations); with a width animation too (`?layout`: the page laid out at each frame) 95 us + ~1.9 ms a frame. `kotonstudio.com` (its local copy: 32 animated elements after the scroll reveals, a pulsing dot running forever): 60-90 us of animation work a frame, ~5 % of a PC core while idle (the dot's redraws); the scroll reveals' transitions are paint-only. On the Pi 4 (Cortex-A72, 5 to 8 times slower on this work): ~0.5 ms of animation work and 5 to 8 ms a frame with its redraw -- 60 frames a second for paint-only animations of a few elements, the adaptive pacing lowering it (to ~40-50) when a layout per frame is needed on a bigger page. `sitesweep.sh`: no crash, the same script errors as before.
- **Tests** (`jstest.sh`): `pages/css-transition.html` (30 checks: opacity, colours, transform, a width laid out, a delay, `all`, a filter, `rotate`, a reversal cancelled, a `:hover` rule's transition, the four events), `pages/css-animation.html` (27: iterations and `alternate`, `forwards`, `paused` then resumed, a negative delay, two animations on one element, `steps()`, a keyframe's own timing function, a missing `@keyframes`, the four events, `getAnimations()`), `pages/js-animate.html` (`element.animate`, the promises and handlers, `pause` / `play` / `currentTime` / `reverse` / `cancel` / `finish()`, fills, `playbackRate`, iterations and direction, `commitStyles`, `requestAnimationFrame`'s pace and times, the replacement of finished filling animations: 31). They sample `getComputedStyle` at known times (real time: `SIM_SLEEP=1`), with margins for the bench's timing. `pages/anim-perf.html` is the frames' measure (above).
- **With the compositing layers** (§21): the animated opacity, transform (rotations, scales), filter values are what the layers draw at each redraw -- the frames give the boxes their styles, the layers paint them.
- **Not done**: animations of pseudo-elements (`::before...`), of `display` and of the other properties that change the box tree; a transition's "reversing shortening" (a transition reversed midway takes its full duration back); `transition-behavior: allow-discrete`; the keyframes of a running CSS animation made again when the base style changes (its implicit ends follow the base, its explicit

keyframes keep the values they had); `composite / iterationComposite` (replace only); `@keyframes` in shadow trees' sheets; scroll-driven animations (`animation-timeline`); `var()` inside keyframes.

23. The feature tests on the Pi: `css3test.com`, `browserscore.dev`; the Popover API; `matchMedia` by `libcss`

What the Pi showed (branch `claude/busy-ramanujan-5enakb`): `css3test.com` "100 %" and `browserscore.dev` "0 %". On the PC bench the live `css3test.com` scored 82 % and `browserscore.dev` 85 % -- the detection itself was right. The cause was the **script time limit**: NetSurf stops a script, an event handler or a promise job after `script_timeout` seconds (10), and the Pi runs QuickJS some five times slower than the PC. `css3test.com`'s test run is one `load` handler (2.8 s on the PC), `browserscore.dev`'s first render one Vue job (9.6 s on the PC); both were cut off on the Pi ("InternalError: interrupted"), leaving `css3test` at its page's "0%" and `browserscore` with no score at all (reproduced on the PC under `valgrind --tool=none`, ~7x slower). `css3test`'s "100 %" is its score with the "CSS 2.2", "CSS 2007" or "CSS 2010" filter (NetSurf passes all of those): the site keeps the filter chosen in `localStorage`, so a filter picked once on the Pi stays. Nothing in NetSurf answers true for everything: `csscheck` linked with the Pi's `libcss.a` and run under `qemu` gives the PC's answers.

- **The time limit** 10 -> 60 s (`desktop/options.h`; `script_timeout` in `SD:/res/Choices`, 0 = none). Browsers do not stop a script at all; the limit only ends a loop that never does (§28: past it, a script still changing the page now runs on).
- **Faster** (the PC; the Pi in proportion): `css3test`'s run 2834 -> 1390 ms, `browserscore`'s render job 9.6 -> 6.6 s.
 - `console.*` formats its arguments only when the log is read (`N.logOn`: `NS_JSDEBUG / jsdebug`, or NetSurf's verbose log), and as bounded JSON (2000 values; no `toJSON()` but a `Date`'s): Vue's development build passes whole component trees to `console.warn` (`browserscore.dev`: 1500 warnings, each its features' `toJSON()` of a whole subtree -- 64 % of its time, a minute with the log on) -- bounded harder since (§28).
 - `querySelector('#id')` from libdom's `getElementById`; `querySelector` walks with `N.nextElement` and stops at the first match (it wrapped every element of the tree first: bliss's `$( '#x' )` thousands of times on `css3test`).
 - `new URL(url, base)`: the parses kept (a copy handed out), a base parsed once, a `"#fragment"` of a base without the parser (`browserscore.dev`: a quarter of its time).
 - `html_rebox (html.c html_rebox_index, object.c)`: the old boxes' objects found by their URL's hash, not by a walk of the whole list for each new box ($O(n^2)$: 23 % of `browserscore.dev`'s time with its thousands of status icons).
- **The bench**: `NS_JSPROF=<file>` samples the scripts (`qjs.c`, from the interrupt handler, at most once a millisecond: the running script's stack) and `tools/tests/netsurf/jsprof.py` sums them up (self / total per function); `NS_PERF` logs each job over 50 ms (`js:job`).
- **Honest answers** found wrong by the new `js-cssdetect.html` (checked against Chromium): `CSS.supports('color', 'red !important')` was true (a value holds no !); `CSS.supports('selector()')` was true; `CSS.supports('color: red')` was false (the spec tries a condition again in parentheses); the end of a condition now closes its blocks, as CSS Syntax does (`selector(:nth-child(even of :not([hidden]))` -- `browserscore.dev`'s test).

- **matchMedia by libcss** (`N.mediaMatch` -> `nscss_media_match`, `css/select.c` -> libcss's new `css_select_onyx_media_match`): the query list parsed and evaluated by libcss for the page's viewport (the window's size until the page is laid out); `.media` is its text with `not all` for a query that does not parse, as a browser serializes it; an unknown feature is kept and false (Media Queries 4's `<general-enclosed>`, as Chrome). The JS matcher before read no range syntax. `MediaQueryList`'s attributes on its prototype, `MediaQueryListEvent`.
- **libcss evaluates the other media features** (`src/select/mq.h`, `mq_onyx_match_feature`): it answered `width`, `height`, `prefers-color-scheme` only. Now `aspect-ratio` / `device-aspect-ratio` (and their `min-` / `max-`), `device-width` / `device-height`, `orientation`, `resolution`, `-webkit-(min-|max-)device-pixel-ratio`, `color`, `color-index`, `monochrome`, `grid`, `hover` / `any-hover` (`hover`), `pointer` / `any-pointer` (`fine`), `prefers-reduced-motion`, `prefers-contrast`, `prefers-reduced-transparency` / `-data` (no-preference), `forced-colors`, `inverted-colors` (`none`), `display-mode` (`browser`), `scripting` (`enabled`), `update` (`fast`), `color-gamut` (`srgb`), `dynamic-range` (`standard`), `overflow-block` / `-inline` (`scroll`); (`width`) as a boolean; and Media Queries 4's three-valued logic: a feature libcss does not know is *unknown*, so `not (bogus)` is false. Style sheets' `@media` rules match by the same code.
- **The Popover API** (`html5.js`; GitHub's `<tool-tip popover="manual">` texts showed on the page): `showPopover` / `hidePopover` / `togglePopover` (`force`, `{ source }`), `beforetoggle` (cancellable when showing) and `toggle` (`ToggleEvent`, a task, coalesced), the auto popovers' stack (showing one hides the others but its ancestors and its invoker's), `popovertarget` / `popovertargetaction` buttons (`popoverTargetElement`, `popoverTargetAction`), the light dismiss (the user's click outside) and Escape (the user's -- a script's events do not dismiss, as in Chrome), `showModal()` hiding the auto popovers, `onbeforetoggle`. The UA sheet (`resources/default.css`) hides `[popover]:not(:popover-open):not(dialog[open])` and gives a showing one Chrome's style (fixed, centred, `z-index` the top layer's stand-in); `dialog:modal` fixed too.
- **:popover-open and :modal in libcss**: `css_select_handler` gained `onyx_node_state` (appended; NULL: the states never match) -- libcss asks the client for a state only the scripts know. NetSurf keeps it on the node (`nscss_node_state_set`: libdom's user data `__ns_onyx_state`, `NSCSS_STATE_POPOVER_OPEN` / `NSCSS_STATE_MODAL`), set by the scripts (`N.setState`), the page laid out again.
- **The IDL attributes on the prototypes** (feature tests look for `'clientX'` in `MouseEvent.prototype`): the events' (`protoFields`: a constructor's value kept in a symbol), `Screen`, `ResizeObserverEntry` / `ResizeObserverSize`, `FontFace` (with `variationSettings`, `ascentOverride...`), `MediaQueryList`; `AnimationEvent`, `TransitionEvent` and their `on*` handlers (NetSurf runs no animation; a script can make and dispatch them), `window.visualViewport` (the layout viewport), `moveTo` / `resizeTo...` (no effect, as for a tab in a browser), `CSSRule.type`, `HTMLImageElement.x` / `.y`; libcss reads `@font-face`'s `font-stretch` (`font-width`'s legacy name).
- **Scores** (the PC bench, the live sites, Chromium 141 headless beside it):

	before	after	Chromium
css3test.com	82 % (5166 / 6419)	83 % (5218 / 6419)	71 % (4375 / 6419)
browserscore.dev	85 % (1261 / 1489 features)	86 % (1274 / 1489)	75 % (1121 / 1489)

NetSurf is above Chromium for the reason §14 gives: libcss parses by the specifications' grammars the properties of drafts no browser ships (css-speech, corner shapes, fill-stroke-3...) -- 1248 css3test tests / 1291 browserscore tests NetSurf passes and Chromium fails; Chromium passes 464 / 390 NetSurf fails (the Typed OM, Web Animations, CSS Values 5's `if()` / `progress()` / `sibling-index()`, CSSOM View's `caretPositionFromPoint()`).

- **Tests:** `jstest.sh: js-popover.html` (22 checks, then the user's click and Escape), `js-cssdetect.html` (24: `CSS.supports`, `element.style`, the CSSOM, `matchMedia` say no to garbage). `libcss-test`, `css-check` pass; `csscheck` with the Pi's `libcss.a` under qemu too.

The media queries follow the window (`html.c html_reformat`): the selection's media size was set once, at the page's load (`html_get_dimensions`) -- a window resized was laid out again at its new width with the load's `@media` answers (a site never switched between its mobile and desktop layouts) and `matchMedia` kept its first answer. Each reformat now reads the window's size (`CONTENT_MSG_GETDIMS`: scroll bars included, as browsers measure the media width -- a scroll bar coming and going never flips a query in a loop); a new size updates the selection's media and the styles are selected again by a rebox (coalesced and throttled; `onxy_restyle.c` drops the kept selections when the media size changes); `matchMedia`'s change events follow. Test: `jstest.sh's css-mqresize.html` (the window maximised across a min-width query).

24. The network as in Chrome: certificates, brotli / zstd, HTTP/2, a disk cache, CORS

The fetcher (`user/netsurf/onxy_fetch.c`), its TLS (`user/netsurf/onxy_nstls.cpp` on `user/tls/onxy_tls.hpp`, mbedTLS 3.6) and NetSurf's cache (`content/llcache.c`) brought near a current browser's. Every change is marked `Onyx`: in the sources.

Certificates checked. The server's chain is verified against the trusted roots of `SD:/res/ca-bundle` (the Mozilla bundle as curl.se publishes it, Sept 2026: 121 roots; read once, parsed at the first TLS connection), its name against the host (SNI sent, the SAN DNS names and IP addresses matched, wildcards as RFC 6125), its dates against the Onyx clock (`kapi_get_datetime`, a day of slack either way; skipped while the clock is not set -- a year before 2025). mbedTLS is built without its own clock (`MBEDTLS_HAVE_TIME_DATE` off): the handshake runs with `VERIFY_OPTIONAL` and a verify callback checks the dates and records each certificate of the chain (its DER and its fault). The EC keys of every curve the bundle uses (secp256r1, secp384r1, secp521r1) are accepted. A refused certificate goes to NetSurf's own flow: `FETCH_CERTS` (the chain, for `about:certificate`) then `FETCH_CERT_ERR` -- the "Privacy error" page with the reason, "View certificate details" and "Proceed" (the host is then accepted for the session: `urldb_set_cert_permissions`; its connections are made without the check and kept apart from the checked ones). `about:certificate` has an mbedTLS implementation (`content/fetchers/about/certificate.c`, `WITH_MBEDTLS`: the names, the validity, the serial, the signature algorithm, the SHA-1 / SHA-256 fingerprints, the SAN names, the RSA / EC key). The perf log / stderr shows `ONYX-TLS refused <url>: certificate i/n: <reason>`. WebSocket and EventSource (`onxy_ws.c`) check the same, and follow a host the user accepted. The PC bench verifies too (`tools/tests/netsurf/host_stubs.c`: OpenSSL with the same bundle, plus the proxy's CA when the machine has one; `NS_MBEDTLS=1` runs the Pi's mbedTLS code instead). `tools/tests/netsurf/tlstest.sh`: badssl.com's expired, wrong.host, self-signed and untrusted-root refused with their reasons, badssl.com, en.wikipedia.org and github.com trusted, "Proceed" and the viewer -- with both TLS stacks.

TLS 1.3. mbedTLS is built with TLS 1.3 besides 1.2 (`MBEDTLS_SSL_PROTO_TLS1_3`): 1.3 runs on mbedTLS' PSA crypto, whose random generator is the app's (`MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG`: `mbedtls_psa_external_get_random` in `user/tls/onyx_tls.hpp`, over `kapi_random`; `psa_crypto_init` once per app). The client offers the cipher suites, signature algorithms and groups in Chrome's order (X25519, P-256, P-384 -- not Chrome's post-quantum hybrid, which mbedTLS lacks; the TLS 1.3 key share for X25519); a TLS 1.3 server's tickets are taken after the handshake (`recv`) and kept like the 1.2 sessions (in memory and in `TLSsessions`): a known host's next connection is a PSK resumption. A handshake is one round trip shorter (en.wikipedia.org on the bench: ~70 ms instead of ~105). `net:conn` ends with the version (`TLSv1.3`); `NS_TLSDEBUG=1` prints each handshake's version and whether a session was offered. The Pi's NetSurf grows by about 170 KB (`libmbedtls.a` 380 -> 550 KB); every app built on `onyx_tls.hpp` (the courier, `httpsget`, `ftpsfs`) gets 1.3 at its next link. Without `MBEDTLS_HAVE_TIME` the ticket's obfuscated age is sent as 0 (the servers use it for 0-RTT only, which is not offered). Google's "unusual traffic" page (`/sorry/`) comes to this bench's address for every client (curl included, with TLS 1.3 and HTTP/2): what TLS 1.3 changes for it cannot be measured from here.

Chrome's request fingerprint (`onyx_fetch.c`, `utils/useragent.c`): bot checks (Google's `/sorry/` and its reCAPTCHA among them) compare a request with the Chrome its User-Agent names. The User-Agents name **Chrome 142** (`ONYX_CHROME_MAJOR`; Chrome's reduced form: Android 10; K, Windows NT 10.0, <major>.0.0.0); the headers go in **Chrome's order** (`onyx_hdr_rank`: HTTP/1.1 from `onyx_order_headers`, HTTP/2 sorted after the pseudo-headers, with Chrome's `priority` header by destination: `u=0`, `i` for a document...); `sec-ch-ua` is Chromium's brand list with its GREASE brand drawn from the major version as Chromium does (`GenerateBrandVersionList`); the **high-entropy client hints** (`-full-version-list`, `-arch`, `-bitness`, `-model`, `-platform-version`, `-wow64`, `-form-factors`) go only to the origins that asked for them by `Accept-CH` (kept per origin for the app's life), as Chrome. Bump `ONYX_CHROME_MAJOR` / `ONYX_CHROME_FULL` now and then: an old Chrome stands out.

NetSurf's own User-Agent by default (2026-10-01, the user's choice; `utils/useragent.c`): claiming Chrome (on Android by default) got Google's and DuckDuckGo's bot checks -- a "Chrome" whose TLS handshake and scripts are not Chrome's -- while their full script applications are too heavy for the Pi anyway. Jet Browser now says Mozilla/5.0 (X11; Linux aarch64) NetSurf/3.12: no client hints (`sec-ch-ua...` only with a Chrome User-Agent), `navigator.vendor` empty, `navigator.platform` Linux aarch64. DuckDuckGo serves its full results to it (the PC bench). "Desktop site" still sends a desktop Chrome's to the sites the user picks, and Choices' `user_agent`: any other (the Android Chrome's is given there). (Since §35: the toolbar's pill -- Standard, Mobile, Desktop per site -- and `jet.ini`'s `mobile`.)

SD: /apps/jet.app/jet.ini (`utils/useragent.c`, `onyx_ini_uu`) makes them editable: an `[user_agent]` section (`default`, `desktop`) and a `[sites]` section (`host = User-Agent`, the host or its subdomains); read at the first request; the order: the site's line, "Desktop site", `default`, Choices' `user_agent`:, NetSurf's own. The client hints and `navigator.vendor` follow the User-Agent the site gets (a Chrome one: Chrome's hints, Google Inc.).

The address bar's search (`frontends/framebuffer/gui.c`, `onyx_browser_go`). The text typed is an address when it has a scheme (`://`, `about:`, `file:`, `data:`, `javascript:`, `view-source:`), is `localhost[:port]`, or has no space and a host part of letters, digits, - and dots ending in an IPv4 address or a label of 2+ letters (example.com, a.b.org:8080/x?q); it then goes to NetSurf's `search_web_omni` (no scheme: `https://`). Anything else (what is my user agent, c++, 3.14, foo) is searched for: `onyx_search_url` takes `jet.ini`'s `[search]` `engine = (jet_search_engine())` in `utils/useragent.c`; default `https://duckduckgo.com/?q=`, escapes the words with `url_escape` (spaces as +), and puts them in place of a %s in it, else at its end. Tested by `uatest.sh` (a %s engine, an appended one with `c++ & co` → `c%2B%2B+%26+co`, and `127.0.0.1:port/ua?addr` opened, not searched).

The keys typed are characters. Onyx's keys (`kapi.h`) come with the keyboard layout already applied: `A` with Shift, an accented letter as its Latin-1 code (`0xA0-0xFF`), the euro as `0x80`, Ctrl+letter its control code -- no Shift / Ctrl events. fbtk's keymaps knew only unshifted ASCII (capitals, `é`, `€` were dropped, and `-`, `=`, `0` were taken for the zoom's Ctrl keys): `onyx_key_to_ucs4` (`frontends/framebuffer/gui.c`) turns them into the characters (Ctrl+A/C/V/X/Y/Z/U the editing keys). The address field (`onyx_chrome.cpp`, `UrlField`) holds Latin-1 (the bitmap font's), turned into UTF-8 for the address opened and the clipboard (`latin1_to_utf8`, `utf8_to_latin1` for a paste). On Windows (`pc/Jet/winkapi.cpp`) `WM_CHAR` is brought to the same codes (the euro `U+20AC` as `0x80`; other characters past Latin-1 dropped -- `0x100..` are the arrows and F-keys). Tests: `jstest.sh` js-accents (`aAéÇ€--=0b` in a field), `uatest.sh` (`café €` searched as `caf%C3%A9+%E2%82%AC`).

Brotli and zstd. Accept-Encoding: `gzip, deflate, br, zstd`; the body is decoded as it comes by the matching streaming decoder (zlib, the brotli decoder already linked, zstd 1.5.7's decompressor vendored in `third_party/zstd-1.5.7, libzstddec.a`, ~70 KB).

HTTP/2 (nghttp2 1.70, vendored in `third_party/nghttp2-1.70.0, libnghttp2.a`, ~115 KB). The first download to an https origin offers `h2, http/1.1` by ALPN; when the server takes `h2`, that download's thread becomes the **connection's owner** (`h2_run`): the origin's other fetches wait for the answer, then go as **streams** on it, queued by the UI thread (`h2_queue`, a word the owner waits on). One connection per origin, 4 origins at once (an idle one closed for a new origin, after 30 s idle anyway); push disabled; flow control with Chrome's windows (6 MB per stream, 15 MB for the connection) and Chrome's SETTINGS, pseudo- header order and priority (some CDNs tell browsers apart by them). The same streaming callbacks reach the core (the head, the body as it comes, decoded). `content/fetch.c` lets the fetches of a multiplexed origin past the per-host limit (32 per host, 64 in all). A server that chooses `http/1.1` is remembered (`HTTP1Hosts`, a week), and so is one that answers the first stream 403: Fastly (bbc.com) refuses HTTP/2 from mbedTLS' TLS 1.2 fingerprint with a Chrome User-Agent -- the request is sent again over HTTP/1.1. `NS_H2=0` turns HTTP/2 off (the before / after), `NS_NETDEBUG=1` dumps each head and frame.

The sockets. The Pi's kernel had 16 TCP sockets for every app (64 since, closed at the app's exit): the fetcher keeps to about 12 (6 download threads, 4 pooled HTTP/1.1 connections, 4 HTTP/2 ones). At the app's end (`fetch_onyx_finalise -> onyx_fetch_shutdown`) every job is cancelled (the TLS handshakes and sends watch a cancel flag), the pooled and HTTP/2 connections closed, the threads waited for 300 ms at most, and `onyx_ws_shutdown` does the same for the WebSockets (`net:shutdown` in the perf log). A connect that finds the kernel's table full (`kapi_tcp_connect -2`) closes our idle connections and waits for a socket (5 s at most) instead of failing.

TLS sessions kept across launches. The session cache (32 hosts) is saved to `SD:/apps/jet.app/TLSSessions` (`mbedtls_ssl_session_save`) with the user data and loaded at start: a known host's first connection is a resumed (abbreviated) handshake.

Timings. With the perf log on (`NS_PERF=1`, or the file `SD:/apps/jet.app/perf`): `net:conn host:port queue dns tcp tls full|resumed h2|http/1.1` per new connection, `net:done url total ttfb protocol` (kept connection) bytes (revalidated 304) per response, `net:cache url fresh (card|memory)` per answer from the cache, `page:load` from the throbber's start to its stop, `cache:*` for the disk cache, `net:preconnect`, `net:tcp` for a failed or delayed connect.

A disk cache (`user/netsurf/onyx_cache.c`: NetSurf's `gui_llcache_table` backing store, replacing the upstream `fs_backing_store.c`). The objects and their metadata go to `SD:/apps/jet.app/cache/<id>.d|.m` (since §33: `RAM:/jet/cache/` by default, the kernel's RAM volume) with an `index` (id, sizes, last use, URL), written by a thread of their own (the UI never waits for the card);

64 MB (Choices' `disc_cache_size`), the least recently used out beyond it. (Since §32: an object is stored the second time it is seen, not over 512 KB, the writes paced, the index written once a minute at most.) In `llcache.c`: an object is written to the card when it is complete and not `no-store`, and it is either fresh for a while or has a **validator** (`ETag`, `Last-Modified`) -- such an object stale or `no-cache` is kept (on the card, and in memory until it is written) and **revalidated** on its next use (`If-None-Match` / `If-Modified-Since`: a 304 is `FETCH_NOTMODIFIED`, the bytes kept); `max-age` / `Expires` / `immutable` are honoured (a fresh object is not asked for at all); a URL with a query but no explicit lifetime is revalidated rather than dropped (RFC 9111; RFC 2616's rule dropped it); a status other than 200 / 203 is not kept. The fetcher now gives the core the cache's headers and lets the conditional ones through. The write bandwidth allowed is 32 MB/s (the upstream 1 MB/s left most objects unwritten).

Preconnect. `<link rel=preconnect>` opens the origin's connection while the page is parsed (HTTP/2 offered: the origin's fetches then go on it; an http/1.1 one goes to the pool), `<link rel=dns-prefetch>` resolves the name (the kernel caches the answer) -- 2 at once, only when an HTTP/2 slot is free, once per origin (`html/css.c` calls `onyx_fetch_preconnect`; `NS_NOPRECONNECT=1` turns it off).

CORS for the scripts' `fetch` and `XMLHttpRequest` (`quickjs/net.js`), as the Fetch Standard: a cross-origin request carries `Origin`; a non-simple one (a method other than GET / HEAD / POST, a header not safelisted, a non-form `Content-Type`) is preceded by a preflight (`OPTIONS` with `Access-Control-Request-Method` / `-Headers`, its answer cached for its `Access-Control-Max-Age`); the response reaches the script only when its `Access-Control-Allow-Origin` names the page's origin (or `*` without credentials, and `Access-Control-Allow-Credentials: true` with them); the script sees the safelisted headers and those of `Access-Control-Expose-Headers` (`Response.type cors`). `credentials` (`same-origin` by default, `include`, `omit`) and XHR's `withCredentials` decide the cookies: without them the fetcher sends no `Cookie` and keeps no `Set-Cookie` (the request header `X-Onyx-Credentials: omit`, taken off). `mode: 'same-origin'` refuses another origin, `'no-cors'` allows a simple request only and gives an opaque response (status 0, no headers, no body). `nettest.sh`'s `net-cors.html` checks each case against a second server (another origin of the same site).

Two fixes found on the way. `libparserutils` (`src/input/inputstream.c`): the Onyx "filter pending" refill of `peek_slow` ran before the end of the input and overran its buffer when a style sheet came in several parts (`github.com`'s sheets, decoded from brotli: "Error processing CSS") -- it now waits for the end (`had_eof`). The C library (`user/libc/onyx_syscalls.c`): `gettimeofday` took the kernel's 100 Hz ticks for milliseconds -- the clock of every app ran ten times slow (`time()`, `Date.now()`, the cache's ages); it reads the generic timer (`cntpct_el0` / `cntfrq_el0`) in microseconds now.

Measured on the PC bench (live sites through the proxy, `NS_MBEDTLS=1`, `tools/tests/netsurf/loadtime.sh`: the first run cold -- no cache, no TLS session --, the next ones a second launch):

Site	Run	<code>page:load</code>	Responses (304)	From the card	Connections (TLS resumed)
bbc.com	cold	32.9 s	343 (0)	0	17 HTTP/1.1, 39 HTTP/2 (13)
bbc.com	warm	7.0 / 7.7 s	57 (1-2)	186-188	11-14 HTTP/1.1,

Site	Run	<code>page:load</code>	Responses (304)	From the card	Connections (TLS resumed)
					32 HTTP/2 (19-22)
github.com	cold	2.1 s (one run; others had not settled in 40 s)	152 (0)	0	2 HTTP/1.1, 2 HTTP/2 (0)
github.com	warm	1.5-3.1 s	102 (0)	50	2 HTTP/1.1, 1 HTTP/2 (1)
en.wikipedia.org	cold	5.2-6.1 s	74-105	0	3-4 HTTP/2 (0)
en.wikipedia.org	warm	5.2-7.2 s	3-8 (2-5)	57-60	2-4 HTTP/2 (0: wikimedia resumes no TLS 1.2 session)

A second launch downloads 5-20 % of what the first did (bbc.com: 57 responses instead of 343, en.wikipedia.org: 5 instead of 105) and resumes most of its TLS handshakes; the bench's `page:load` gains little beyond bbc.com because the scripts and the layout dominate it (the cold bbc.com run also had a slow network: its sizes vary from one run to the next).

Before this work (HTTP/1.1 only, no disk cache): bbc.com 21.4-22.7 s with about 107 connections, en.wikipedia.org 7.6-10 s, github.com 3.9-4.7 s; HTTP/2 alone: bbc.com about 15 s (16 HTTP/2 and about 35 HTTP/1.1 connections), en.wikipedia.org 5.9-8.0 s. The bench's page times are dominated by the scripts and the layout (QuickJS, one core), not the network; the Pi's gain is in the connections (a TLS handshake costs ~100-150 ms of the Pi's CPU) and in what is not downloaded again.

Not done. GREASE and the other extensions of Chrome's ClientHello (mbedTLS writes its own set); OCSP / CRL revocation, Certificate Transparency, HSTS preload; HTTP/3; CSP; CORS for EventSource, `<img crossorigin>`, fonts and module scripts (their loads are the core's, not `net.js`); the cache partitioned by top-level site; `Vary` beyond NetSurf's; a back-forward cache; preload hints (`<link rel=preload>`, `modulepreload`, 103 Early Hints).

25. GPU compositing: the page in a band, retained layers, one composite a frame

Stage 2 of docs/07 §6: the frames of the browser's view are assembled by the compositing service `user/gpucomp` (the V3D on the Pi, its CPU path elsewhere or when the GPU fails), from pixels kept between redraws. Choices' `gpu_compositing` (default 1; the PC bench: `NS_GPU=0 / 1 / cpu`); `gpu_compositing:0` is the painting as before (the back buffer copied into the canvas). The code: `frontends/framebuffer/onyx_comp.c` (+ `.h`), the core's offer in `html/redraw.c` (`onyx_fx_retain`), the

plotters' hooks (`framebuffer.c`, `onyx_layer.c`), `gui.c`'s `redraw`, `user/nsfb/onyx_surface.c` (the canvas's part the compositor writes: `onyx_surface_hole`).

- **The band.** The page is painted by the core (the CPU plotters, as ever) into a RAM surface as wide as the view and three views high (at most 8 M pixels), a **ring** of document rows: row *y* is the band's row *y* mod its height. Its valid rows are one run that always holds the view's. A **scroll paints nothing** already there: the view moves over the band (one or two pieces of the ring, whole texels, `GPC_L_OPAQUE` | `GPC_L_NEAREST`); the rows that come into view are painted; in idle turns (30 ms after the last redraw) the rows around the view are painted ahead, a third of a view a turn, more of them in the direction of the last scroll. A sideways scroll, a resize or a new page start a new band. The band is a `gpucomp` texture updated **only where it was painted** (`gpc_tex_update` of the painted rectangle).
- **Damage.** The core's rectangles (`fb_window_invalidate_area`) are kept in document px and painted where the band holds them, in view or not -- but damage farther than half a view from the view waits until the view comes near (an animation out of view costs nothing, as when only the view was painted). A whole-document invalidation (a reformat) paints the view now, the rest of the band ahead later.
- **Retained layers.** A group the frontend can composite itself -- an opacity and / or a transform, no filter, no blend mode, no backdrop (§21) -- is **offered** by the core (`onyx_layer_begin` (`ctx`, 1, `ONYX_LAYER_OFFER`), `struct onyx_layer`'s `retain`, `key` = the box, `lm` / `ox`, `oy` = its matrix about its origin; `netsurf/onyx_paint.h`). Accepted, it is left out of the band; its pixels -- its whole rectangle, untransformed -- are painted apart as an isolated group's (over black then white: premultiplied ARGB; one pass when it is known opaque) into a buffer of its own, uploaded, and each frame composites it over the band through its matrix, in its clip, at its opacity (bilinear when transformed, whole pixels else). Its pixels are painted again only where damage reaches them (the rectangle in its own px, and through its inverse matrix). Refused: painted as ever -- inside a group the CPU paints (in place or apart), under a rounded clip, larger than 4 M pixels, past the budget.
 - *Painting order.* A plot operation painted into the band **after** a retained layer and over it would come out under it: every operation's rectangle is noted while the band is painted (`onyx_comp_note`, from the plotters); one over a layer offered earlier in the same piece **demotes** it (painted in place from then on; its area painted again). The layers keep the order they were offered in.
 - *The clip.* The core gives a layer its clip within the piece being painted; a side on the piece's edge is not known (open) until a piece shows it.
 - *Stale layers.* A layer not offered again where the band was painted over all it covers is dropped (its box lost its effect, or its box is gone); a new one's area is painted again (it may be there in place).
 - *Churn.* A layer whose pixels are painted again frame after frame (what it holds animates, it moves by a translation) is demoted: its passes would cost more than the in-place painting.
 - *The CPU path* retains a layer only once it was animated (below): there a transformed layer would be resampled at each frame, where painted into the band it costs nothing more.
- **Composite-only animations and hovers.** When a box's style changes only in its opacity or transform (`css_computed_style_effects_only_change`, `libcsc` -- an animation's frame through `onyx_hover_restyle_elements`, a `:hover`), `onyx_hover.c` gives the box its new style and asks

`html_redraw_layer_update` (`html/private.h`): the frontend's hook (`onyx_layer_props`, `netsurf/onyx_paint.h` = `onyx_comp_layer_update`) updates the retained layer's matrix / opacity and asks for a frame -- **nothing is painted** (the element's other boxes, its text, are not redrawn either: a transform or an opacity paints the group, not them). Refused (no layer, or its new footprint would reach what was painted after it -- a turned layer may use the square its rectangle sweeps about its centre, when nothing painted later lies there), the rectangles are redrawn as before. `pages/anim-layers.html` (a spin and a pulse): 133 frames of 150 a composite, 17 painted.

- **The frame.** The band's pieces in view, then the layers that reach the view, **one gpc_composite straight into the window's canvas** (its page area -- since §32 only the part that changed since the last frame; `GPC_C_CLEAR` on the GPU: the band covers it, the canvas need not be loaded); the back buffer's copies leave that part alone (`onyx_surface_hole`); the caret is drawn into the band.
- **Safety.** At start, a **self-test** composes a small scene by the GPU and by `gpucomp`'s CPU path -- the band alone (texels copied: at most 1 apart), the layers (the GPU filtering's allowance), and the layers **over what the target holds** (no clear: the V3D loads the target first -- the path that was wrong on the Pi, `kern/v3d_cl.h`); a mismatch, or the GPU lost, and the CPU path is used for the session. The kernel log (`stderr`) says `netsurf: compositing on: GPU: V3D 4.2 ... (or ... the GPU failed its self-test (<why>): the CPU, compositing off)`. A GPU lost later (`GPC_LOST`): every texture uploaded again, the CPU from then on. No memory for the band: compositing off, painting as before. **Budget:** the band (at most 8 M pixels) and the layers (24 M pixels, ~96 MB of textures) -- the layers out of the band's rows dropped first.
- **Fixed boxes** scroll with the page here as before (NetSurf lays `position: fixed` out in the document), so they are part of the band and repaint nothing on a scroll either; a fixed box that stays in the view would be a layer without the scroll's translation (with the hit test and the scripts' rectangles following) -- not done.
- **Two bugs of the CPU painting found on the way** (both modes): a box whose **shadow** reached the redraw's clip but not its border box set an upside-down clip, which the knockout refused -- **the rest of that redraw was dropped** (cards cut or missing after a scroll: `kotonstudio.com`'s, `github.com`'s lists; `html_redraw_box_inner`); a redraw queued before a scroll kept its window coordinates (`fb_pan`: now moved with the pixels).
- **Tests** (`tools/tests/netsurf/gputest.sh`): the composited frames against the CPU painting (`NS_GPU=0`), pixel by pixel (at most 0.1 % of pixels off by more than 16), for the layer pages (opacity, transforms, filters, hovers, transitions, animations stopped), loaded and scrolled; hovers of a layer's transform / opacity (`pages/css-fxlayer.html`); composite-only animation frames (`pages/anim-layers.html`); a long page scrolled notch by notch against one jump (`pages/gpu-scroll.html`: a 720 x 740 `mix-blend-mode` group, a blur, rotated and faded cards); the **software V3D** (host.mk `SOFTGPU=1`, `GPC_SOFTGPU=1`: `gpucomp`'s GPU path, the kernel's fragment shader in the QPU simulator and its own target packets) and the self-test's fallback when the GPU hangs (`GPC_SOFTGPU=hang`); the scroll frames' cost. All pass; `fxtest.sh` (against Chromium) and `jstest.sh` pass composited (the default); `sitesweep.sh` (which now scrolls 12 notches down and 4 up): no crash.
- **Timings** (`NS_PERF`): `ONYX-SCROLL <us>` for each frame of a scroll (both modes), `frame painted / composite, present WxH gpu|cpu N layers, band ahead, layer WxH retained`, and every 50 frames `ONYX-COMP n frames: painted, composite only, present`. The PC bench (1280 x 800 window, `gpucomp`'s **CPU** path; 20 wheel notches after loading):

page	CPU painting, a scroll frame	composited
kotonviolins.com (copy)	mean 5.9 ms, max 70 ms	mean 0.8 ms, max 1.1 ms
kotonstudio.com (copy)	mean 2.6 ms, max 14.7 ms	mean 2.1 ms, max 5.7 ms (its scroll handler and header repaint each frame)
github.com/stephaneweg/Onyx	mean 2.4 ms, max 6.9 ms	mean 1.0 ms, max 2.9 ms
bbc.com	mean 3.5 ms, max 33 ms	mean 2.2 ms, max 7.3 ms
pages/gpu-scroll.html	mean 3.0 ms, max 7.9 ms	mean 0.9 ms, max 1.3 ms

A composited scroll frame is the present (a copy of the view on the CPU path: ~0.5-1 ms on the PC); the rows ahead are painted in idle turns (5-8 ms a piece on the PC). On the Pi (5-8 times slower at painting, the V3D composing a 1920 x 1080 frame in a few ms: `gpcdemo bench`), a scroll frame over rows already in the band goes from the redraw of a strip -- 5 to 25 ms, 50 to 350 ms when a blurred or blended group is in it (github.com's hero: 11-20 ms a redraw on the PC) -- to one composite, and animated transforms and fades from a redraw of their rectangles to a composite.

- **A new page drops the retained layers** (`onyx_comp_page_changed`, from the window's `GW_EVENT_NEW_CONTENT`): they were kept per window -- an animated layer of the page before (kotonstudio.com's CSS-animated screenshot) stayed over the next page (kotonviolins.com). A page closed (another shows in its window, it is kept for the history) pauses its animations (`html_content.onyx_closed`; `oa_tick` looks again every second). Test: `gputest.sh 3b` (gpu-nav-a.html -> gpu-nav-b.html, composited = CPU).
- **Not done**: overflow scrollers as layers (an inner scroller's scroll paints its box again, in the band); groups with retained layers inside (a filter or opacity group holding layers: the CPU paints what it holds, into the band or the parent layer -- docs/07 §6 step 6's ARGB target and re-upload are not used); `will-change` (not parsed); fixed boxes that stay in view (above); `GPC_F_ASYNC` (the composite waits); a translation animated as a composite (it moves the box as the layout does: its rectangles are redrawn).

26. Layout and rebox performance: the flex layout memo, the kept style selections, partial restyles

Profiled on the PC bench (`NS_PERF=1`, the sampling profiler `NS_PROF=<file> + prof.sh`; the Pi is ~5-20x slower -- m.facebook.com's layout pass was 175 ms on the PC and 3.5 s on the Pi). Two costs dominated: m.facebook.com's **layout** (each pass ~170 ms, 64 % of the samples in the flex layout, nested up to 10 levels) and github.com's **reboxes** (the box tree built again 8-9 times while it loads, ~150 ms each, nine tenths of it the style selection of every element).

- **The flex / grid items' layout memo** (`layout_flex.c: layout_memo_*`, `layout_internal.h`). A flex item was laid out to measure it (its height at an auto height, in a column; its cross size, in a row) and again to place it (at its flexed or stretched size) -- at each nesting level, so nested flex containers cost 2^{depth} layouts. Within one `layout_document()` pass, two tables keyed by the box: the size a layout gave per inputs (its width, its height or AUTO, `DEF_HEIGHT`, the available

width, its paddings and borders), and the inputs its subtree is laid out at now. An item's layout asks the memo: a size known is taken without laying out ("size only"); the container lays its items out again at the end (`fx_ensure_item`) only when the memo gave a size without the subtree's state (and before a baseline is read). Grid items take the memo when their subtree is already laid out at the same inputs. Outside a pass (an early layout, a textarea) nothing is kept.

m.facebook.com: a layout pass 165-175 ms -> under 1 ms (the first one 8 ms) on the PC, the pixels the same; the box dumps of every bench page identical.

- **The kept style selections** (`html/onyx_restyle.c`). Each element keeps its last selection (the cascade's result, before the transitions and animations) in its DOM user data (`__ns_key_onyx_style_memo`), and a box tree selects again only the elements a change since can have restyled. The marks come from the mutation events (`dom_event.c`: `DOMAttrModified`, `DOMNodeInserted` / `Removed`, `DOMCharacterDataModified`; and the scripts' states, `n_set_state`), stamped with the next box tree's serial:
 - an element's attribute changed or the element inserted: it and its subtree (descendant and child combinators);
 - a child of P inserted or removed, its children or text changed: P's "children" mark -- the children whose selection looked at their position (`NSCSS_STRUCT_SELF`: `:nth-child`, `:first-child`, `:empty`, recorded by the selection callbacks in `css/select.c`) or at their previous siblings (`NSCSS_STRUCT_SIB`: sibling combinators, `:nth-child(of S)`); a child's attributes changed: P's "children's attributes" mark, for the `SIB` ones only (a class toggled in a list striped by `:nth-child` restyles one row); and every descendant whose selection looked at the structure around another node (`NSCSS_STRUCT_ANC`) when an ancestor has such a mark. An element selected again for a mark gives its subtree the mark. A kept selection is also checked against its parent's and the root's computed styles (interned by libcss: the same pointer is the same style; references held), its parent's custom properties (libcss keeps them in the node data, not in the computed style: `css_onyx_node_vars_ref` / `css_onyx_vars_same`), the `:hover` state of each node its selection tried `:hover` on (the element and its ancestors -- libcss asks it of every element; a `:hover` tried on another node is not kept) whose notes for `onyx_hover.c` are replayed, a link's `:visited` state (the history changes without the DOM), and an epoch that a new selection context or a media change moves. When the new context only adds or takes out sheets (a late `<link>`, a script's `<style>`, a `<style>`'s text changed -- github.com loads its sheets after the page), a kept selection of an epoch since stays for an element no selector of those sheets matches (libcss `css_select_style_onyx_probe` on a context of those sheets only; a probe that looks at another node's `:hover` or at the structure counts as a match); the sheets taken out are kept alive meanwhile (`onyx_restyle_keep_sheet`). Never kept: the elements in a shadow tree, shadow hosts and their light children (the first shadow root does not move the epoch: a light element's selection is the same through `onyx_shadow_style`). The PC bench: `NS_NORESTYLE=1` selects every element; `NS_RESTYLE_CHECK=1` compares each kept selection with a new one (`RESTYLE-MISMATCH` lines: 0 on github.com, bbc.com, m.facebook.com, en.wikipedia.org, reddit.com, amazon.fr, lemonde.fr, yahoo.com, duckduckgo.com, developer.mozilla.org, youtube.com); `NS_PERF` prints `styles: K of N` kept.
- **Selection costs that were n^2** (`css/select.c`, libcss): `:nth-child` and `:last-child` counted an element's siblings for each element -- the counts are now found for all a parent's children at once and kept until the next mutation event (`nscss_dom_changed`); libcss's style-sharing search walked back over every previous sibling of the same name that could not share (a list striped by `:nth-`

child) -- it tries 8 candidates. A 2500-row list (pages/rebox-perf.html, 10 000 elements): a rebox 380 ms -> 20-40 ms.

- **Reboxes coalesced and throttled** (html.c html_script_dom_changed): the first change sets when the rebox comes (a later one no longer puts it off -- a script changing the DOM every few ms kept it from ever coming), and while the scripts keep changing the DOM, one every max(50 ms, twice the last rebox's cost) at most; a script asking for a geometry still has its rebox at once (html_script_layout_now). A script turn whose DOM changes were all under a display: none ancestor of the last box tree (head, script, template, a hidden panel: onyx_restyle_node_hidden) reboxes nothing (html_script_dom_changed_by_script, from qjs.c).
- **Attribute-only changes restyled in the boxes** (html_restyle_in_place, onyx_hover_restyle_nodes): when every DOM change since the last rebox set attributes (a class toggled by a timer, a style attribute, aria-expanded, a popover state), the changed elements' subtrees -- with their following siblings whose selections looked at previous siblings -- are styled again in their boxes with the :hover machinery (§9): redrawn when only how they are painted changed, else laid out again without building the boxes; anything else (an unboxed element shown, a pseudo-element appearing, an image to fetch) builds the boxes again. rebox-perf.html (a class toggled every 100 ms): rebox + layout 60-100 ms -> 5-9 ms. NS_NOINPLACE=1 turns it off.

Measured on the PC bench (the sites live, the same session; layout / rebox:boxes summed over the load with its scripts, ~16 s), before (the branch without these changes) -> after:

Site	layout	rebox:boxes	note
m.facebook.com	10 passes, 1971 ms -> 1 pass over 1 ms, 8 ms	37 -> 11 ms	each pass ~170 ms -> < 1 ms
github.com/ stephaneweg/ Onyx	30 -> 23-42 ms	8 reboxes, 1613 -> 87-280 ms	a rebox 150 ms -> 6-20 ms (the first one full)
bbc.com	~9 ms a pass (unchanged)	~33 -> ~8-15 ms a rebox	a sheet taken out restyles the root: two full reboxes left
en.wikipedia.org	~45 ms a pass	237 -> 110-195 ms a rebox	its startup script changes <html>'s class: the first rebox is full
rebox-perf.html (10 000 elements)	40-60 ms -> none (paint-only)	380 ms -> 5-9 ms (in place)	a class toggled every 100 ms

On the Pi (~20x for m.facebook.com's layout): its cookie dialog's 3.5 s passes should drop to ~20-160 ms; github's reboxes from ~1.5-3 s to ~0.1-0.4 s. The scripts are now the bulk of the time on these sites (bbc.com: 86 % of the samples in QuickJS).

Left: an **incremental layout** (a pass lays out the whole tree: wikipedia ~45 ms on the PC; the dirty boxes and their ancestors only, from the nearest box whose size cannot change), **incremental box construction** (a rebox builds every box even when every style is kept: ~3 µs an element on the PC), the in-place restyle for child-list changes (a node inserted into a flex container) and for elements whose

parent has no box (`display: contents`). Tests: `jstest.sh -- js-restyle` (46 checks: each kind of DOM change, sibling combinators, `:nth-child`, `:empty`, moves, custom properties, sheets added / taken out / changed, a change under a hidden ancestor); perf pages `rebox-perf.html`, `flex-deep-perf.html` (a Facebook-like dialog of nested flex containers).

- **Attributes that build boxes** (`dom_event.c`, `onyx_attr_builds_boxes`): a change of `src`, `srcdoc`, `srcset`, `sizes`, `data`, `type`, `colspan` / `rowspan` / `span`, `alt`, `value`, `multiple`, `size`, `rows`, `cols`, `usemap`, `poster`, `href`, `rel`, `media`, `placeholder`, `start`, `reversed` is not an attribute-only change: the boxes are built again (an `iframe`'s new `src` loads -- §29's frames are synced at the rebox -- an image's new source is fetched, a cell spans again). Found by `iframetest.sh` once §29 met the in-place restyle.

27. WebAssembly (wasm3) and Web Crypto (mbedTLS)

Two P1 gaps of docs/07 §2-§3: pages that need WebAssembly stopped (QuickJS has none), and `crypto.getRandomValues` / `randomUUID` used `Math.random` (predictable session tokens and nonces) with an empty `crypto.subtle` (logins and SPAs call `subtle.digest`). Both work in a page's context and in a worker's (`js_newthread`, `qjs_worker_create`: set up after `net.js`).

WebAssembly

- **The engine:** `wasm3` (`third_party/wasm3-0.9.2`, MIT; `main` at `28ecb9a`, which says 0.9.2) -- an interpreter in portable C99 whose operations are threaded by tail calls, ~200 KB of AArch64 code, nothing to port (WAMR's fast interpreter needs its OS layer -- `mmap`, threads, stack guards -- and is several times larger; its AOT / JIT is the step after, docs/07 §2). Its Onyx settings (`src/m3_onyx_config.h`, read by its `m3_config.h`: no stack switching, snapshots, typed function references or gas metering, no guarded memories, 1.5 MB of native stack for a module's recursion, memories of 1 GiB at most) and one patch (the validator refuses an opcode the compiler lacks -- SIMD's `0xFD` --, so `WebAssembly.validate` answers for the whole module: pages detect features that way) are in its `README.onyx`. Its spec suite (wg-3.0: 27878 tests) passes with these settings. The Pi links `libm3.a` (committed; `user/netsurf/Makefile`, `-O3`), the bench compiles the sources (`host.mk`).
- **The API** (`quickjs/qjs_wasm.c`, the classes in C; `quickjs/wasm.js`, compiled in as `qjs_wasm.js.h`): `WebAssembly.validate`, `compile`, `instantiate` (bytes or a `Module`; the import object read a microtask after the call, as a browser does once it has compiled), `compileStreaming` / `instantiateStreaming` (a `Response` or a promise of one: `ok`, `application/wasm` over `http(s)` -- a file: / data: URL has no type), `Module` (the bytes copied and every function body validated at once; `Module.exports`, `imports`, `customSections`), `Instance` (`exports`: frozen, null prototype, the same function object each time), `Memory` (`buffer`, `grow`), `Table` (`get`, `set`, `grow`, `length`; funcref and `externref`), `Global` (`value`, `valueOf`; the seven value types), `CompileError`, `LinkError`, `RuntimeError`.
- **Values:** `i32` (`ToInt32`), `i64` as `BigInt` (a `Number` is a `TypeError`, as in browsers), `f32` / `f64`, funcref (null or a function `WebAssembly` exported), `externref` (any value), several results as an array (and from an import, an iterable). A JavaScript import is a raw function of `wasm3` (`qw_host_call`) that calls it; a function exported by the same store is linked directly (a Wasm-to-Wasm call). Re-entrancy works both ways (Wasm calls JavaScript which calls Wasm...).
- **Errors:** a trap is a `RuntimeError` ("unreachable executed", "integer divide by zero", "out of bounds memory access"...), Wasm's stack overflow a `RangeError` (Chromium's), a JavaScript exception thrown by an import crosses the Wasm frames and reaches the caller as the same object; link

failures (a missing import, a wrong type, a memory too small) are `LinkErrors`, bad bytes `CompileErrors`. A module past the page's `script_timeout` traps (`m3_Yield`, which `wasm3` calls at each call, asks `qjs.c: qjs_ctx_timed_out`).

- **Stores:** an instance is a module loaded into a `wasm3` runtime -- a store -- under a name of its own. `wasm3` links a module's imported memory, table and global to another module's export in the same runtime, so an instance goes into the store of the Memory / Table / Global objects it imports (the imports are renamed to their owner module and export: `qw_rename`), else into a store of its own. `new WebAssembly.Memory / Table / Global` make a small module exporting the one thing (`qw_make_holder`), in the context's shared store (the Emscripten pattern: `env.memory`, `env.table`, `__stack_pointer` made by JavaScript, then imported). A function of another store put in a table or passed as a funcref gets a trampoline -- a tiny module of this store importing it (Emscripten's `addFunction` puts a small instance's export in the main table so). Every JavaScript object of a store references a hidden store object, which owns the JavaScript values Wasm holds (the imported functions, the externref values) and shows them to the garbage collector (its `gc_mark`): cycles through imports are collected, and when nothing of a store is left its runtime -- linear memories, compiled code -- is freed (on the bench: 300 instances of 2 MB made and dropped, at most 9 stores alive at once). Linear memories are outside QuickJS's heap: past 64 MB more of them since the last collection, one is run (`qw_pressure`). A store's memories together are capped at 512 MB (the app has 2 GB of heap).
- **Memory buffers:** `Memory.buffer` is an `ArrayBuffer` over the linear memory itself (no copy); it holds the store (so a view outliving the Instance stays valid) and is detached when the memory grows or moves -- checked each time Wasm returns to JavaScript (after a call, before an import runs) and by `grow`. A Memory made by JavaScript and imported keeps its buffer (the module uses its bytes).
- **Speed** (the PC bench, `js-wasm.html`'s C program, `wasm/build.sh`): SHA-256 x 20000: `wasm3` 104 ms, V8 (Node 22) 12 ms, native gcc -O2 7 ms; a sieve to 2e6: 53 / 11-28 / 10.5 ms; fib(27): 9 / 1-1.5 / 0.4 ms; a Mandelbrot 320x240x256: 64 / 19-28 / 15.6 ms -- 3.5 to 9 times V8, as an interpreter. Not measured on the Pi yet (a Cortex-A72 at 1.5-1.8 GHz: expect 3-4 times the PC's times).
- **Not done:** SIMD (`wasm3` has none: `validate` says false, pages fall back to their scalar build), threads and shared memories (`shared: true` is a `TypeError`; `crossOriginIsolated` is false anyway), `WebAssembly.Tag / Exception` (a Wasm exception reaching JavaScript is a `RuntimeError`; tags are not exported), JSPI, the JS string builtins, ESM integration (`import` of a `.wasm`), `Memory.toResizableBuffer`, the i64 address of memory64 descriptors. `Module.imports` lists functions, tables, memories, globals, tags in that order (`wasm3`'s), not the import section's. Imports from two different instances' stores (a memory of one, a table of the other) are a `LinkError`. externref values stay referenced while their store lives. A loop without calls is not interrupted by `script_timeout`. A buffer `transfer()`red keeps pointing at the memory (not detached on grow).
- Tests: `tools/tests/netsurf/pages/js-wasm.html` in `jstest.sh` (100 checks: modules written byte by byte by a small assembler in the page -- add, memory with a data segment and grows from both sides, a Memory imported, imports and re-entrancy, a JS exception through Wasm, traps, a stack overflow, a start function's trap, tables with `call_indirect` and an imported Table, the `addFunction` pattern, i64 / BigInt, globals, f32 rounding, multi-value, externref; the promises and the streaming forms; `wasm/bench.c` compiled by clang to `js-wasm-bench.wasm` (SHA-256, sieve, qsort, recursion, Mandelbrot, an import, memory grown by its allocator); a worker; 300 stores

made and dropped). The same page in V8 (Node) passes all checks but "SIMD is false" (V8 has SIMD). `NS_WASMDEBUG=1` logs the stores made and freed.

Web Crypto

- **Random:** `crypto.getRandomValues` (an integer typed array, 65536 bytes at most: `TypeMismatchError` / `QuotaExceededError`) and `randomUUID` take their bytes from mbedTLS's CTR-DRBG (AES-256), seeded by the best entropy there is -- on the Pi the hardware RNG (`kapi_random`, Circle's `CBcmRandomNumberGenerator`, as `user/tls/onyx_tls.hpp`), on the PC bench `getrandom` -- and reseeded by mbedTLS every 10000 requests (`qjs_crypto.c`).
- **crypto.subtle** (`quickjs/crypto.js`, the natives in `quickjs/qjs_crypto.c` on `third_party/mbedtls-3.6.3`, which the app links for TLS): `digest` (SHA-1 / 256 / 384 / 512), `generateKey`, `importKey` / `exportKey` (raw, jwk, spki, pkcs8), `sign` / `verify` (HMAC, ECDSA on P-256 / P-384 / P-521 with IEEE P1363 signatures, RSASSA-PKCS1-v1_5, RSA-PSS), `encrypt` / `decrypt` (AES-GCM with its tag lengths and additional data, AES-CBC with PKCS#7 padding, AES-CTR counting in the counter's low `length` bits, RSA-OAEP with a label), `deriveBits` / `deriveKey` (ECDH, PBKDF2, HKDF), `wrapKey` / `unwrapKey` (AES-KW and the ciphers above; raw, pkcs8, spki or a JWK as JSON). Each method returns a promise; the algorithm is normalized as the standard says (case-insensitive names, `hash` normalized) and its arguments copied when called; failures reject with the standard's `DOMExceptions` (`NotSupportedError`, `SyntaxError` for usages, `InvalidAccessError` for a key's type / usage / extractability, `DataError` for key data, `OperationError`), parameter types with `TypeError`. `CryptoKey` (`type`, `extractable`, `algorithm`, `usages`) keeps its material in a `WeakMap`: a secret key's bytes, an asymmetric key's DER -- PKCS#8 (written here around mbedTLS's SEC1 / PKCS#1) or SPKI --, parsed by mbedTLS at each use. JWK: `key` oct / EC / RSA with `alg`, `key_ops`, `ext`, `use` checked; EC private keys checked against their point.
- **Speed** (PC bench; Chromium's in brackets): SHA-256 of 1 MB 5-7 ms (8), AES-GCM of 1 MB 14 ms (7), an ECDSA P-256 signature 1 ms (0.2), RSA-2048 4 ms (1). RSA key generation is synchronous: seconds for 2048 bits on the Pi (the promise resolves after).
- **Not done:** Ed25519 / X25519 (mbedTLS 3.6 has neither), AES-KW with PKCS#8 of odd lengths (the standard's), a `CryptoKey` sent to a worker (the structured clone does not know it), `SubtleCrypto` on the workers' own thread (there is none: §19). ECDSA signatures are deterministic (RFC 6979: mbedTLS's; valid everywhere, Chromium's are random).
- Tests: `tools/tests/netsurf/pages/js-crypto.html` in `jstest.sh` (88 checks): the standards' answers (FIPS 180 digests, RFC 4231 HMAC, RFC 6070 PBKDF2), and against Chromium's API -- `tools/tests/netsurf/crypto/mkvectors.js` runs Node's WebCrypto (the same API) and writes `pages/js-crypto-vectors.js`: its keys (every format, the three curves, RSA-2048) imported, its signatures verified, its ciphertexts decrypted, its derived bits (ECDH, PBKDF2, HKDF) and its deterministic outputs (AES, HMAC, AES-KW, RSASSA-PKCS1-v1_5) made the same; generated keys, round trips, the error names, a worker. The page passes in Chromium too.

28. Vue's first render on the Pi (browserscore.dev "0 of 0 features"); the time limit

What the Pi showed (after §23): browserscore.dev still at "0 of 0 features, 0 %". Its kmsg: 1646 [Vue warn] lines (the site's own: Vue's development build warns the same in Chrome), then JS job: `InternalError: interrupted at refreshComputed and ONYX-PERF js:job 60030626 us` -- Vue's first

render, one promise job, cut off by the 60 s `script_timeout`. On the PC bench the job took 8.5 s with the log on (`NS_JSDEBUG`; the Pi had its `jsdebug` file), 6.6-7 s without; the Pi is 5-7 times slower.

The profile (`NS_JSPROF + jsprof.py`, `NS_PROF + prof.sh`): 85 % of the job is QuickJS interpreting the site's code and Vue's (the component mounts, the reactivity's proxies and `track()`, Score.js's recalculation of every ancestor at each feature -- quadratic, the site's own); DOM-in-JS work (`insertBefore`, `setAttribute`, `createElement`, `textContent`) is 2-3 %. What was ours and slow:

- **The console with the log on:** each warning's arguments (Vue's component trace: reactive proxies of the features, every value read a tracked `get`) formatted up to 2000 values -- ~2 s of the job. Now (`dom.js fmt, fmtJSON`) an object is JSON's text but 40 values, 4 levels and 160 characters at most (a long string cut with `...`), and once the line passes 256 characters the further objects are only named (`[Object ...]`): the `kmsg` line is cut at ~128 characters anyway, and a string argument (Vue's message) costs nothing. A function shows as `function name()`, `undefined` as `undefined` (it printed nothing).
- **Map / Set / WeakMap keyed by objects** (`quickjs.c map_hash_key`, marked Onyx): the hash of an object key was `pointer * 3163` -- the low bits of an 8- or 16-byte aligned pointer stay zero, and the bucket is the low bits: one bucket in 8 to 16 used, chains as long (`js_map_get` spent its time in `js_same_value_zero`). The pointer is now mixed (murmur3's finalizer). Vue's `targetMap` / `reactiveMap` (WeakMaps of every reactive object) and its dependency maps are looked up at each property read: a WeakMap / Map lookup 1.4-1.7x faster. The Pi's `libquickjs.a` rebuilt.
- **`new URL(link, base).href`** (`dom.js`): the parse cache held 1024 URLs and was cleared when full -- `browserscore.dev` makes 3000 links a render (its features' spec and draft links), so every URL was parsed again; now 8192. A URL keeps the cached parse itself (`_c`, shared, never changed) and copies it only when something reads its record to change it (`_u`, a getter; the getters read `_r`), and its `href` is serialized once per cached parse (a WeakMap). The lone surrogates' replacement (`toUSV`) is the native `String.prototype.toWellFormed` (a regexp with a look-behind before). `3000 new URL(...).href` 126 -> 36 ms, again 93 -> 13 ms.
- **`getComputedStyle(e1).getPropertyValue('--color')`** after a render (the site's favicon): `N.cstyle` laid the page out first (a whole rebox of 10000 elements: 0.4-0.5 s on the PC, seconds on the Pi) to answer a property it does not compute (`dom.js` answers a custom property from the style attribute). `qjs_cstyle_known` lists what `n_cstyle` answers; for the others it returns at once, no layout. (Also: `fill` / `stroke` / `stroke-width` read the style found, not `box->style`: an element without a box crashed there.)
- **`getElementsByClassName`** (`carbon ads` asks it of the whole document after the render): it wrapped every element of the tree and split its classes in JS; `N.descendants(root, names)` matches the class attribute natively (`qjs_has_classes`; ASCII white space, as the spec).
- **`CSS.supports / element.style`**'s value check runs its string-stripping regexp only when the value holds one of `;{}!`.
- **`NS_PERF`** logs a layout a script's read forced when it is over 50 ms, with the script's stack (`js:forced-layout at cstyle (native)|at get (dom.js...)|...`).

The timings (the PC bench, the job's CPU time; the machine shared, ± 0.5 s):

browserscore.dev's render job	before	after
log on (<code>NS_JSDEBUG</code>)	8.3-9.4 s	6.4 s
log off	7.0-7.1 s	6.5-6.6 s

browserscore.dev's render job	before	after
the next job (favicon's <code>getComputedStyle</code>)	0.8-1.4 s	0.07 s

The score is unchanged (86 %, 1283 / 1489). The goal of ~3 s on the PC is not reached: what is left is the interpreter itself running the site's code (the JIT / bytecode work on QuickJS is separate). On the Pi the job is some 35-45 s -- under the limit with the log on or off, and the limit no longer stops a render that works:

The time limit spares a working script (`qjs.c qjs_interrupt`, `QJS_TIMEOUT_MAX`): browsers do not stop a script at all (Chrome asks after a while: "Page unresponsive"); NetSurf has no such dialog, and its limit only has to end a script that is stuck. The scripts' DOM changes are counted (`QJS_DIRTY`, where a change marks the page to lay out again: `qjs_dom_writes`); past `script_timeout` a script (a call or one promise job) runs on while it still changes the page -- its last change less than a quarter of the limit ago (15 s for 60) -- up to 4 times the limit (240 s). A loop that never ends changes nothing and stops at the limit as before; one that changes the page for ever stops at 4 times the limit. The first time a script runs past the limit the log says JS: a script past 60 s still changing the page: let run. The default stays 60 s (`script_timeout` in SD:/res/Choices, 0 = none); `NS_SCRIPT_TIMEOUT=<s>` overrides it on the PC bench.

Tests: `jstest.sh`'s `js-scripttime.html` (run with `NS_SCRIPT_TIMEOUT=1`): a script changing the page for 2.5 s finishes, a stuck one is stopped at 1 s, a runaway that changes the page for ever at ~4 s; `getElementsByClassName` (several names, white space, none, a miss), a custom property through `getComputedStyle`, the console's JSON for a small object, a 5000-key object named fast, a long string whole. `urltest.sh` (WPT's URL data) as before: 896 / 896, 278 / 278, 72 / 87. `jstest.sh`, `nettest.sh`, `fxtest.sh` pass; `sitesweep.sh`: no crash.

The CSSOM's changes written back once a turn (`dom.js`, `CSSStyleSheet._changed`): each `insertRule` / `deleteRule` rewrote the whole `<style>`'s text at once, and NetSurf parsed and applied the whole sheet again -- google.com's search page inserts its rules one by one by the thousand: 10 000+ sheet conversions on the Pi, the page "running mad". Now the sheets changed are written back together at the end of the script's turn (a microtask) or before a layout read (`offsetWidth`, `getBoundingClientRect`, `getComputedStyle`); and `CSSRuleList._fill(from)` redefines the index properties from the changed index only (an append: one). 2000 rules: 1049 -> 40 ms on the PC, 2000 -> 1 sheet conversion. Test: `jstest.sh`'s `js-insertrule.html`.

29. Iframes and the messaging between windows (reCAPTCHA's frames)

The aim: the sites that work through frames -- Google's reCAPTCHA (the "I'm not a robot" box of www.google.com/sorry/ could not be validated), embedded videos, login and payment widgets, consent managers -- working as in Chrome. What was there: NetSurf's iframes drawn and given the mouse, but each in a JavaScript world of its own (its own QuickJS runtime) with `window.parent` = `window.top` = itself, `contentWindow` / `contentDocument` null, `window.length` undefined, `postMessage` to itself only, no load event on the element; an iframe without `src` (`srcdoc`, `about:blank`) or hidden (`visibility: hidden`: reCAPTCHA's challenge frame waits so, off the page) had no frame at all; and **every DOM change of the parent made all its frames again** (a rebox destroyed the iframes' windows and loaded them anew: their scripts' state lost at each change of the page around them).

The frames follow the document's elements (`desktop/frames.c`, `onyx_frames_sync`). A document's frames are its `<iframe>` elements -- all of them, shown or not, with a `src`, a `srcdoc` or neither -- each a browser window **kept by its element** (`bw->onyx_el`):

- `bw->iframes` is an array of pointers (each window its own allocation), in the document's order; the sync (at the document's first layout, at each rebox, and when a script asks: `contentWindow` right after `appendChild`, `window.length`, `window[i]`) keeps the windows of the elements still there, makes the new ones, destroys the ones whose element left (`html_rebox` used to destroy them all). The box tree only links them: `box->iframe` set on the new boxes (`content_html_iframe.node`); a frame whose element has no box (`display: none`, its document still loading) is laid out at 300 x 150 and not drawn.
- A frame is **navigated when what its element asks for changes** (`bw->onyx_src: U:<url> / D:<srcdoc>`): a script's `iframe.src = ...` loads it again; the window's own navigations (its links, `location`) do not touch it. `srcdoc` is loaded as a `data:text/html` URL whose document takes the parent's base URL (`onyx_frames_srcdoc_base`, read by `html.c` where the base URL is set); no `src` (or a `javascript:` one): `about:blank`. A frame showing one of its ancestors' URLs (or 10 frames deep) stays blank, as in Chrome.
- `sandbox`: read when the frame is navigated (`ONYX_SANDBOX*`); without `allow-scripts` the document gets no JavaScript thread (`CONTENT_MSG_GETTHREAD` refused), without `allow-same-origin` its origin is opaque. `allow`, `loading`, `referrerpolicy`... are reflected and otherwise ignored.
- **load on the element**: when a frame's document has loaded (its window's load event fired: `qjs_load_later`; a document without scripts or not HTML: `onyx_frame_content_done`; a failed load too, as Chrome) -- again at each navigation of the frame. A window's own load waits for its frames' (`onyx_frames_loading`, within the 30 s the load waits at most).
- An `iframe`'s window as the scripts see it: its size (`innerWidth` / `innerHeight`), its scroll (`scrollX` / `scrollY`, `scrollTo`) are its scrollbars' (`qjs.c`: the frame has no gui window), and a scroll of the frame sends its document a `scroll` event (`frames.c`'s scrollbar callback). The scrollbars of an `iframe` are decided from its content alone (a frame first laid out at another size kept a horizontal scrollbar it did not need).

One runtime per tab. The frames of a tab run in its QuickJS runtime (`js_heap_share`: the `iframe`'s window takes its parent's heap; `js_destroyheap` gives each share back), each document in a realm (context) of its own: a same-origin frame's objects are the objects themselves, as in a browser. What that needed in `qjs.c`: a script's changes to another frame's DOM (or the runtime's shared promise jobs) lay that frame out again (`qjs_leave` looks at the heap's other threads); a thread freed while another realm may still hold its objects keeps its record -- its natives find a closed thread with nothing in it, not freed memory -- until the heap goes (`zombie`).

WindowProxy (`net.js`, the frames section; the natives in `quickjs/qjs_frames.c`). Each window has a frame id (`onyx_frame_id`); another window -- `iframe.contentWindow`, `parent`, `top`, `frames[i]` / `window[i]`, `event.source`, `window.open(url, name)`'s result -- is a `Proxy`, one per window in each realm (`event.source === iframe.contentWindow`), that asks at each use what the window's document is now (it follows its navigations):

- same origin (scheme, host, port; `file`: URLs are one origin among themselves, shown as "null" as Chrome does; `about:blank` and `srcdoc` take their parent's): the window's global object behind it (`contentWindow.document`, its functions and variables, `frameElement`);
- another origin: what HTML lets through -- `postMessage`, `location` (`set`, `replace`, `assign`; not `read`), `closed`, `length`, `frames`, `window`, `self`, `parent`, `top`, `opener`, `close` / `focus` / `blur`, indexed and

named child frames -- the rest a `SecurityError` ("Blocked a frame with origin ... from accessing a cross-origin frame."); then and the symbols undefined (a promise resolved with a window does not throw);

- no document running scripts yet (a frame just inserted): those fields, and the language's built-ins of the asking realm (`Array`, `JSON`... -- the scripts that take "clean" built-ins from a fresh iframe).

`window.parent`, `top`, `length`, `frameElement` (same origin only), `name` (the frame's target name, settable), `window[0..31]` and `iframe.contentDocument` (same origin) are defined per document; the **named frames** -- `window[name]`, `frames[name]` (the consent managers' stubs look for `frames['__tcfapiLocator']`) -- are getters the frames' sync defines on the window below its own properties (`js_frames_changed`), and the `WindowProxy` finds them by name too; `window.open(url, name)` to a frame's name navigates that frame (and `_self`, `_parent`, `_top`). **Navigating another window** (its `location` set through the `WindowProxy`): its own frames and a same-origin window freely; an ancestor (`top`, `parent`: "frame busting") or another frame only within 5 s of the user's click or key in the caller's document, and from a sandboxed frame only with `allow-top-navigation` -- Chrome's rules, simplified (an ad frame cannot send the page elsewhere by itself).

Nodes between documents (`dom.js: insertAdopting`): a node of another document -- the parent's element appended into a same-origin frame's document (ad verification scripts do so), a frame's node into the parent's, a `DOMParser` document's -- is adopted when it is inserted, as in browsers. `libdom` cannot move a node between documents (its import keeps the old document as the copy's owner, and the insertion then fails: "not a Node" / `WRONG_DOCUMENT` before), so the subtree is made again by the target document (elements with their attributes, texts, comments) and the original taken out of its parent; the copy is inserted and returned. Another realm's node is a node (`isNode: N.isNode`).

postMessage between windows (`framePost`): the value written by QuickJS's serializer in the sender's realm and read in the receiver's (`net.js' toWire / fromWire` around it: `Blob`, `File`, `Error`, `ImageData`, the ports) -- every object of `event.data` is the receiver's realm's (`Object.getPrototypeOf(e.data) === Object.prototype`): nothing of a cross-origin sender leaks through it. Delivered as a task (one scheduler callback for the queue, the receiver's microtasks between messages), to the window's document of that moment; the `targetOrigin` checked then (`*`; `/`: the sender's origin; an origin: the receiver's, never an opaque one) and the message dropped otherwise; `event.origin` the sender's, `event.source` its `WindowProxy` (to reply). `postMessage(msg, '/', { targetOrigin, transfer })`, a bad `targetOrigin` (`SyntaxError`), a function in the message (`DataCloneError`) as Chrome. A window posting to itself keeps `html5.js'` path (no serializer: a `setImmediate` polyfill posts at each task).

MessagePort across realms: a channel within one realm stays `html5.js'` (React's scheduler posts on one at each task); a port transferred to another realm -- in the transfer list, or in the message itself (`{ port: ch.port2 }`) -- becomes a pair of ids in `qjs_frames.c` (`portPair`, `portOwn`, `portPost`, `portClose`): its other end, still here, posts through it; the messages that wait for a port in flight are given to the realm that takes it, those it had not given its scripts yet go with it (`portQueue`); `start()`, `onmessage` starting it, `close()` (the other end gets nothing more), a port transferred twice or detached (`DataCloneError`). Both ends of a channel can go to two other realms (reCAPTCHA's anchor and challenge frames talk so).

BroadcastChannel reaches the same-origin documents of the app and their workers (`qjs_net.c: n_broadcast` looked at a page's own workers only).

The tests (`tools/tests/netsurf/iframeptest.sh`, checked against Chromium headless for the same pages): `pages/frames-api.html` (55 checks: the parser's and a script's frames, `srcdoc`, `about:blank` written by its parent, a `src` changed, a frame removed (`closed`), the parent's DOM changed (the frame kept with its state), a frame in a frame talking to the top window, sibling frames, sandbox, the structured clone

each way, the `targetOrigin` checks, ports in the transfer list and in the value, a port closed, `BroadcastChannel` across frames, `window.open` to a frame name); `pages/frames-input.html` (a checkbox clicked, an input focused and typed into, Enter, the wheel in a frame; a link targeting the frame's name); `pages/recaptcha-outer.html` (a local mimic of reCAPTCHA v2 over two servers -- two origins: the page renders the anchor frame and a hidden challenge frame as `api.js` does, the frames say they are ready, the page gives each a port and the two of them a channel of their own, the anchor's checkbox clicked shows the challenge, its tile and "Verify" clicked, the token goes to the anchor, then to the page, which fills the hidden `g-recaptcha-response` and calls the site's callback; the cross-origin limits checked both ways, a message for another origin dropped, the anchor's try at sending the page elsewhere refused). Google's own reCAPTCHA is not touched by the tests.

A node adopted by another frame's document keeps its object (`qjs.c qjs_wraps_take`): since the in-place adoption (§18: `dom_document_onyx_adopt`, the same node, not a copy), the wrappers of the adopted subtree move to the adopting document's realm -- its wrapper table, its prototypes -- so `e1` appended into a frame's body is the frame's node (`e1.ownerDocument === frame.contentDocument`), and a frame's node appended here is this document's (`node.parentNode === document.body`).

A frame placed only in a laid-out tree (`desktop/frames.c onyx_frames_link`): the `rebox` syncs the frames before its new boxes are laid out; placing a frame then walked `box_coords` over boxes with no float container and no scroll bars yet -- a crash (on the Pi: `kotonstudio.com`, then `google.com`; a frame in a float on the PC). The frame is placed by the layout anyway (`layout.c`). Test: `iframetest.sh's frames-rebox-float.html`.

Not done: a frame's initial `about:blank` is not there synchronously (a script that writes into `contentDocument` right after inserting the `iframe` finds null until it has loaded, a few ms -- the built-ins above aside); `document.domain`; a node adopted from another document is a copy (its listeners and form state stay on the original, which a script may still hold); `location.href` of a `srcdoc` document is its data: URL (Chrome: `about:srcdoc`); the frames share the tab's process and runtime (no site isolation: a cross-origin frame's long script holds the page, as it did); `loading="lazy"` frames load at once; focus does not move between frames by script (`contentWindow.focus()` does nothing; a click does); a frame inside a shadow root has no window; `MessagePort` to a worker.

30. JavaScript speed: the interpreter, the code cache, a responsive window, the JIT

What the Pi's users met: `github.com` "hangs a lot before I can use it", `m.facebook.com` and `google.com` slow to load (Google's 1 MB script: 4.9 s on the Pi, the window not pumping its events for 2 s; 0.5 s of preludes at each launch). Profiled on the PC bench (`NS_PROF` + `prof.sh`, `NS_PERF`; the Pi is 5-10x slower):

- **github.com** (a repository page): the box tree rebuilt in full 9 times (`html_rebox`, ~130 ms each on the PC) and 0.46 s of layout outweighed its 1 s of scripts. In the scripts: 13 % in the module loader's regular expression (below), the forced layouts of geometry reads, `for...of` iterations, selector matching.
- **m.facebook.com**: 2 s of layout in 10 passes, 64 % of the samples in nested flex boxes (`layout_flex_item` 6-10 levels deep: each level lays its items out to measure them, then again to place them -- exponential); scripts 1.9 s, of which parsing a good part.
- **google.com**: the scripts' time split about evenly between parsing / compiling and running.

The layout costs went to the layout work (\$26: the flex layout memo, partial restyles); what follows is the JavaScript side.

The interpreter (`third_party/quickjs-ng-0.17.0/quickjs.c`, `README.onyx`)

Measured with callgrind's instruction counts (the shared PC's clock moves by 30 %): the bench `tools/tests/netsurf/jit/ -- qjsrun.c` (QuickJS-ng alone: `print`, `console.log`, `performance.now`, test262's \$262, the promise jobs; built for the PC and, static, for AArch64 Linux under `qemu-aarch64` by `build.sh`), `bench.sh` (Octane's CPU benchmarks and React 18's server rendering of 1000 rows, fetched once into \$CACHE; `COUNT=1`: callgrind), `test262.py` (test262 with its harness, strict and sloppy, async; with `--bin2` the tests whose outcome differs between two builds), `icount.sh` (the exact AArch64 instruction count of a run: `qemu` logs one instruction a block -- slow, small workloads).

- `OP_strict_eq` / `OP_strict_neq` decide inline: two ints, the same object or symbol, two strings of different lengths or both string atoms (unique by their text -- not a symbol's description, an atom of another type), `undefined` / `null` against anything; the rest calls `js_strict_eq_slow`. React compares strings against literals all the time (`switch (type)`).
- `JS_FreeValue`'s decrement inlined in the loop (`js_free_value_inl`); a string atom pushed as its own value (`OP_push_atom_value`, `typeof`'s result) without `JS_AtomToValue`; the interrupt poll always inlined.
- `for...of` over a dense array reads its elements directly (`js_array_iterator_next`).
- Instructions (x86, callgrind), before -> after: React SSR 4992M -> 4426M (-11.3 %), Richards 166.9M -> 157.5M (-5.7 %), DeltaBlue 290.4M -> 277.2M (-4.5 %), RayTrace -3.3 %, NavierStokes -4.1 %, Crypto -2.5 %, Splay -1.7 %, EarleyBoyer -4.1 %. test262 (35543 tests: 33785 pass) gives the same outcome for every test before and after.

The DOM glue (`qjs.c`, `dom.js`)

- ES modules: the imports a module names were found by a regular expression whose backtracking over github.com's 81 modules was 13 % of its script time; `N.moduleImports` does the same scan in C (the same specifiers on all 81 modules and the edge cases: comments, strings, `import(...)`, `export * from`).
- Class matching: `matches` / `querySelector*` / `closest`, `getElementsByClassName`, `classList.contains` and `[attr~=v]` split the attribute into an array for each node; `N.hasToken(node, name, token)` checks it in C (ASCII whitespace, as HTML says).

The code cache (`quickjs/qjs_codecache.c`)

A browser does not parse a script it has seen: V8 keeps its compiled code. Here the scripts' QuickJS bytecode is kept on the card, `SD:/apps/jet.app/jscache/<16 hex digits>.bc` (since \$33: in RAM, RAM: `/jet/jscache/`, by default):

- **What:** every classic script of 8 KB or more (external or inline) -- since \$32 the second time its source is seen (written paced, the index once a burst is over) --, and the preludes (`dom.js`, `html5.js`, `canvas.js`, `net.js`, `wasm.js`, `crypto.js`, `intl.js` -- their bytecode was already kept in memory for a process's next contexts; the first context now reads it from the card too). `js_exec` compiles (`JS_EVAL_FLAG_COMPILE_ONLY`) or reads the function, then runs it (`JS_EvalFunction`: the same as `JS_Eval`).
- **The key is the source:** its SHA-256 (mbedtls) names the file (the first 8 bytes); the file's header holds the whole hash, the source's length, the engine's build (`JS_GetBuildId`: the version and

quickjs.c's build date -- a new build writes its own, an old file is not read), the bytecode's length and checksum. A file is used only if all match: the same text, byte for byte, gives the same code wherever it came from (another URL, inline). A page's script that changes at every load (Google varies its scripts between requests) gets a new entry -- the same limit as V8's cache.

- **The functions' source is kept** (`Function.prototype.toString` gives the same text as without the cache): the files are ~3x the source (QuickJS keeps each function's own text).
- **Its size:** `jscache/index` lists the files (their key, size, last use); past 32 MB the least recently used are removed. **Writing** is a thread of its own on the Pi (`kapi_thread_create`; the stores queued in order, up to 16 MB waiting), the bench writes at once; `NS_JSCACHE=0` (the bench) turns the cache off. A file unreadable or not matching is compiled again.
- **Measured** (PC bench): the preludes 80 -> 11 ms (the first context of a process: 0.5 s on the Pi); a 1.2 MB script (Octane's TypeScript compiler) compiles in 62 ms, its bytecode reads back in 5 ms (on AArch64 under qemu: 506 -> 18 ms); google.com's gstatic script (244 KB) 81 -> 36 ms (compile + run -> read + run).

A responsive window during the scripts

- **Between the scheduler's callbacks** (`framebuffer/schedule.c`): every 4 ms the frontend is asked whether a click, a wheel turn, a key, a resize or the close box came (`onyx_chrome_input_pending`, `user/net surf/onyx_chrome.cpp`: it pumps the window's events without waiting); if so the scheduler returns and the main loop takes them before the next callbacks -- a click no longer waits for the 40 ms budget of timers and fetch callbacks.
- **During a long script** (`qjs.c`'s interrupt handler, every 100 ms of a script): the window's events are pumped in a deferred mode (`onyx_chrome_pump_deferred`): the system's queue is drained (no "not pumping" freeze, nothing dropped), the fetch threads' posts run (they only schedule), and the window's pointer and key events are kept (moves merged) and handled once the script is done, before anything else (`replay_deferred` at the next pump).

The JIT (`third_party/quickjs-ng-0.17.0/quickjs-jit.c`; experimental, off by default)

A baseline JIT for QuickJS's bytecode on AArch64, a function at a time, after `js_jit` calls (Choices: `js_jit:N`; 0, the default: the interpreter only; `JS_SetJIT` in `quickjs.h`, the executable memory from `kapi_code_alloc`). The design, kept small by leaning on the interpreter:

- **The frame stays the interpreter's:** `JS_CallInternal` sets it up (arguments, locals, the value stack in memory) then calls the function's code; the code keeps the stack pointer in `x19` (the locals `x20`, the arguments `x21`, its `JitFrame` `x22`, the realm `x23`, the bytecode `x24`, the stack frame `x25`, the runtime `x26`).
- **Templates** for the frequent instructions: constants and string atoms, locals / arguments / closure variables (with the reference counts inline), the TDZ checks, stack shuffles, `if` on a boolean / int / null / undefined, `goto` (the interrupt poll on the backward ones), integer `+` `-` `*` `&` `|` `^` `<<` `>>` `>>>` (overflow -> the slow case), comparisons -- fused with the branch after them when that is not a branch target --, `++` / `--` and `+=` on locals, `!`, `this`, `return`. **Inline caches** for `o.x` reads, writes and `.length`: each site keeps (in the code, branched over) the shape its objects had and the property's index; the template checks the object's shape, then the index, the atom and the flags in that shape again (a shape not shared can change in place: a property deleted or made an accessor), and reads / writes `p->prop[idx]` without a call; a miss calls the helper, which fills the cache when the object itself has the data property. **Helpers** in C for the rest (prototype properties, dense

arrays' elements, calls -- tail calls included --, globals, closures, {}, === on non-ints: the interpreter's fast paths).

- **Every other instruction, and every slow case, is the interpreter's:** the code calls `jh_step`, which runs that one instruction in `JS_CallInternal` (`JS_CALL_FLAG_JIT_STEP`: the frame resumed from a *copy* of the instruction followed by an invalid opcode, `OP_JIT_STEP_END`, whose dispatch returns -- the interpreter's own dispatch is untouched; a backtrace maps a pc in a copy back, `js_frame_pc`). So every instruction keeps the interpreter's exact semantics.
- **Leaving:** a return; an exception (the interpreter unwinds it: a `catch` goes on interpreted); an instruction whose next one is not the next in the bytecode (a `finally`'s `ret`, `with`): the interpreter goes on from there. Generators and async functions are not compiled.
- The code: blocks by powers of two in 1 MB chunks, the blocks of freed functions reused (process-wide; a spin lock). The interpreter with the JIT built in but off runs as fast as without it (the instruction counts: within 2-3 %; taking the address of the interpreter's `sp` / `pc` for the JIT had put them in memory: +28 %, found by `icount.sh`).
- **Measured** (AArch64 instructions, `icount.sh`, the JIT after 2 calls): a counting loop 3.7x fewer (53.3M -> 14.3M), a loop of small calls -11 %, Octane's Richards -8.8 % (56.9M -> 51.9M), DeltaBlue -3.8 % (98.2M -> 94.5M), RayTrace -2.8 % (530.4M -> 515.4M: its doubles go through C), React's server rendering even (211.4M -> 212.9M: calls and property reads on prototypes dominate it, both still through C). The interpreter with the JIT built in but off: +0.5 % (React) to +1.8 % (DeltaBlue). Before the inline caches and the slow cases' own helpers (a step cost ~100 instructions: the interpreter's prologue) it was +4 % to +9 % slower than the interpreter -- the reason they came first. Not yet measured on the Pi: hence off by default.
- **Validation:** test262 under qemu with the JIT after 1 call against the interpreter (`test262.py --bin "env QJS_JIT=1 qemu-aarch64 qjsrun-a64" --bin2 ...`, the interpreter built without the JIT): `test/language` and ten of `test/built-ins`' folders (Array, Function, Object, String, RegExp, JSON, Promise, Map, Math, Number), 35394 tests, the same outcome for every one. A first run found 384 differing: a step passed no function to the interpreter, which `OP_init_ctor` (a derived class's default constructor) reads. Plus the edge cases of the caches (a property deleted, made an accessor or read-only, a frozen object, a shape grown past its hash) and of the arithmetic (every pair of 19 values through `+ - * / < <= > >= == != ! - %`) against the interpreter. `jstest.sh`, `nettest.sh` and `iframetest.sh` pass (the PC bench is x86: no JIT there).
- **Next:** the prototype's methods in the caches (`o.f()`: a second shape), calls from compiled code to compiled code without `JS_CallInternal`'s prologue, re-entering the code at a `catch` and after a `finally`, float arithmetic, then measured on the Pi.

WebAssembly: an AArch64 JIT?

WAMR (the Bytecode Alliance's runtime) was considered for compiled Wasm: its fast JIT generates x86-64 code only (its build refuses other targets), its LLVM JIT / AOT is far too heavy for the Pi. `wasm3` stays (§27: an interpreter 3.5-9x slower than V8); a template JIT of its operations, on this JIT's assembler, is the way if the sites need it.

31. The name: Jet Browser (2026-10-01)

Decided by the user on 2026-09-30: the browser is **Jet Browser**. What changed:

- **The app:** `sdcard/apps/netsurf.app` became `sdcard/apps/jet.app` (`main`, `app.txt` -- `name = Jet Browser`, `category = Internet`, `stack = 8M` -- and the same `icon.bmp`); it is launched as `jet` (`run`

jet [url]). There is no netsurf alias: run netsurf no longer exists. user/netsurf/netsurf-app.mk's stage writes apps/jet.app; the binary is still built as netsurf.elf and the sources stay in third_party/netsurf/ and user/netsurf/ (comparable with upstream).

- **The window:** titled "Jet" (onyx_chrome.cpp, NsWindow). **Help ► About Jet Browser...** (AboutDialog): "Jet Browser -- the Onyx web browser, based on NetSurf", NetSurf's copyright (2003-2023 The NetSurf Developers), the GPL v2 (with no warranty; NetSurf's artwork MIT), the libraries and their licences (the NetSurf libraries MIT; QuickJS-ng, wasm3, PlutoVG / PlutoSVG, brotli, nghttp2 MIT; Mbed TLS Apache 2.0; FreeType's licence; zlib, libpng, IJG libjpeg, libwebp, zstd; the fonts' licences), and where the full texts are (about:licence, about:credits, SD:/res/fonts/LICENSE*). The User-Agent is unchanged (Chrome's, §18 / utils/useragent.c).
- **The dock:** the Internet drawer's default app is jet (user/dockconf.h, sdcard/etc/dock.ini; the Setup app's dock page, user/Apps/setup/main.cpp; sdcard/etc/quicklaunch.txt).
- **The user's files** moved with the app: ONYX_NS_DATAPATH is /apps/jet.app/ (user/netsurf/compat/onyx_nsconfig.h) -- Cookies, History, desktop-sites, TLSsessions, HTTP1Hosts, jscache/, cache/, the perf and jsdebug flag files. (Choices stays a resource: SD:/res/Choices.)
- **The data carried over** (gui.c, onyx_carry_old_data, before the options are read): for Cookies, History and desktop-sites, a file missing from SD:/apps/jet.app/ and present in SD:/apps/netsurf.app/ (ONYX_NS_OLDDATAPATH) is copied, once, silently -- the logins, the pages visited and the sites shown in their desktop version survive the rename. The caches are not copied (they rebuild themselves); nothing is deleted, the old folder can be removed by hand. Not on the PC bench (host.mk sets its own ONYX_NS_DATAPATH).

32. An idle Jet Browser, a quiet card: the system stays fast (2026-10-01)

What the Pi showed: "when Jet is running, the whole system gets slow" -- even a simple app such as the theme settings. On the Pi every app runs on core 0, so what Jet burns is taken from the other apps, the compositor, the input and the remote desktop. Two causes were measured:

1. **CPU while a page is shown and nothing happens.** kotonstudio.com (its copy, a 1920 x 1080 screen, the page loaded, then 10 s idle) took 6.5 % of a PC core with NS_GPU=0 and 13.7 % with NS_GPU=1 -- most of a Pi core (5-8x slower). The Pi's log: ONYX-COMP 100 frames: 100 painted, 0 composite only, frame painted 21-29 ms each frame. The cause: the page's .pill-dot runs animation: pulse 2s infinite on its box-shadow -- a paint property, not a transform or an opacity, so no frame of it can be composite-only (§25: the transform / opacity animations are, pages/anim-layers.html: 40 composite-only frames of 50). Each of the ~60 frames a second restyled the dot (walking the whole box tree twice), painted its 23 x 23 px into the band, then **composited the whole view** into the canvas (1.4-1.7 ms on the PC's CPU path) and presented the window.
2. **The SD card's write stalls.** The Pi's log: stall: jet:cache ran 103-213 ms without yielding (in Circle's CEMMCDevice::TimeoutWait: the SD driver busy-waits while the card writes, core 0 frozen for everyone). The disk cache (user/netsurf/onyx_cache.c) and the JS code cache (quickjs/qjs_codecache.c) wrote a lot: every cacheable object, every script's bytecode (~3x its source), at every visit when Google's scripts change.

The frames: fewer, smaller, none unseen

- **Their pace** (`html/onyx_anim.c`, `oa_schedule`): ~**30** frames a second (was ~60), **15** when the last frames changed only a few pixels -- painting only, the changed elements' boxes (their descendants' bounds, 8 px around) at most 48 x 48 px in all (`OA_TINY_PX`: a pulsing dot, a small spinner) -- and half that (15 / 8) when the window does not have the keyboard. A frame whose work is long still waits 1.5x its work. `requestAnimationFrame` follows the same pace (30 a second focused, 15 unfocused).
- **The window's state** (`frontends/framebuffer/gui.c fb_view_poll`, 4 times a second; `user/neturf/onyx_chrome.cpp onyx_chrome_view_state`): from the window manager -- `kapi_win_geometry`'s state (`KAPI_WIN_MINIMISED`, `KAPI_WIN_OFFDESK`, `KAPI_WIN_KEYS`) and `kapi_win_list` for what lies above the window (the opaque windows above it -- not minimised, not on another desk, alpha 255, not see-through -- subtracted from its client area: nothing left = covered). Three states (`onyx_view_state`, `html/onyx_anim.h`): focused, shown without the keyboard, **hidden** (minimised, on another workspace, covered).
- **Hidden** (as a background tab in Chrome): no animation frame at all -- the timelines park (`onyx_anim_set_view_state` wakes them when the window shows again; the animations' clocks went on: they jump to where they are), no `requestAnimationFrame` callback, the scripts' repeating timers -- and the timers a timer sets -- wait a second at least (`qjs_timer_delay`), animated GIFs stop (`image/gif.c`), and **nothing is painted** (the main loop keeps the redraws pending until the window shows again). `document.hidden` / `document.visibilityState` answer, `visibilitychange` is dispatched to every document (`js_view_visibility_changed`: a page can pause its own work). On the PC bench a hidden `kotonstudio.com` costs what an empty page does.
- **The restyle of a frame walks only the animated elements' boxes** (`html/onyx_hover.c`, `onyx_hover_restyle_elements`, `hv_scope_box`): an element whose box holds all its boxes (a block-level box: not an inline's pieces, which are their line's siblings) has its subtree walked (check, then apply) instead of the whole tree twice; else as before. `kotonstudio.com`'s frames: 45.8 -> 18.0 M instructions in `oa_tick` over the same run (`valgrind --tool=callgrind --toggle-collect=oa_tick`).
- **Only what changed is composited** (`frontends/framebuffer/onyx_comp.c`, `present`): the frame's changed rectangle -- the band's pieces painted, a retained layer where it showed and where it shows (`clayer.shown`), its pixels painted, it dropped or demoted -- is composited into the canvas (a target of that rectangle, the layers moved by its corner, as `gpucomp.h` allows); a scroll, a new view, a new band, the GPU lost, a dialog closed over the page (`onyx_comp_present_all` from `onyx_browser_redraw`) or nothing known composite the whole view as before; rows painted ahead out of view present nothing. The dot's frame: 1.4-1.7 ms -> ~0.05 ms of present on the PC. (The window is still handed to the kernel whole: `kapi_present` has no rectangle -- a kernel matter.)
- **The other steady costs looked at**: an idle HTTP/2 connection (kept up to 30 s for the next request) looked at its socket 100 times a second -- now 10 when it has no stream (`onyx_fetch.c`, `h2_run`: a new job still wakes it at once); the caret is drawn into the band only when its rectangle is painted, there is no caret blink nor throbber animation (the toolbar's reload button turns into stop); the WebSocket / EventSource threads poll their socket every 50 / 100 ms only while a page holds one (a socket's `recv` does not block on Onyx).

Measured (PC bench, `tools/tests/neturf/idlecpu.py`: the process's CPU over 10 s, the page loaded 15 s before, a 1920 x 1080 screen; the bench's own floor -- `pages/basic.html`, the simulator's main loop turning every 16 ms where the Pi's sleeps until an event -- is 0.7-0.9 %):

page	mode	before	after (focused)	unfocused	hidden
kotonstudio.com	NS_GPU=0	6.5 %	1.7-1.9 %	1.3 %	0.8 %
kotonstudio.com	NS_GPU=1	13.7 %	1.8-1.9 %	1.4 %	0.9 %
kotonviolins.com	NS_GPU=0	0.7 %	0.8-0.9 %		
kotonviolins.com	NS_GPU=1	0.9 %	0.7-0.9 %		

Over the floor, kotonstudio.com's idle cost went from ~6 / ~13 % to ~1 % of a PC core (on the Pi: from most of core 0 to ~5-8 %), none when the window is not seen.

google.com (the live page, `--env SIM_REALNET=1 --settle 30`), idle: at the floor before and after, its light page (NetSurf's User-Agent) and its desktop page (`desktop-sites`) alike (0.6-0.9 %): Google's timers do little once the page is up. What it costs the Pi is its load (its scripts) and what follows it: the writes (below), and its HTTP/2 connections, kept 30 s for the next request, each looking at its socket 100 times a second -- now 10 (`strace -c` of the idle page: 363 `polls` in 10 s for 4 connections).

The card: write less, write gently

(Since §33 the two caches are in `RAM`: by default; the rules below are the card's — Choices' `cache_on_card:1`; the History and the Cookies, on the card always, keep theirs.)

- **The disk cache stores an object the second time it is seen** (`onyx_cache.c`, `oc_store`): its URL's hash met in an earlier launch (the hashes seen are kept in the index, - `<hash>` lines, 4096 of them). Most of what a page fetches is never asked for again; a site visited again gets its objects stored then, and read from the card from the visit after. **Nor a body over 512 KB** (`OC_MAX_OBJECT`). A declined object stays in memory (llcache's RAM copy) for this launch.
- **A bug that made the disk cache write-only on the Pi**: the writer took `kapi_save_file(...) == 0` for success, but the kernel answers the bytes written -- every write was taken for a failure, so no object was ever listed in the index nor read back in a later launch, and each was written again at every visit. (The bench's `save_file` answers 0: the PC never showed it.)
- **The JS code cache writes a script's bytecode the second time its source is seen** (`qjs_codocache.c`, as V8 does): the first time its hash goes into the index (an entry of size 0, no file), the next time -- this launch or a later one -- the file is written. A script that changes at every visit is never written.
- **Paced writes** (`user/netsurf/onyx_io.h`, implemented in `onyx_cache.c`: `onyx_io_save`): a file over 16 KB is written in 16 KB pieces (`kapi_file_out` + `kapi_stream_write`) with an 8 ms sleep between them (the stall each write can cause is a piece's, not a whole file's 64 KB chunks back to back); a smaller one by `kapi_save_file`. The writers wait before each file -- and between pieces -- while the user acts (a click, the wheel, a key: `onyx_io_activity`, the last second) or a page loads (`onyx_io_loading`, from the throbber's start / stop; 60 s at most) -- up to 30 s, then write anyway.
- **The indexes are written rarely**: the disk cache's once a minute at most and at the end (was 3 s after the last writes, again and again on a Google visit); files a lost index left (the Pi switched off) are removed 30 s after the next start (`oc_sweep`). The code cache's once its writer is done, 3 s after the last store (was after each file).

- **The history, cookies and connection state** (`gui.c`, `onyx_save_user_data`, written whole on the UI thread 3 s after a page's load): once a minute at most (and at the end).

Measured (PC bench, google.com with the real network, three launches from an empty data folder, `strace` of the writes under the data folder; `NS_PERF=1` also prints each paced write as `ONYX-PERF io:write <path> <bytes>` and the disk cache's tally at the end):

visit	before	after
1st	10.4 MB (disk cache 2.2 MB in 27 files, code cache 8.2 MB in 17 files)	5 KB (indexes, cookies, history)
2nd	1.5 MB (code cache 1.2 MB: the scripts that changed)	5.7 MB (what was seen twice: disk cache 1.2 MB, code cache 4.5 MB)
3rd	365 KB (code cache 315 KB)	13 KB

From then on, a visit whose scripts changed writes nothing for them (before: their bytecode each time, 0.3-1.2 MB a visit here, more on the Pi's Google), and the writes that remain are paced and wait for a quiet moment.

The kernel side (not changed here): the stall is Circle's `CEMMCDevice` busy-waiting for the card (`TimeoutWait`) while it programs a block -- a write of 16 KB can still hold core 0 for the card's own latency (tens of ms, more during its garbage collection). The right fix is in the EMMC driver: wait for the transfer-complete / not-busy state by yielding to the scheduler (or an interrupt) instead of spinning, so that a writer thread blocks and the others run. Also: `kapi_present` presents the whole window -- a present with a rectangle would let the desktop's compositor copy only what changed.

Tests: `jstest.sh` (`pages/view-hidden.html`: the window minimised then shown again -- `visibilitychange`, `document.hidden`, no `requestAnimationFrame` callback while hidden, a 50 ms interval slowed to a second, the frames going on after), `fxtest.sh`, `gputest.sh` (composited = CPU painting, with the partial presents), `nettest.sh`, `iframestest.sh`; the desktop simulator has a `winstage <n>` step (the window's `KAPI_WIN_*` state); `idlecpu.py --floor <pages> (--state unfocused|hidden, --max <pct> to fail above a budget).`

33. The caches in RAM: (2026-10-01)

The decision: Jet Browser's disk cache and JS code cache live in **RAM**: the kernel's RAM volume (docs/02 §16, kernel v71), by default — **no browsing data is written to the SD card**:

- **Speed:** the card's writes freeze core 0, which every app shares (§32: `stall: jet:cache ran 213 ms`); §32 made them rarer and gentler, this removes them. A store into **RAM**: is a copy into the kernel's pages.
- **Privacy:** the content of the pages visited (their images, scripts, styles, the scripts' bytecode) never reaches the card.
- **The cost, accepted:** both are **lost when the Pi restarts** — the first visit of a site after a restart fetches everything again. Within a boot they survive Jet Browser closed and opened again (**RAM**: is not tied to the app). The **History** and the **Cookies** stay on the card (`SD:/apps/jet.app/`), as the TLS session tickets and the HTTP/1-only hosts.

Where (frontends/framebuffer/gui.c, before netsurf_init; Onyx patch): RAM:/jet/cache (the disk cache: Choices' disc_cache_path when it is not set) and RAM:/jet/jscache/ (the code cache: qjs_cc_set_dir, new in quickjs/qjs_codecache.c). **Choices' cache_on_card:1** (frontends/framebuffer/options.h, new, default 0) puts both back under SD:/apps/jet.app/ (cache, jscache/) as before §33; so does a kernel without RAM: (kapi_vol_info ("RAM:") -1: an older kernel, system.ini ramfs=0). A disc_cache_path in Choices still wins for the disk cache. The kernel log says caches: in RAM: or on the card; NS_PERF's cache:open line says (RAM).

What changes in RAM (the card's rules of §32 are kept for cache_on_card:1):

	in RAM:	on the card (cache_on_card:1)
A disk-cache object stored	the first time it is seen (llcache's rules: fresh or revalidatable, not no-store)	the second launch it is seen in
Its size	≤ 2 MB (OC_MAX_OBJECT_RAM)	≤ 512 KB
The disk cache's size	Choices' disc_cache_size (64 MB), at most half of RAM: (vol_info)	disc_cache_size
Its index	written 2 s after a change (OC_INDEX_TICKS_RAM), and at the end	once a minute at most, and at the end
A script's bytecode	written the first time it is compiled	the second time its source is seen
The code cache's size	32 MB, at most a quarter of RAM:	32 MB
Its index	as soon as the writer is done	3 s after the last store
The writes	at once, whole (kapi_save_file), no wait	16 KB pieces, 8 ms naps, waiting while the user acts or a page loads (onyx_io.h)

onyx_io_is_ram (path) (user/netsurf/onyx_io.h, in onyx_cache.c) tells a RAM: path: onyx_io_save writes it whole at once, and the writers (oc_write_some, oc_sweep, the index; cc_write_job, the code cache's writer) skip onyx_io_wait_quiet for it. The sweep of the files a lost index left (30 s after the start) runs in RAM: too (Jet Browser killed before its index was written). The folders are made with their parents (RAM:/jet first: the kernel's mkdir does not make parents).

The code cache's files through the kapi: qjs_codecache.c read its files with newlib's fopen (on the Pi, kapi_open underneath) and, on the PC bench, the host's; it now reads and writes them with the kapi (kapi_open / kapi_read, kapi_save_file, kapi_remove, kapi_mkdir: cc_read), on the Pi and on the PC alike — so the bench's stand-in kernel maps RAM: for it too.

On the PC bench (`tools/tests/desktop_sim/fakekapi.cpp`): `RAM:` is the folder `SIM_RAM`; unset, a fresh temporary folder per run (deleted at its end: each run a boot of its own). The stand-in now removes and renames files on `RAM:` (the card stays read-only) and answers `vol_info` (`RAM::` 128 MB). In `cache_on_card:1` mode, and for the code cache always, the files now go through the stand-in (`SIM_WRITES/<the data folder>`) instead of the bench's real `$(OUT)/data/jscache` — so a run no longer inherits an earlier run's code cache.

Tests (`tools/tests/netsurf/httptest.sh`): its two "launched again" disk-cache checks failed since \$32 (an object was stored only when seen in an earlier launch, so the second launch could not find it). They now run as Onyx does by default: the two launches share one `SIM_RAM` (one boot), the second finds the images in `RAM:` (no request) and revalidates the style sheet (a 304); the disk cache and the code cache are in `SIM_RAM`, nothing in the card's folder; a launch with a new `SIM_RAM` (a restart) asks for the images again. Then `cache_on_card:1` (appended to the bench's Choices): three launches — the first sees, the second stores (still asks for the images), the third reads the images from the card (no request) and revalidates the style sheet, drawn the same; the 1.9 MB image is asked for each time (over the card's 512 KB cap; `RAM:`'s 2 MB keeps it). The kernel side: `sh tools/tests/run_ramfs_test.sh` (docs/02 §16); on the Pi, `/bin/ramtest`.

To try on the Pi: `df` (the `RAM:` line); a site visited, Jet Browser closed and opened, the site again (it loads from `RAM::` `ls RAM:/jet/cache`, `NS_PERF`'s `cache:read` lines when `SD:/apps/jet.app/perf` exists); no `stall: jet:cache` line in `kmsg` while browsing; after a restart `RAM:` is empty; `cache_on_card:1` in `SD:/res/Choices` brings the card's folders back.

34. Jet Browser for Windows (2026-10-01)

A **Windows x64 build** of Jet Browser, to try sites on a PC and compare what it does with the Pi (`css3test.com`'s run, a site's scripts, the timings): `pc/dist/Jet/Jet.exe` and what it reads (`pc/dist/Jet.zip`: the folder, for a download). The same sources as the Pi's — the library stack, NetSurf's core, the framebuffer frontend, the Onyx glue (`onyx_fetch.c`, `onyx_cache.c`, `onyx_ws.c`, `onyx_chrome.cpp`: the wtk toolbar, URL bar, menus), mbedTLS through `onyx_nstls.cpp` — over `pc/Jet/winkapi.cpp`, the Onyx kernel's table (`KAPI_TABLE_VA`) on Win32, grown from Koton's (`pc/Koton/winkapi.cpp`). Built on Linux by `sh pc/Jet/build.sh` (`pc/build.sh` calls it): MinGW-w64 (the posix-threads variant), `pc/Jet/jet.mk` compiling every library from its source as `tools/tests/netsurf/host.mk` does, statically linked (`-static`; ~8 MB `Jet.exe`, the folder ~17 MB, the zip ~8 MB); the objects outside the repository (`JET_WIN_OUT`, default `/home/user/jetwin-build`). User guide: `pc/Jet/README.txt` (copied into the folder).

	On the Pi	On Windows
The C library	<code>newlib + crt0libc</code> , <code>compat/onyx_compat.c</code>	MinGW's; <code>pc/Jet/compat:</code> <code>regex.h</code> , <code>iconv.h</code> (the Pi's stubs: no <code>regex</code> , an <code>iconv</code> passthrough)
Files	<code>/res/...</code> , <code>/apps/jet.app/...</code> , <code>RAM:/...</code> through <code>newlib</code> to the <code>kapi</code>	the same paths: <code>fopen</code> , <code>open</code> , <code>stat</code> , <code>access</code> , <code>mkdir</code> , <code>rmdir</code> , <code>unlink</code> , <code>remove</code> , <code>rename</code> , <code>opendir</code> / <code>readdir</code> / <code>closedir</code> are wrapped at the link (<code>-Wl,--wrap=</code> , <code>jet.mk</code> 's <code>WRAP</code>): <code>__wrap_*</code> in <code>winkapi.cpp</code> map <code>/x</code> and <code>SD:/x</code> to the <code>exe</code> 's

	On the Pi	On Windows
		folder, RAM: /x to data\ram\x, C: /x and /C: /x (a file: URL's) to themselves, UTF-8 to the wide calls, always binary. The kapi's file calls the same
The data folder (<code>ONYX_NS_DATAPATH</code>)	<code>SD:/apps/jet.app/</code>	<code>/data/</code> = data\ (Cookies, History, jet.ini, TLSessions...); RAM: (the caches) is data\ram\, kept across launches; <code>vol_info</code> says 1 GB
The window	a wtk <code>Root</code> with its frame	a Windows window, its client area the canvas (no Onyx frame); a resize is told as the frame's maximise button (<code>GUI_EVENT_WINCTL_KAPI_FRAME_MAXIMISE</code> , twice once maximised: wtk's <code>Root::maximise</code> fills the work area = the client area); the menu bar a Windows menu; Alt released after Alt+Left / Alt+Right does not enter the menu bar's modal keyboard mode
Network	the kernel's TCP / DNS	Winsock (<code>tcp_*</code> , <code>net_resolve</code> : IPv4 first, as the Pi has no IPv6), a lock around the socket table
Threads	kernel v67 threads, mutexes, events, barriers, <code>wait_word</code> , <code>kapi_post</code> + <code>pump_wait</code>	Win32 threads, objects, <code>WaitOnAddress</code> ; <code>pump_wait</code> = <code>MsgWaitForMultipleObjectsEx</code> , a post wakes it (<code>PostMessage</code>)
TLS	mbedtls, <code>SD:/res/ca-bundle</code>	the same mbedtls and the same bundle (<code>res\ca-bundle</code> , not Windows' store): a certificate the Pi refuses is refused; the PSA RNG from

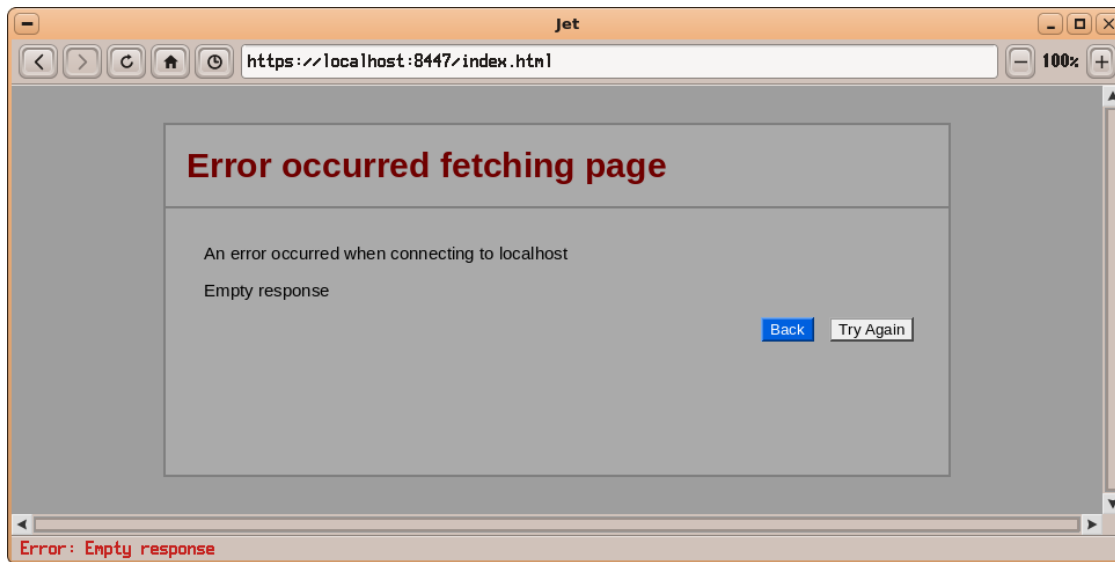
	On the Pi	On Windows
		<code>BCryptGenRandom</code>
JavaScript	QuickJS + the AArch64 JIT (<code>js_jit</code>)	QuickJS's interpreter (the JIT is AArch64's); <code>Date</code> in Windows' time zone (<code>quickjs.c</code> without <code>__ONYX__</code>)
Compositing	the GPU (V3D), else the CPU	<code>gpu_info</code> answers 0: Choices' <code>gpu_compositing</code> takes the CPU path (<code>compositing on: CPU</code>)
The log	the kernel log (<code>kmsg</code>)	<code>data\jet.log</code> (a new one each launch), or a console window (<code>--console</code>)
<code>NS_PERF / NS_JSDEBUG</code>	the files <code>perf / jsdebug</code> in <code>SD:/apps/jet.app/</code>	<code>--perf, --jsdebug, --netdebug (NS_NETDEBUG)</code> , or an empty file <code>perf / jsdebug / netdebug / console</code> beside <code>Jet.exe</code> (or <code>perf / jsdebug</code> in <code>data\</code>); any <code>NS_xxx=value</code> argument

The source changes for it (none in the Pi's behaviour): the page's event value is `long` `long` in `onyx_chrome.h` (`onyx_chrome_handler`) and `user/nsfb/onyx_surface.c`, `gui_value` in `onyx_chrome.cpp`'s handlers (a pointer event packs its wheel and buttons above bit 32: `long` has 32 bits on Windows; the same 64 bits on the Pi). The rest is in the build (`jet.mk`): `libc++` in `-std=c99` (MinGW's `math.h` defines `OVERFLOW` otherwise), NetSurf's `inet_pton` renamed out of `ws2_32`'s way, `wasm3`'s default `m3_Yield` renamed (its weak attribute is empty on Windows: `qjs_wasm.c`'s is the one), `-Wl, --stack,16777216` (QuickJS's 4 MB recursion limit on the main thread: the Pi app's stack is 8 MB), `-D_POSIX_THREAD_SAFE_FUNCTIONS` (`gmtime_r`).

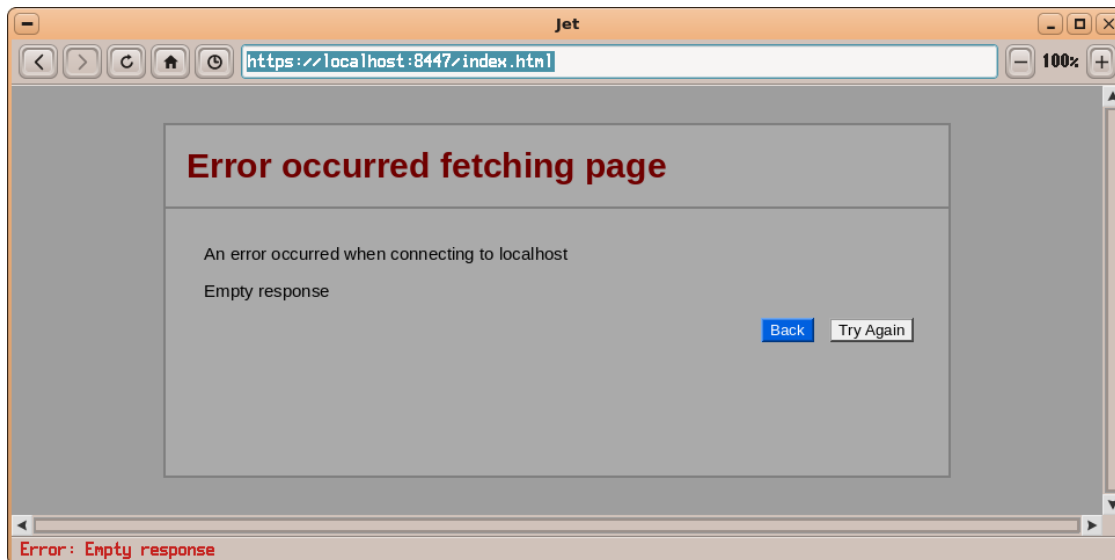
Tested (on Linux, under Wine 9 in a virtual X display, `xdotool` for the input): the welcome page; a local file (`Jet.exe C:\test.html`: a script, `console.log` in the log, `fetch` of a file); `https://example.com/` (TLS 1.3, HTTP/2, brotli; the sandbox's re-signing proxy refused as a self-signed chain, as it should, then accepted with its root appended to a test copy's bundle); **css3test.com: 83 %** (5263 of 6419), the window responsive after the run (the wheel scrolls, ~10–15 ms a frame), the process idle; the window shrunk and grown (the page laid out again), typing an address, `Alt+Left`, `Alt+F4` (a clean exit: History, Cookies, TLSessions written). Not tried on a real Windows machine.

35. The toolbar's padlock and the site's version (2026-10-01)

Two half pills joined to the address field, as in Chrome (`user/netsurf/onyx_chrome.cpp`):



The site's version: the pill, right of the field. A blue half pill (flat against the field, rounded at its right end: `wk_rbox` with `WK_TR` | `WK_BR`, the field drawn with its right end past its canvas so the two read as one control) shows the version the page's site gets -- **Standard** (the default User-Agent: NetSurf's own, or `jet.ini`'s `[user_agent]` default), **Mobile** (Chrome on Android: `Mozilla/5.0 (Linux; Android 10; K) ... Chrome/142.0.0.0 Mobile Safari/537.36`, the one Jet Browser sent by default before \$24, or `jet.ini`'s new `mobile`), **Desktop** (Chrome on Windows, or `jet.ini`'s `desktop`) -- and **Custom** (grey) when `jet.ini`'s `[sites]` has the site's own line, which wins and is not changed from the toolbar (the tooltip and the menu say so). A press opens its menu (`ModeMenu`, a `wtk::Modal` like `PopupMenu`, its right edge on the pill's): the site's name (`user_agent_site_of`: the registrable domain, `ua_site_of`; an IP address is its own site now -- it was cut to its last two numbers), the three versions with what each sends, the current one checked; a choice calls `onyx_browser_set_site_mode` (`gui.c`): `user_agent_set_site_mode`, then the page reloaded (`browser_window_reload(bw, true)`, as the old toggle). `Navigate` ► `Site Version...` opens the same menu. The pill is hidden for a page that is not `http / https` (`file:`, `about:`).



Kept per site (`utils/useragent.c`): `SD:/apps/jet.app/site-modes`, one `site mode` line per site switched (`standard` lines are not written: no line is `Standard`); the older `desktop-sites` (one site a line, each the desktop version) is read while `site-modes` does not exist, and the first change writes its sites into `site-`

`modes.user_agent_site_mode(host)` (USER_AGENT_STANDARD / MOBILE / DESKTOP / CUSTOM), `user_agent_set_site_mode`, `user_agent_for_host`'s order: jet.ini's `[sites]`, the site's version (Desktop: jet.ini `desktop` or the built-in Chrome on Windows; Mobile: jet.ini `mobile` or the built-in Chrome on Android), then `user_agent_string()` (jet.ini default, Choices' `user_agent:`, NetSurf's). `user_agent_is_desktop` / `set_desktop` stay (wrappers). The client hints and `navigator.userAgent` follow, as before.

The disk cache keeps the versions apart (`onyx_cache.c`, `oc_key_as`): the key is the URL with D| (Desktop), M| (Mobile) or C| (jet.ini's own) before it, nothing for Standard. An object is **stored under the version it was fetched as**, not the site's version when llcache writes it: the fetcher notes each URL's version when its fetch starts (`onyx_cache_fetched_as`, a 2048-slot table of URL hashes; a cache hit notes it too), `oc_store` and `oc_release` use it -- llcache writes an object out only once no page uses it, which after a switch is when the page of the other version replaced it (the old key, computed at the write, filed the Standard page under D|). The look-ups use the site's version now.

The padlock: left of the field. A half pill rounded on its left (`LockSeg`), the field's face tinted: **green** (`0x1E8E3E`) for an https page whose certificate was verified, **red** (`0xD93025`) for an https page loaded past a certificate warning, **grey**, **struck in red** for an http page ("Not secure"), hidden for the rest. The state is NetSurf's `browser_window_get_page_info_state` (`gui.c onyx_site_state -> onyx_chrome_set_site`): `PAGE_STATE_SECURE` green; `SECURE_ISSUES` (parts over http) green, the tooltip says so; `SECURE_OVERRIDE` (urlldb's certificate permission: the user's "Proceed") red; `INSECURE` grey; `INTERNAL` / `LOCAL` / `UNKNOWN` none. It is recomputed when the address is set, at a new content and when a load stops; the widgets repaint only when their state changes (the field moves only when the padlock or the pill appears or goes). A click (Navigate ► Page Security / Certificate...) opens the **certificate viewer** (`about:certificate`, §24) with the page's host's chain: the fetcher now keeps **the chain of every checked connection** (`onyx_chain_keep`, the 24 hosts used last, under a lock: the download threads write it), not only a refused one -- `onyx_nstls_connect` fills `*chain` when it connects too (mbedTLS: the chain the verify callback recorded, each certificate's fault -- an accepted one's are kept; a resumed session, which exchanges no certificate: the server's certificate the session kept, `mbedtls_ssl_get_peer_cert`; the PC bench's OpenSSL the same, `SSL_get1_peer_certificate`), and a resumed session's lone certificate does not replace a whole chain. `onyx_fetch_cert_url(host)` makes the viewer's address (`cert_chain_to_query`), `onyx_browser_show_certificate` navigates to it (NetSurf's own `browser_window_show_certificates` opens a new window, which this frontend has not); none known: a message box says what the padlock says.

Keyboard. F6 / Ctrl+L focus the field; from there **Tab** goes to the pill and **Shift+Tab** to the padlock (a hidden one skipped; past either end: back to the page), Enter / Space (and Down on the pill) open them, Esc goes back to the page. While the field has the focus the padlock's outline takes the accent too (one control).

Tests. `tools/tests/netsurf/uatest.sh` (a local server: `httpsrv.py`'s new `/ua` shows, logs and lets the page's script print the User-Agent; fresh for an hour): Standard sends NetSurf's; the pill's menu clicked -- Mobile then Desktop, each sent and seen by the page's script, the choice in `site-modes`; launched again (the same `RAM:`): Desktop from the disk cache, Mobile and Standard each their own copy (never the other version's), Standard again from the cache; `desktop-sites` still read; jet.ini's `[sites]` wins (Custom, the menu changes nothing); no green padlock on http. `tlstest.sh` (live badssl.com, both TLS stacks): the trusted sites' padlock green, red after "Proceed", and a click on each opens the viewer (the red one shows "The certificate is self signed"). The screenshots: `shots.sh jet` (a local https page from `tools/tests/desktop_sim/sd/jet/` over `h2srv.py`, its certificate made and trusted for the run).

36. The quadratic audit (2026-10-01)

The question: what in Jet Browser costs $O(n^2)$ or worse in the size of a page — its DOM, its style sheets, what its scripts do by the thousand (google.com, facebook, github, bbc, the Vue / React apps)? On the Pi, 5–10x slower than the PC, a quadratic step that takes 0.3 s on the PC bench takes 2–3 s. The earlier finds of this kind: `insertRule` writing the whole `<style>` back each time (§28's follow-up: coalesced once a turn), the rebox's object lookup (a hash), `:nth-child` counts in libcss (cached), libcss's style sharing search (capped), `new URL()` (a parse cache), the flex measurement (memoised), QuickJS's Map / Set hash of objects (§28).

The method: the code read (dom.js, html5.js, qjs.c, libdom, libhubbub's tree builder, NetSurf's html content, fetchers, lllcache, scheduler, libcss's selection), then each suspect measured by a micro-benchmark at $n = 1000, 2000, 4000, 8000$ — a linear step doubles, a quadratic one multiplies by 4. `sh tools/tests/netsurf/quadtest.sh [case,case...] pages/perf-quadratic.html` (65 cases, each in its own script turn; a case whose work ends later — a sheet's conversion — is polled) and prints the table with the growth from 1000 to 8000 (linear $\approx 8x$, quadratic $\approx 64x$); the fixed cases must stay under 20x, else it fails. `pages/js-quadratic.html` (in `jstest.sh`, 39 checks) checks the behaviour the fixes must keep.

The findings (PC bench, ms; n = the case's size)

What	Complexity before → after	Before	After	
<code>el.childNodes[i] / el.children[i] / .length</code> read in a loop (each read built the list: <code>N.children</code> , one wrapper per child)	$O(n^2) \rightarrow O(n)$	4000 children: 6182 / 10144	4 / 3	fixed
<code>childElementCount</code> in a loop	$O(n^2) \rightarrow O(n)$	8000: 11656	6	fixed
<code>getElementsByTagName(t)[i]</code> asked in the loop, <code>document.forms / images / scripts</code>	$O(n \cdot N) \rightarrow O(N)$	4000: 15501	6	fixed
<code>select.options[i], select.length, option.index</code> in a loop (<code>getElementsByTagName('option')</code> at each read; <code>index</code>	$O(n^2) \rightarrow O(n)$	2000: 4510; 4000: 7673; 4000: 9503	5; 4; 20	fixed

What	Complexity before → after	Before	After	
spread and searched the options)				
<code>form.elements[i]</code> in a loop (a query of the form and one of the document per read)	$O(n \cdot N) \rightarrow O(N)$	800 inputs: 1827	2	fixed
<code>getElementById</code> (libdom walked the tree from the root at each call; <code>querySelector('#x')</code> uses it)	$O(N)$ a call → $O(1)$	8000 lookups of 8000 ids: 2745; with a write each: 2741; 1000 misses: 707; <code>querySelector('#id')</code> : 2973	75; 84; 1; 79	fixed
<code>getElementById</code> of an element just appended	$O(N) \rightarrow O(1)$	8000: 2495	221	fixed
<code>compareDocumentPosition</code> (it listed the whole document, a wrapper per node, at each call: jQuery's <code>uniqueSort</code> , <code>.add()</code> , multi-selectors)	$O(N)$ a call → $O(\text{depth})$	sorting 1000 elements: 6098	23 (8000: 259)	fixed
<code>:nth-child()</code> , <code>:nth-last-child()</code> , <code>:*of-type</code> in <code>querySelectorAll</code> / <code>matches</code> (each element's position searched among its siblings, the	$O(n^2) \rightarrow O(n)$	(read)	8000: 47 / 60	fixed

What	Complexity before → after	Before	After	
of-type ones filtered for each)				
<code>addEventListener</code> of n functions on one target (the duplicate check searched the list)	$O(n^2) \rightarrow O(n)$	8000: 4406	15	fixed
<code>removeEventListener</code> (a search and a splice per call)	$O(n^2) \rightarrow O(n)$	8000: 1249	9	fixed
n <code>MutationObservers</code> : each change walked the ancestors once per observer and target	$O(\text{changes} \times \text{observers} \times \text{depth}) \rightarrow O(\text{changes} \times \text{depth})$	8000 appends, 800 observers: 5178	66	fixed
<code>setTimeout</code> / <code>clearTimeout</code> of n timers (NetSurf's scheduler: a linked list searched at each <code>schedule</code> , walked again from its head after each callback; qjs.c's timer list searched by id and to unlink)	$O(n^2) \rightarrow O(n \log n)$	8000: 781	21	fixed
Timers due together ran newest first (<code>setTimeout(a</code>	—	c,b,a	a,b,c	fixed (a bug)

What	Complexity before → after	Before	After	
<code>, 0); setTimeout(b, 0) ran b, then a)</code>				
<code>document.style sheets[i]</code> in a loop (a walk of the whole document at each read; emotion looks for its <code><style></code> 's sheet in it at each rule)	$O(S \cdot N)$ a read → $O(1)$ within a run	2000 reads, 200 sheets: 5510	3	fixed
Inline <style>s converted one per 10 ms (the <code>x-ns- css:</code> fetcher's poll — and the <code>javascript:</code> one's — ended after its first fetch: freeing the ring's head moved the ring, which the loop took for its end)	10 ms a sheet → all in one poll	100 <code><style>s</code> : 1014; 400: 4069	18; 61	fixed (a bug)
A selection context made for each sheet change: each sheet's media <code>"screen"</code> parsed by a new CSS parser; <code>onyx_restyle_ sheets_change d</code> compared the old and	$O(S^2)$ a change → $O(S \log S)$	(in the line above)	800 <code><style>s</code> : 312	fixed

What	Complexity before → after	Before	After	
new sheet lists element by element				
libcss's selector hash: 64 slots for ever, an insertion walked its chain (a sheet's n rules ending in the same element, ... a:hover: n ² /2 steps); an element's class lookup walked chains of (classes in the sheet) / 64	O(n ²) a sheet → O(n); lookups O(C/64) → O(1)	16000 .rN > a:hover rules: 3684	77	fixed
A sheet's rules appended as text over many turns (re-parsed whole each turn)	O(R ² /k)	2000 rules: 462	53	(the fetcher fix)
A leaf element removed: every sheet's node walked up to the root (html_css_nod e_removed)	O(S·depth) a removal → O(S) pointer tests	8000 removals, 100 sheets: 44	42	fixed (small)
libdom's live NodeList.item (i) (NetSurf's C code: iframes, image maps) walked from the start at each index	O(n ²) a loop → O(n)	—	—	fixed (a cursor)
llcache's search for a	O(objects) a request →	—	—	fixed (a hash)

What	Complexity before → after	Before	After	
cached object: the whole list of cached objects at each request	O(1)			
<code>style.setProperty</code> of k properties on one element (each set parsed the whole attribute in JS, then wrote it back)	O(k ²) parse → O(k ²) copy	800 custom properties: 1626	148	parse fixed; the write-back stays (below)

The fixes, by where they are:

- **Change counters (libdom):** `dom_onyx_tree_generation()` moves on at each node inserted or removed in any tree (`_dom_node_attach_range`, `_dom_node_detach_range`, `_dom_node_replace`, both ends), `dom_onyx_attr_generation()` at each attribute set, changed or removed (`_dom_attr_set_value`, the attribute list's insertions and removals: `_dom_element_onyx_attrs_changed`). They see every change, the parser's and the C code's too. `dom.js` reads them as `N.treeGen()` / `N.attrGen()` (qjs.c).
- **dom.js:** `kidsOf(node)` keeps a node's child list (and its elements, their index maps, the lists by local name) while the tree counter is unchanged; `childNodes`, `children`, `childElementCount`, `elementSiblings` (the structural pseudo-classes) use it, and the lists given to the scripts are the **same object** until the tree changes (`e1.childNodes === e1.childNodes`, as with live lists; still a snapshot after a change, as before). `getElementsByTagName` keeps its lists per root and name the same way; `select.options` is one collection per tree state — its **writes** are now the HTML options collection's (`options.length = 0` empties the select, `options[i] = new Option(...)` adds or replaces: they did nothing before); `option.index` uses an index map. `compareDocumentPosition` compares the two ancestor chains and the children's indexes where they part (the same answers: 20 / 10 / 4 / 2, 1 for a node out of the document). The listener lists keep their records by function (`byFn`), a removed record is marked and the list compacted once half of it is removed (the dispatch skipped removed records already). Mutation observers are indexed by the node they observe (`observedBy`): a record walks up its target's ancestors once. The inline style's parsed declarations are kept with the attribute's text (`_kept`). `document.styleSheets` is kept while neither counter moves, within one script run (a `<link>`'s sheet arrives between runs).
- **html5.js:** `form.elements` kept while neither counter moves.
- **getElementById (libdom, core/document.c):** an index made at the first lookup (a walk), then kept by the insertions (an inserted subtree's ids, before its mutation events: a script they run finds them) and by the attribute changes (`_dom_onyx_id_set`); each element has at most one entry (`dom_element.onyx_id`), taken out when it is destroyed or adopted by another document. An entry is checked at the lookup (that id still, in this document), so a removal or an id changed

needs no work; two elements with the id (a duplicate) fall back to the walk, for the first in tree order. `_dom_document_set_id_name` drops the index.

- **The scheduler (`frontends/framebuffer/schedule.c`):** a binary heap by due time then by the order of scheduling, and a hash by (callback, context) for the "one per callback and context" rule; the callbacks due when a run starts run soonest first (same budget, same input check). `qjs.c`'s timers are a doubly linked list with 256 buckets by id.
- **The fetchers (`html/css_fetcher.c`, `javascript/fetcher.c`):** the poll takes each pending fetch (as many as were there when it began) from the ring's head until it is empty — every inline sheet in one poll. A page's parsed `<style>`s go this way too: google's ~50 inline sheets were half a second of waiting.
- **libcss:** `css_select_ctx_insert_sheet` makes "screen" / "all" without the parser; `select/hash.c`'s tables grow (x8 when they hold 4 selectors a slot; a new chain takes the entries of one old chain, in order, so stays sorted) and keep each chain's tail (a selector going last — the rule order: most do — is appended at once).
- **NetSurf:** `onxy_restyle_sheets_changed` / `osr_probe_alive` compare sorted copies of the sheet lists (bsearch); `html_css_node_removed` tests a removed leaf against the sheets' nodes only; `llcache` keeps its cached objects in 1024 buckets by URL hash (`nsurl_hash`: the components `NSURL_COMPLETE` compares, without the fragment); libdom's `NodeList` keeps a cursor (its last `item()`), good while the tree counter is unchanged).

Not fixed (the risks left, with an estimate)

- **style.setProperty on one element:** each set serialises the element's declarations and writes the attribute back (NetSurf parses it again): O(k) a set. 800 custom properties on `<html>`: 148 ms on the PC (~1 s on the Pi); 200 (a theme): ~10 ms (~70 ms). A per-turn write-back (as the CSSOM's) would need every reader of the `style` attribute to flush first.
- **Array.prototype.shift** in QuickJS moves the whole array: a queue drained with `shift()` is quadratic (32000 items: 167 ms). The engine's; our own queues (`html5.js`: ports, streams) are short.
- **The selection context for each sheet change:** O(S log S) a change (the context made again from all the sheets); S sheets arriving one by one: 800 `<style>`s, ~0.3 s on the PC.
- **llcache_catch_up_all_users:** every "not caught up" event walks all the cache's objects (O(objects) an event: each network chunk). Thousands of objects × hundreds of chunks: tens of ms on the PC.
- **libcss's universal chain:** a selector whose last compound has no id, class or element name (`[data-x]`, `:hover`, `*`, `::before` alone) is tested against every element: O(N × U). And a descendant combinator walks the ancestors (the bloom filter skips most).
- **The parser's text:** a text node growing across C network chunks is copied whole at each (`dom_characterdata_append_data`): O(L × C) bytes — a 2 MB inline script in 16 KB chunks, ~128 MB copied (tens of ms on the Pi). From a file (one chunk): linear (8 MB of text, 74 ms).
- **Removing a subtree** (not a leaf) still walks each sheet's node to the root: O(S × depth).
- **querySelectorAll / getElementsByClassName** are O(N) a call (a wrapper per element); asked once per element, O(N²) — as in browsers, with a larger constant. The radio group check and `input.labels` query the document at each call.
- **getElementById of a duplicated id** walks the tree (rare); `NodeList.length` (C) walks the list.
- **innerHTML += in a loop:** quadratic in every browser.
- The wrapper table never shrinks (memory, not time).

- The layout cases (8000 blocks, inline spans, flex items, a 2000-row table, a 1000-deep nesting) are linear run alone; late in the full run their 8000 step grows ~5x (the heap then: the GC), a thing to watch, not a quadratic step found.

To try on the Pi: a page with many inline `<style>`s (google's results) styled at once instead of over a second; `css3test.com` / `browserscore.dev` (jQuery-free, many `getElementById` and child-list loops); a long `<select>`; the NS_PERF lines `js:forced-layout` and the scripts' time. Not tried on the Pi yet (the bench only); the Pi's `libdom.a`, `libcss.a` and the app need a rebuild.

The Pi's libraries and the headers (fixed after the audit). `user/netsurf/Makefile` (the Pi's `libdom.a`, `libcss.a`...) did not track headers: the audit's new fields in `libdom's struct dom_document` (`src/core/document.h`) recompiled `document.c` but not the files only including it -- `html_document.c` and the HTML elements kept the old layout, and the Pi's Jet applied no style at all (not even its own pages'), while the PC bench (built from scratch) was right. The objects now carry their header lists (`-MMD -MP -MF <obj>.o.d`, included by the Makefile; ignored by git): a header changed rebuilds its users. `libdom` and `libcss` were rebuilt from scratch.

37. Acid2 and Acid3 (2026-10-01)

The tests, kept as published (with their support files) in `tools/tests/netsurf/pages/acid2/` (`acid2.acidtests.org: index.html, reference.png, reference.html`) and `pages/acid3/` (`acid3.acidtests.org: index.html, empty.*, support-*.png, svg.xml, xhtml.1-3, font.ttf/.svg, reference.*`). Acid3 needs its original server's statuses and content types (`empty.css` served as `text/html`, `support-a.png` a 404, `support-b.png` as `text/html`, the `xhtml.N` as `text/xml`, `svg.xml` as `image/svg+xml`): `tools/tests/netsurf/acidsrv.py` serves `pages/acid3/` that way (anything else: 404).

Run them: `OUT=/tmp/nsbench PORT=8160 sh tools/tests/netsurf/acidtest.sh [acid2|acid3]` (both when no argument; the bench is built in `$OUT/build`, the pictures and logs go to `$OUT/acid/`).

- **Acid2:** the test opened at `#top`, as a browser shows it, in three modes: composited (the default), painted by the CPU (`NS_GPU=0`), and scrolled down twice and back (the fixed boxes and backgrounds painted again where the scroll moved them). The face (168 x 168 px below "Hello World!") is found in the view and compared with `reference.png` pixel by pixel, no tolerance; the whole view is compared with `reference.html`'s render (the reference image in a page). **Result: identical in the three modes (0 pixels differ).**
- **Acid3:** the page over `acidsrv.py` (`SIM_REALNET=1`), `NS_INJECT` adds a report that prints `ACID3-SCORE n (index run)` and one `ACID3-FAIL nn: message` line per failed subtest (from the test's own log, `NS_JSDEBUG=1`); F5 reloads it five times and the last report counts. It fails when the score is below **ACID3_MIN (94)** or when fewer than 100 subtests ran. **Score: 51 -> 94 / 100.**

What Acid2 found (all fixed)

- **position:** `fixed` painted where the viewport is: the fixed boxes are laid out at scroll 0 and moved by the scroll offset when painted (`html_box_viewport_fixed, redraw.c`); a scroll repaints them where they were and where they are now (`html_fixed_scrolled`, called from `browser_window_scrolled`), composited or not. **background-attachment:** `fixed` is positioned in the viewport.
- The painting order of CSS 2.1 Appendix E inside each block formatting context: the in-flow blocks' backgrounds, then the floats, then the lines (`redraw.c`'s phases: `onyx_phase_*`,

`html_redraw_box_body`). Acid2's eyes and nose overlap that way. A layered float's inline container is no longer culled (`HAS_FLOATS` in the descendant bounds).

- `<object>` fallback content when its resource fails: a fetch error, an HTTP error status (libcache's code ≥ 400) or a type it cannot show; the element's children are boxed instead (`html_object_fallback`, `box_object`).
- A float after a block takes the collapsed margins before it only (the following blocks' margins put it too high: `layout.c`'s margin walk).
- Relative offsets in `em` from the bottom / right (libcss returned the offset's type without its unit: 1px); auto heights never negative.
- A fragment (`#top`) scrolled to again once the page is shown -- the new content's scroll reset undid it -- unless the user has scrolled away meanwhile.
- A 1x1 opaque PNG used as a tiled background painted nothing (the fill colour's transparency byte, `image.c`).
- `medium` / `thick` borders are 3 / 5 px (as Chrome; also outlines and column rules).
- The alpha blend (`libnsfb_plot_util.h`) is now exactly $\text{floor}((s*a + d*(255-a)) / 255)$: the reference's semi-transparent pixels were off by one.

What Acid3 found (fixed)

- **Selectors (libcss, NetSurf's `css/select.c`):** `:enabled`, `:disabled`, `:checked` (an input's checkedness, an option's selectedness), `:lang()` (the `lang` attributes up the tree), `:empty` ignoring empty text nodes; attribute values compared case-sensitively except for HTML's list of case-insensitive attributes; `~`= splitting on any whitespace; a descendant or `~` combinator backtracked when the rest of the selector has another combinator (it took the first match only; bounded to depth 32).
- **Media queries (libcss):** an invalid query drops itself, not the whole list (recovery at the next top-level comma); `not` negates the whole query.
- **The CSS3 cursor keywords** (15 of them; the computed style's cursor field is now 6 bits).
- **Style sheets and scripts in the same turn:** a script that changed a `<style>` and reads a geometry or `getComputedStyle` sees the new rules (`html_css_flush_sync`: the modified inline sheets converted now, their `x-ns-css` fetches polled, their contents caught up -- also when the scheduled conversion already started them); a `<style>` inside an inserted subtree (`innerHTML`, `document.write` into a frame) gets its sheet (`html_css_style_inserted`, from `DOMNodeInsertedIntoDocument`); images with a `data:` URL are converted before a script's read (`html_object_flush_sync`), so a pending image's box is the image's, not its alt text's. More forced layouts a turn when they are cheap (16, or 100 ms).
- **DOM (`dom.js`, `html5.js`, `qjs.c`, `libdom`):** a full `Range` with live boundary updates on insertions, removals, `splitText` and data changes; `NodeIterator`'s pre-removal steps; the pre-insertion validity checks (`HierarchyRequestError`); `replaceData` with offsets (all the `CharacterData` edits through it); real `DocumentType` nodes (`createDocumentType`, `createDocument` with a doctype, `publicId` / `systemId`); the XML Name / QName checks (`InvalidCharacterError`, `NamespaceError`); `localName` without the prefix; `document.open` / `write` / `close` on a frame's document after it was parsed; the table API (`caption`, `tHead`, `tBodies`, `rows`, `insertRow`, `cells`, `rowIndex...`); `initUIEvent`; `defaultSelected`; `<object>`'s `data` / `type` reflected; `document.forms` / `images` named collections; a node seen from another frame keeps its identity (the wrapper of the realm whose document owns it); `getComputedStyle` answers the keyword properties.

- **libdom**: class names split on ASCII whitespace; `nodeValue` on a text node fires its mutation events; an input's checkedness and dirty value are state, not the attribute (`checked` set by script no longer changes `defaultChecked`); a detached text node's data change no longer crashes.
- **NetSurf**: an image map's `coords` no longer corrupted by `strtok` on the DOM's string; the user's checkbox / radio toggles `restyle (:checked)`.

The subtests still failing (94 / 100)

All six pass since §43 (XML documents, the SVG DOM): Acid3 100 / 100. The table is kept as it was found:

Test	What	Why
69	the support files loaded (seven frames' <code>load</code> events), then the SVG frame's DOM	<code>svg.xml</code> (<code>image/svg+xml</code>), <code>empty.xml</code> and the <code>xhtml.N</code> (<code>text/xml</code>) are not documents in Jet's frames: no XML parser builds a DOM for a frame, so not all the loads come and the SVG document has no elements (times out)
74	<code>getSVGDocument()</code> on an <code><iframe></code> / <code><object></code>	no SVG documents (above)
75	SVG DOM interfaces (<code>SVGRectElement...</code>)	no SVG DOM: SVG is drawn by PlutoSVG from its source, not from DOM nodes (§12)
77	<code>SVGTextContentElement.getNumberOfChars()</code>	no SVG DOM
79	an SVG font in a frame's document	no SVG documents in frames
80	scripts in an XHTML frame (<code>xhtml.1</code>)	XHTML served as <code>text/xml</code> is not parsed as a document

All six need **XML documents in frames** (an XML parser into libdom -- libdom has an expat binding -- and a content handler for `text/xml`, `application/xhtml+xml`, `image/svg+xml` in a frame) and then an **SVG DOM** over those nodes. Not done: weeks of work for the SVG DOM, rare on the web (docs/07 §3).

The Pi: not built here. Rebuild `libcss.a` (the computed style's layout changed: the cursor bits), `libdom.a` (`struct dom_html_input_element` changed), `libhubbub.a`, `libnsfb` (`libnsfb_plot_util.h`), then link Jet again (`make -C user/netsurf`, then `make -f user/netsurf/netsurf-app.mk link stage`); the Makefile now tracks headers, but a clean build of libcss and libdom is the safe choice after the generated headers changed.

After the merge (seen on the Pi). Two leftovers on the real Acid3 page: the hidden link of test 48 ("YOU SHOULD NOT SEE THIS AT ALL", `#linktest`) stayed red -- it points at the page the `selectors` iframe has

just loaded, which a browser counts as visited, but NetSurf marked visited only the pages of the top window's history; now a frame's page is visited for `:visited` too (`nscss_frame_visited_add` from `browser_window_content_ready`, a ring of the session's last 1024 frame pages, hashed -- `node_is_visited` checks it after `urldb`; not in the History); and the status bar showed "Error: UnacceptableType" -- test 69's frames (`svg.xml`, an XML type Jet does not show) failed and the bar took that frame's error for the page's: it now keeps a fetch error only when the window shows that error page (its address `about:query/fetcherror...`, or the failed address itself).

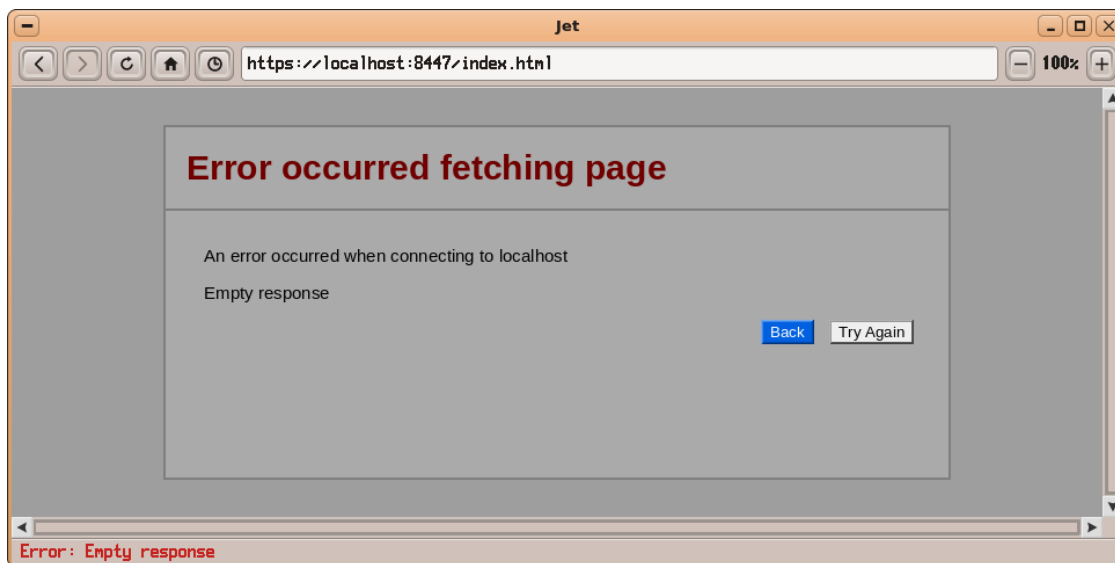
text-shadow and font-weight: bolder (seen on the Pi). Acid3's title is bolder with a silver shadow 3 px down-right: Jet showed neither. `libc` computed `text-shadow` but NetSurf never drew it:

`html_redraw_text_box` now draws the text first in the shadow's colour at its offset (one shadow; a blur is not drawn -- a blurred shadow's colour goes half way to the background, a blur with no offset (a glow) is skipped). And `bolder` / `lighter` stayed unresolved (the font code took them for normal):

`css_compute_absolute_values` (`libc` `computed.c`) resolves them against the parent's weight (CSS Fonts' table: `bolder` 100-300 -> 400, 400-500 -> 700, 600+ -> 900; `lighter` the reverse).

38. The page zoom, the status bar, the downloads (2026-10-01)

Three things of every browser that Jet Browser lacked: `Ctrl+` / `Ctrl-` (the user: "a zoom in the toolbar, `Ctrl+` and `Ctrl-`, `Ctrl+0` back to the original size"), a status bar ("Loading or Ready, the HTTP error, the link under the pointer") and downloads ("a dialog to save the file, its name filled in; `SD:/Downloads` made if missing").



The zoom. NetSurf scales a whole browser window (`browser_window_set_scale`: the layout is done at the viewport's width / the scale, the plotters draw at the scale -- text at its real size, sharp, not a stretched bitmap; the frontend's `fbtk` keys for it never came on Onyx, whose keys are characters). The toolbar's **zoom control** (`ZoomCtrl`, right of the pill: a framed -, the percentage, a framed +; the percentage in the accent when not 100 %) and the keys call `onyx_browser_zoom (step)` (`gui.c`): Chrome's steps -- 25, 33, 50, 67, 75, 80, 90, 100, 110, 125, 150, 175, 200, 250, 300, 400, 500 % -- in or out, or back to 100 % (a click on the percentage, `Ctrl+0`); the scale is Choices' `scale` (100 by default) times the zoom; the view keeps the same part of the page at its top. Keys (`key_event`, before the address field and the page: `Ctrl` held, not `Alt`): **`Ctrl++` / `Ctrl+=`** (the keypad's + too), **`Ctrl+-`, `Ctrl+0`, `Ctrl+wheel`** over the page (`ptr_event`: not a scroll); View ► Zoom In / Zoom Out / Actual Size. **The kernel** had to change for the keys: Circle's keymap

looks a key with Ctrl up in its Ctrl column (unless it is a letter) and the layouts (`SD:/etc/keymaps/*.kmap`) leave that column empty for `- = 0` -- nothing came. `CInputTask::SetKeyMapData` (kernel.cpp) now fills the empty Ctrl entry of the key whose own character is `-`, `=` / `+`, or `0` (its own or, AZERTY, its Shift one) with the keypad's key of that character (`KeyKP_Subtract`, `KeyKP_Add`, `KeyKP_0`), whose string Circle sends with or without Ctrl: the app gets the character, `kapi_get_modifiers` () says Ctrl (no kapi change; docs/02 § the keyboard). Ctrl+Shift+= (a US `+`) still gives nothing: Circle's Shift column wins. The Windows build sends the same characters from `WM_KEYDOWN` (`VK_OEM_PLUS` / `MINUS`, '0', the keypad's) and drops their `WM_CHAR` (Ctrl+- made a 0x1F).

Per site, kept. The zoom belongs to the page's host (`www.example.com`; `file` / `about` for those pages), as in Chrome; `SD:/apps/jet.app/view` keeps it -- `zoom <site> <percent>` lines (100 % is not written), with the status bar's `status_bar 0|1` --, read at the start, written at each change. A new page gets its site's zoom **before its first layout**: a core hook (`netsurf/onyx_jet.h`, `onyx_zoom_hook`, called by `browser_window_content_ready` for a root window -- the new content is the current one, the page before is gone, its iframes follow), so a site keeps its zoom without being laid out twice; the hook also sets the toolbar's percentage.

What the page's scripts see (`qjs.c`): CSS px are the scale's device px -- `qjs_scroll` divides the frontend's scroll offsets by the scale, `n_scroll` the viewport's size (`innerWidth`, `innerHeight`), `n_scroll_to` multiplies; so `getBoundingClientRect`, the events' `clientX` / `clientY`, `elementFromPoint`, `scrollX` are in CSS px (a click on a box at 200 % lands on it: `dltest.sh`). `devicePixelRatio` is the scale (a getter: `N.scroll` ()[6], rounded to 1e-4), as Chrome's zoom makes it.

GPU compositing (§25) at a scale other than 1: the band works as ever (the core paints it through the scaled plotters); the retained layers are not offered (`onyx_fx_retain` refuses when `scale != 1`: painted in the band, as on the CPU). `gputest.sh` §3a: `gpu-scroll.html` at 150 %, scrolled, composited = CPU.

The page's origin at a zoom (`gputest`'s 3a failed after §37: "the fixed bar 1 px off composited"). The core takes a redraw's origin in CSS px: `browser_window_redraw` divides the device px by the scale and truncates. The CPU painting's origin is minus the scroll offset, so at 150 % scrolled 500 px the page was painted from -333 CSS px = -499.5 device px, half a pixel below where the compositor's band (origin 0) has it; each strip a scroll exposed was off its own way (the CPU's own seams), and the fixed boxes -- moved by the scroll in whole CSS px (§37) -- landed a pixel apart in the two modes. The bar's edge showed it; the page's gradients had hidden it. Now the origin is always a multiple of the scale's **period** (`onyx_scale_period`, `onyx_comp.c`: the fewest device px that are whole CSS px -- 3 at 150 %, 11 at 110 %, 5 at 125 %, 7 at 175 %, 2 at 200 %): the CPU painting (`fb_redraw_page`, `gui.c`) paints such a redraw into a scratch surface whose origin is one and copies it in (only when the scale is not 1 and the origin is not one); the band's rows are a multiple of the period and its first column one (the band `period - 1` columns wider, remade when the zoom changes the period). `browser_window_redraw` rounds an origin within 0.01 of whole CSS px (-495 / 1.1f is -449.99998); `html_redraw_box` floors a box's scaled position instead of truncating it (a box above the view, at a negative y, lands where it does at a positive one). The two modes are now the same pixels at every step (`gputest.sh` 3a: 110, 125, 150, 175, 200 %, the view paged down before the zoom and two notches after -- 827, 912, 1054, 1196, 1338 px --, 100 px sideways at 175 / 200 %; before, 4 of the 5 failed). A fixed box can still sit up to a device px above its place at a zoom (whole CSS px), the same in both modes. `host.mk` stamps the link's kind (`link-softgpu.stamp` / `link-plain.stamp`): a `SOFTGPU=1` build after a plain one in the same folder links again (`gputest`'s software-V3D checks used a stale binary).

The status bar (`StatusBar`, a wtk widget at the window's bottom, `ONYX_STATUSBAR_H` = 22 px: the page's area is that much shorter -- `onyx_chrome_page` gives the page's height without it, the pointer's events

below the page go to the wtk tree, and View ► Hide / Show Status Bar is a resize of the page, `g_resized`, kept in `view`; the window's size does not change). Left: the page's state -- `onyx_status_start` (the throbber's start) "**Loading...**", four times a second "**Loading... 12 of 30**" (the fetcher's fetches made and ended since: `onyx_fetch_counts`, `onyx_fetch.c` -- the page, its sheets, scripts, images); `onyx_status_stop` (the throbber's stop): "**Ready**", or in red the page's **HTTP status** when 4xx / 5xx (`onyx_browser_window_http_code`, `browser_window.c`: the current content's l1cache code; the RFC's reason phrase: "404 Not Found", "500 Internal Server Error") or "**Error:** " when the fetch failed (about:query/fetcherror's reason, through `onyx_fetch_error_hook`: "Connection failed", a timeout...), kept until the next load. The **link under the pointer** replaces it: NetSurf's `set_status` texts are its own state messages ("Loading", "Fetching...", "Done (0.3s)", "Processing": the bar shows its state again) or the pointer's (a link's address, a form field's hint): those are shown, cut in their middle with "..." when too long. Right: a download's progress or the last one's end. The widget is repainted only when its text changes; the page is not (a hover costs the bar's blit and a present). The ftk status text left of the horizontal scroll bar is gone (`toolbar_status_size 0`).

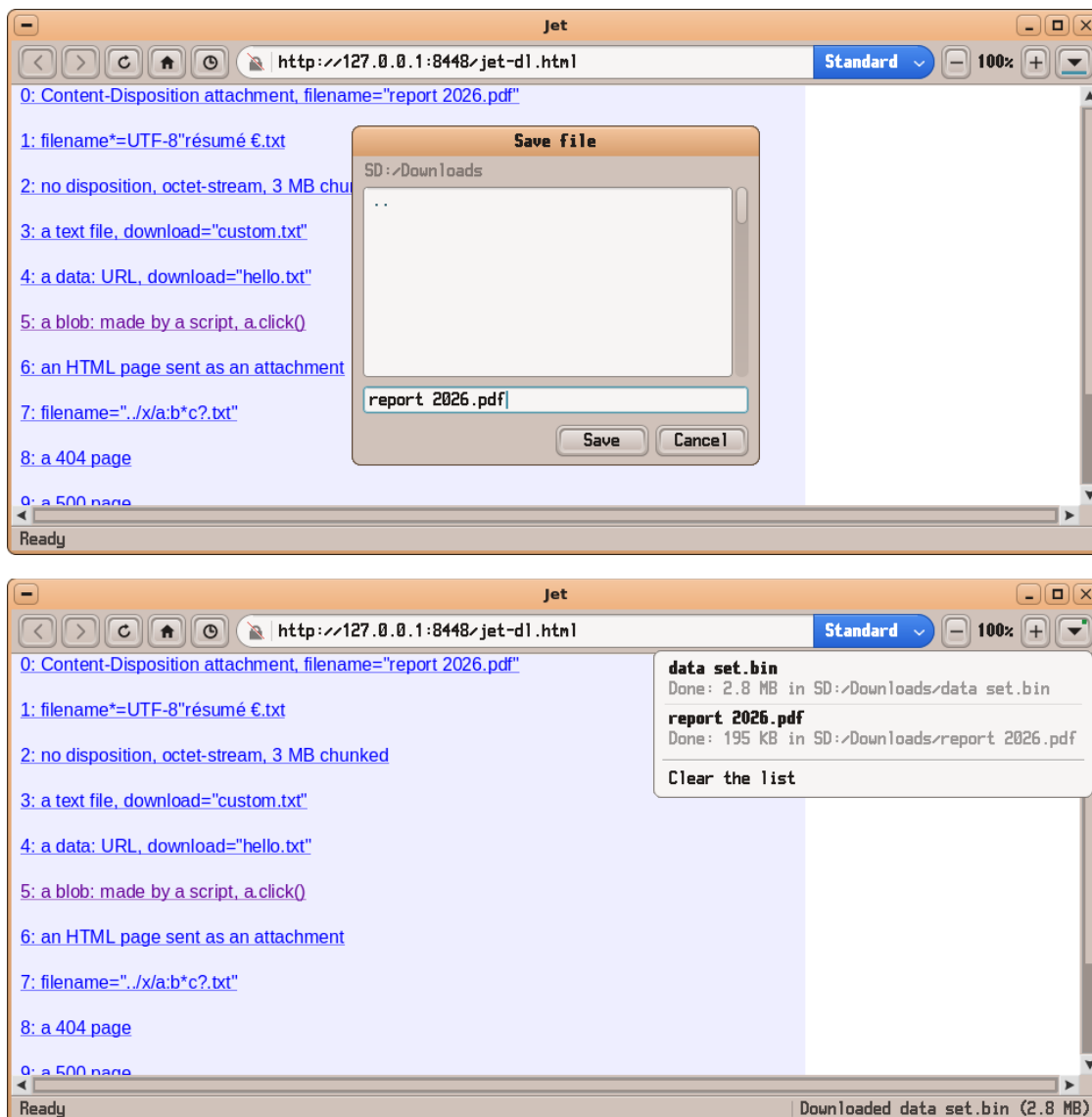
Downloads. NetSurf's core makes a **download context** (`desktop/download.c`) for a response not shown: a type it does not display (hlcache's `CONTENT_MSG_DOWNLOAD`), and now also a page load answered **Content-Disposition: attachment** whatever its type (hlcache_migrate_ctx, for a navigation that may download), and a **link with a download attribute** (`onyx_link_download`, `html/interaction.c`: the nearest `<a>` of the box clicked; `BW_NAVIGATE_DOWNLOAD`, the attribute's value as the name hint -- `download_onyx_hint`, taken by the next context; a Content-Disposition filename wins, as the spec says) -- a script's `a.click ()` too (dom.js activate: `N.download (url, name)`). A **blob**: link's bytes are the page's (html5.js keeps them): the click calls `__onyxBlobDownload (html5.js)`, which hands them to `N.download (bytes, name, type) -> onyx_download_bytes_hook`; a **data**: link goes through NetSurf's data: fetcher. The name: `filename*=charset'lang'%..` (RFC 6266 / 5987, UTF-8 or ISO-8859-1) first, else `filename=` (quoted or not), its last path segment; else the link's; else the address's last segment, %-decoded; "download" when none (data: URLs). The address bar goes back to the page's address (it showed the download's while asked).

The frontend's table (`frontends/framebuffer/onyx_download.c`): **create** answers at once -- the name made safe for the card (`dl_safe_name`: ASCII -- the card's names are code page 850 bytes on the Pi, UTF-16 on Windows; accented letters without accent, € "EUR", others `_` -- no `/ \ : * ? " < > |` nor control characters, no dots / spaces at the ends, 60 characters at most, the extension kept; one from the type when it has none: `.pdf, .zip, .txt...`) -- and the **Save dialog** is shown from the main loop a moment later (`framebuffer_schedule (0)`: never inside the core's callback): `SD:/Downloads` made when missing (`kapi_mkdir`), wtk's `FileDialog` in save mode (`onyx_chrome_save_dialog`: its name box focused, Enter = Save, Esc = Cancel -- wtk's `FileDialog` has those keys now -- and a question before a file is replaced). The bytes that came meanwhile are kept; NetSurf waits while the dialog is open, and the fetcher's HTTP/1.1 threads stop reading past 8 MB not taken (`ONYX_JOB_BACKLOG`, `onyx_fetch.c`): a big file waits in its socket, not in memory. **Cancel** aborts the fetch (`download_context_abort, _destroy`). **Saved**: the file is opened (`kapi_file_out`) and a **writer thread** (`jet:download`) writes the bytes as they come, 64 KB at a time -- the core's `data` callback only queues them (the card's writes never on the UI thread; 64 MB queued at most) --; **done**: the writer closes the file, then posts the end (`kapi_post`). Without threads (an older kernel) the UI thread writes. The core's context is destroyed at its `done / error`, or from the main loop when the user cancels.

Progress, cancel, the end. `onyx_chrome_downloads` hands the list (16 at most, the newest first) to the chrome: the **downloads' button** (`DlButton`, an arrow at the toolbar's right end, shown from the first download: a bar of the running ones' progress along its foot; a green / red dot: the last one's end) opens

its menu (DlMenu; also File ► Downloads...): each download's name and state ("1.2 MB of 2.6 MB (45%)", "Done: 2.8 MB in SD:/Downloads/...", "Failed: ...", "Cancelled"); a running one clicked -- after a question -- is **cancelled** (onyx_browser_download_cancel: the fetch aborted, the writer stopped, the partial file removed); "Clear the list" forgets the ended ones. The status bar's right part: "Downloading x: ...", "Downloaded x (2.8 MB)". At the end a desktop notification ("Download complete", notifyd -- when it runs; not on the PC builds), a failure a message box. The log: `ONYX-DOWNLOAD start / ask / save / done / failed / cancelled` lines. Jet Browser closed: the running downloads stopped, their partial files removed (onyx_download_finalise). The fetcher now passes a plain 200 response's Content-Length to the core (it dropped them all: the inflate makes an encoded one wrong), so a download's size is known.

Windows (§34): `SD:/Downloads` is the user's own `%USERPROFILE%\Downloads` (winkapi's `winpath`), else `Downloads\` beside `Jet.exe`.



Tests. `tools/tests/netsurf/dltest.sh` (`httpsrv.py`'s new `/dl/<name>?n=&cd=` -- n bytes, a Content-Disposition per `cd`, chunked or slow on demand -- and `/status/<code>; pages/jet-zoom.html`, `pages/jet-dl.html`): the zoom's keys, buttons and Ctrl+wheel in Chrome's steps, `devicePixelRatio / innerWidth`, a click mapped through 200 %, the zoom kept and applied at the next launch, the

percentage resetting it; the status bar's "Ready", the link under the pointer, 404 / 500 / a refused connection, hidden and shown (the page 22 px taller, kept); downloads: SD:/Downloads made, the dialog's name (filename=, filename*=, the address, download="...", data; a script's blob; an HTML page as an attachment, a dangerous name), the bytes exactly (a 3 MB chunked body), Esc: nothing, a slow download cancelled from the menu (the server sees the connection closed, no file left). The desktop simulator's stand-in kernel (fakekapi.cpp) got kapi_file_out (a file written in pieces in SIM_WRITES) and removes what an app wrote there. Logged for the bench (ONYX_HOST_SIM): ONYX-STATUSBAR (each repaint of the bar: its text), ONYX-CHROME (the band's places), ONYX-STATUS (the load's end); ONYX-ZOOM (a zoom changed) and ONYX-DOWNLOAD everywhere. jstest.sh's two clicks at a panel's foot moved up 22 px; uatest.sh's pill is left of the zoom control now; gputest.sh's nav-fragment compares the page only (the address fields differ). Screenshots: shots.sh jet (jet.png, jet-menu.png, jet-save.png, jet-downloads.png).

Not done. Pause / resume, a download's speed and time left, opening the file or its folder from the menu; a per-page (not per-site) zoom; the status bar's text in the page's font (the toolbar's bitmap font: Latin-1, a link's other characters as ?); Ctrl+Shift+= (US +).

39. NetSurf's dead code removed (2026-10-01)

Why: after the licence audit, the user kept Jet on NetSurf (GPL-2.0) but decided that the NetSurf code Jet does not use goes — out of the build, then out of the repository — with no behaviour change. The task and its method are in JET-DEAD-CODE.md: the dead code was found by the linker (-ffunction-sections, then --gc-sections --print-gc-sections) on the Pi build and on the PC bench's, every finding then checked on the three builds (Pi, PC bench, Windows) by building without it.

What went (three commits, one per step):

Step	What	Size
§4: never compiled	the other frontends (amiga, atari, beos, gtk, monkey, qt, riscos, windows); the Duktape backend, WebIDL, its generated bindings (user/netsurf/gen/duktape, gen-duktape.sh) and third_party/nsgenbind; javascript/none; the image handlers not built (jpegxl, nssprite, rsvg, svg — libsvgtiny —, video); fetchers/curl.c; the framebuffer's fb_search.c (empty stubs) and font_internal.c (the bitmap font: every build uses FreeType); NetSurf's tests, docs, own build	1 893 files, 600 500 lines, 22 MB; the Pi's netsurf.elf byte-identical

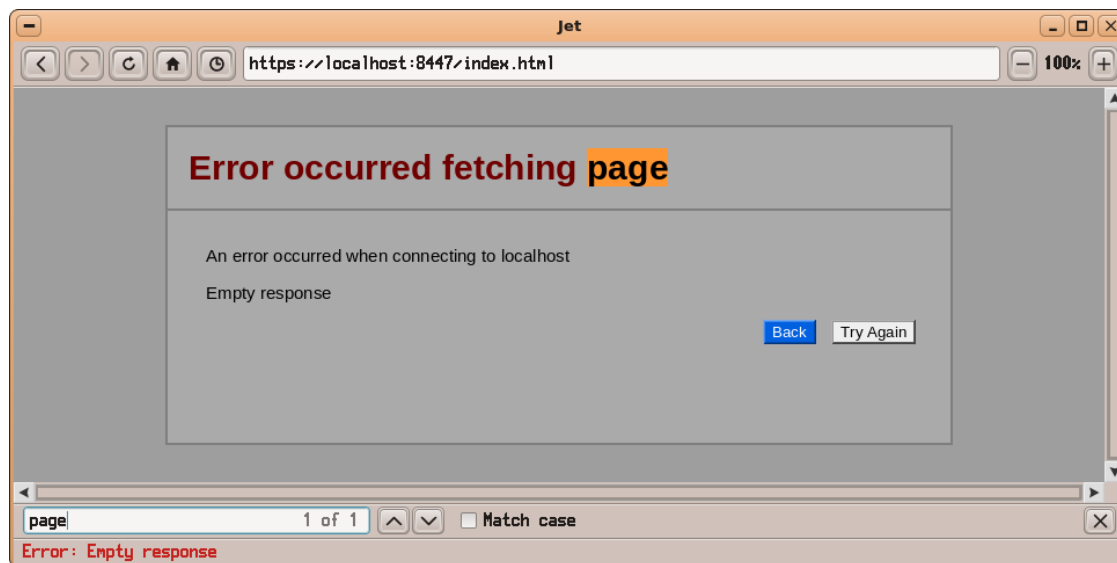
Step	What	Size
	(every Makefile*, tools/ but convert_font.c / convert_image.c); the resources no build stages (other languages' pages, other frontends' icons, the framebuffer res/ copies)	
§1: compiled, entirely dead	content/ fs_backing_store.c (onyx_cache.c is the store), desktop/save_complete.c , printing and save as PDF (print.c, printer.h, save_pdf.c, font_haru.c), desktop/mouse.c, utils/hashmap.c, the HTTP auth header parsers (utils/http/challenge.c, www-authenticate.c); generated: the internal bitmap font (font-ns-sans.c, 49 KB of data only font_internal.c read) and two toolbar bitmaps nothing draws (reload_g, history_image_g)	6 200 lines; Pi binary –129 KB
§2: alive only through calls that fed nothing	desktop/ global_history.c, hotlist.c, cookie_manager.c, page-info.c and treeview.c — NetSurf's tree view windows, never shown (Jet's History dialog, onyx_chrome.cpp, reads urlldb); the calls left (global_history_add, hotlist_update_url, cookie_manager_add / _remove, page_info_init / _fini) are static inline no-ops in their headers, so the call sites keep their upstream shape	10 900 lines; Pi binary –82 KB

The Pi's `netsurf.elf`: 9 439 232 → 9 228 400 bytes; Windows' `Jet.exe` (unstripped) 10 526 291 → 10 374 888. The upstream NetSurf files Jet compiles: 149 files / 123 200 lines (112 000 identical to upstream) → 135 / 107 600 (96 400 identical). The bench tests (`jstest`, `layouttest`, `html5test`, `css3test`, `httpstest`, `uatest`, `gputest`) and the jet screenshots are unchanged.

Kept: `desktop/search.c` and `content/textsearch.c` (find in page — pending the user's decision on Ctrl+F); everything about downloads; `tools/convert_image.c` (the toolbar bitmaps); `tools/convert_font.c` and `res/fonts/glyph_data` (no build runs `convert_font` any more, but `tools/fonts/gen_nssans.py` reads `glyph_data` for wtk's `ns-sans.fnt`); `COPYING` and the licence / credits pages. **Left for later** (JET-DEAD-CODE.md §3): the dead functions inside live files (`talloc`, `browser_window.c`, `urldb.c`, `content.c...`), once the sessions editing those files are done.

40. Find in page, copy and paste, the context menu (2026-10-01)

The user: "a Ctrl+F; and a Ctrl+C to copy the selected text or a selected image (to paste it into Paint)". NetSurf has a free text search in its core and a text selection; Jet Browser had no way to reach the first, and the second copied into a buffer of the framebuffer frontend's own that no other app could read.



The find bar (`FindBar`, `onyx_chrome.cpp`: 30 px above the status bar while shown -- the page that much shorter, a resize of the page as for the status bar): **Ctrl+F** (Edit ► Find in Page..., the context menu) shows it and focuses its field with its words selected; the field (`FindField`, a `LineEdit` -- the address field's editor, now shared: Latin-1, UTF-8 to the search and the clipboard) says "3 of 17" in its right end, or "No matches" with the field tinted red; ^ / v buttons, **Match case**, x. Keys: **Enter** / **Shift+Enter** in the field, **F3** / **Shift+F3** and **Ctrl+G** / **Ctrl+Shift+G** anywhere (key_event, before the page); **Esc** in the field or x closes the bar and clears the highlights (its words kept: the next Ctrl+F finds them again). A new page loaded while the bar is open is searched again (`onyx_find_page_loaded`, from the status bar's "load ended", without moving the view).

The search (`frontends/framebuffer/onyx_edit.c`, `onyx_browser_find`): `browser_window_search` with the words in UTF-8 -- a new search when the words, the case or the page changed (the same words again: cleared first, so a search starts at the first match), else a step. **Typed words are searched at once** while a search costs little; when the last one took more than 30 ms (a long page on the Pi) the next waits for a pause of 300 ms in the typing (`framebuffer_schedule`, replaced at each key). The core asks to scroll the current match into view (`CONTENT_MSG_SCROLL` -> `gui_window_set_scroll`, which put its top left corner at

the view's): taken while a find runs (`onyx_find_take_scroll`) and done as Chrome does -- nothing when the match is all in view, else the match in the view's middle (the zoom's scale applied: the core's rectangle is in CSS px).

content/textsearch.c, rewritten for the Pi. NetSurf kept the matches in a list, **each with a selection object** (`selection_create` + `selection_init`, a walk of the whole box tree to count its text, then `selection_set_position`, another walk to repaint it) -- a search for "e" on a long page was $O(\text{boxes} \times \text{matches})$; and **each text box painted looked through every match** (`content_textsearch_ishighlighted`) -- again $O(\text{boxes} \times \text{matches})$ per paint. Now:

- the matches are an **array in document order** (the box walk finds them in order; one out of order would be put in place), no selection objects;
- a painted text's matches are found by a **binary search** (`content_textsearch_onyx_ranges`: every match in the text -- NetSurf highlighted only the first --, the current one told apart);
- a new search repaints **the matches' boxes** (the content's `textsearch_bounds`: `box_coords`, no walk), or the whole content past 64 matches; a step repaints **the two current ones**; closing repaints what was highlighted. With GPU compositing (§25) these are damage rectangles like any: the band is painted again there (`findtest.sh` compares the composited frame with the CPU's);
- the search is **literal** (NetSurf's # and * wildcards are gone: "C#" finds C#) and, unless Match case, **ignores case and accents**: `fold_next` folds ASCII and Latin-1 letters to their lower-case base letter (é è ê ë É Ê -> e, ç -> c, ñ -> n, Œ -> œ), the typographic quotes to ' and "; the no-break space is a space either way. A match's bytes in the page may be longer or shorter than the typed ones (é is two bytes, e one): the match's length is the page's;
- a step **wraps around** at the ends (NetSurf stopped);
- **a layout made after the search** (the page changed, the window resized, the find bar shown) left the matches' boxes freed and their text offsets stale: `html_reformat` now bumps `content->textsearch.layout_gen`; the matches are found again at the next paint, step or count (`refresh_matches`, the current one kept at its place) and the bar told the new count (the core's `CONTENT_TEXTSEARCH_MATCH` -> the frontend's `gui_search_table.status` -> the count, from the main loop).

The highlight (`html/redraw.c`, `onyx_find_redraw`): the text drawn as ever, then each match in it on **yellow** (`#ffff00`), the current one **orange** (`#ff9632`, Chrome's), the text in black clipped to the match. A selection still wins over a match (as before). Text in a plain-text page (`textplain.c`) is searched and highlighted as before (its first match a line, through the old call).

Cost: the bench's 4000-paragraph page (88 000 "e"s): the search with its repaint requests 8 ms on the PC, so ~0.1 s on the Pi -- past 30 ms, the typing is then waited for; a paint's lookups are a binary search a text box.

The clipboard (`frontends/framebuffer/clipboard.c`): the kernel's (`kapi_clipboard_set` / `_get`, `CLIP_TEXT`, UTF-8; 64 KB at most: a longer copy is cut at a character's start) both ways -- the core's copy of a selection (`selection_copy_to_clipboard`: Ctrl+C = `NS_KEY_COPY_SELECTION`, already mapped by `onyx_key_to_ucs4`) reaches every app, and the core's paste into a form's field (Ctrl+V, `NS_KEY_PASTE`) reads what another app copied; Ctrl+X, Ctrl+A in fields as before. **Edit ► Cut / Copy / Paste / Select All** send those keys where the keys go (the address field, the find field, else the page). The log: `ONYX-CLIPBOARD text <n> bytes`.

The context menu (a right press on the page: `ptr_event` takes it -- the page does not get it, nor its release): `onyx_browser_context_menu` asks the core what is there (`browser_window_get_features`: a link, an object, a text field; `browser_window_get_editor_flags`: a selection, cut allowed) and the chrome shows wtk's `PopupMenu` (`onyx_chrome_context_menu`):

Under the pointer	Items
a link	Open Link, Save Link As... (<code>BW_NAVIGATE_DOWNLOAD</code> : §38's Save dialog), Copy Link Address
an image	Open Image, Save Image As..., Copy Image (greyed for an image without pixels: SVG), Copy Image Address
a text field	Cut, Copy (greyed without a selection), Paste (greyed without text on the clipboard), Select All
selected text	Copy
none of those	Back, Forward (greyed when not possible), Reload
always	Select All, Find in Page...

Paste or Select All on a field without a selection first clicks it (the caret there). The menu is driven by the arrows and Enter too. The log: `ONYX-CONTEXT` at `x,y flags=...`, ... `command n`.



Copy Image. The kernel's clipboard holds 64 KB -- not a picture -- but Paint's Paste already reads **an image file copied** (`clip_get_file` + `img_is_image_name` + `img_load`: the File Viewer's Copy). So the image's **decoded bitmap** (as the page shows it: a GIF's current frame, the alpha kept; `content_get_bitmap`, the frontend's nsfb surface, R G B A bytes) is written as a **PNG** (`img/pngsave.hpp`'s encoder, `onyx_chrome_save_png`: RGB when opaque) to **RAM:/jet/clip/image-1.png** or **image-2.png** in turn (the copy before stays readable while a new one is written; two files at most) and the clipboard set to that path (`CLIP_FILES`). In Paint, `Ctrl+V` pastes it as a floating selection. On **Windows** the pixels go to

the Windows clipboard as a **DIB** (`CF_DIB`, laid on white: `onyx_win_clip_image`, `pc/Jet/winkapi.cpp`) -- Paint, Word, a mail take it; Windows' text clipboard (`CF_UNICODETEXT`) was already the kapi's.

Also fixed: a page made shorter (the status bar shown again, now the find bar) left the horizontal scroll bar's arrows unpainted: the compositor's hole (\$25: the canvas it writes itself) still covered the old page's height when the frame after the resize copied the back buffer -- `gui_resize` now clears it (the next composite sets it again).

Tests. `tools/tests/netsurf/findtest.sh` (`pages/jet-find.html`, `pages/jet-find.png`; the stand-in kernel's clipboard is real now: `SIM_CLIP` -- its text at the start, or `files:PATH` --, `SIM_CLIPFILE` -- each copy written there --, `SIM-CLIPBOARD type= len=` logged): the bar opened, the words found as typed ("c" 9, "cafe" 5: Café, cafe, CAFE, café, cafeteria), the highlights' pixels (yellow, orange), Enter / Shift+Enter / F3 / Shift+F3 around the ends, Match case (2), `a*b` and `#` literal, `l'ete` finding l'été, a match 3000 px down scrolled into view, Esc clearing, Ctrl+F finding the kept words again, the composited frame = the CPU's, a 4000-paragraph page's time; a line selected by a drag + Ctrl+C (its exact UTF-8), the menu's Copy, Copy Link Address, Copy Image (the PNG's pixels = the image's, alpha included; `image-1` / `image-2` in turn), **Paint** (the desktop simulator's build) pasting it with Ctrl+V, the menu's Paste into an unfocused field, Ctrl+V, Ctrl+A + Ctrl+C and Ctrl+X in a field. Screenshots: `shots.sh jet` (`jet-find.png`, `jet-context.png`).

Not done. A search across a tag (`foo <b>bar</b>`: NetSurf searches each text box) and in a page's frames; a new search starts at the page's top (Chrome: after the view's top); the selection is not set to the current match on Esc; double / triple click to select a word / a line (NetSurf has neither); a drag past the end of a shrink-to-fit block's line selects nothing more (NetSurf's `box_pick_text_box` looks only inside the line's container); Copy Image of an SVG; Ctrl+C on a clicked image (Chrome does not either).

41. facebook.com on the desktop: the frozen login screen, fixed dialogs, the network (2026-10-01)

The reports (the Pi, `www.facebook.com` with the toolbar's pill on "Desktop": Chrome on Windows' User-Agent): the cookie dialog ("Autoriser l'utilisation des cookies de Facebook sur ce navigateur ?") showed but the wheel scrolled nothing; its Accept button worked, late; "a script makes the window lag" on the login screen; after logging in, "Jet eats the Pi's network" (the remote desktop and telnet lag).

Reproduced on the PC bench (real network, `SIM_REALNET=1`, `facebook.com desktop` in the build's `data/site-modes`, 1024 x 768): the bench's network leaves from the US -- Facebook shows no cookie dialog there (the page's `InitialCookieConsent` config: `shouldShowCookieBanner false`) --, so the dialog was rebuilt from Facebook's own code (its modules read on the bench: Comet's `BaseCometModal`, `BaseDocumentScrollView`, `BaseDialog`, `FDSDialogPage`, `BaseScrollableArea`) as test pages; the login screen itself was measured live.

The frozen login screen: `offsetParent` (the cause of the three first reports)

The log showed JS `timer: InternalError: interrupted at get offsetParent` and `ONYX-PERF js:call timer 60001363 us`: one of Facebook's timers (its "visual completion" tracker, `VisualCompletionUtil`) climbs the offset chain -- `for (r = e; r && r.offsetParent; ) { n += r.offsetTop; r = r.offsetParent; }` -- and `offsetParent` was **always the body, the body's own included**: the loop never ended, and ran until the script time limit (60 s). Meanwhile the window's events are kept for after the script (\$30, "a responsive window"): the wheel scrolled nothing, a click (Accept) was handled a minute later, the window lagged. It runs after the load and after an interaction, so the dialog met it on the Pi.

`offsetParent`, `offsetTop`, `offsetLeft` are now CSSOM View's (`N.offset`, `qjs.c n_offset`, from the boxes): the nearest positioned ancestor, else the body (a `td` / `th` / `table` for a static element); **null** for the body, the root, a `position: fixed` element and one without a box; the offsets the border box's from that parent's padding box (from the document's origin when it is the body or none), the scrolls between them left out. Same answers as Chromium on <pages/js-modal-doc.html>. **Measured** (the login page, PC): its longest timer 60 s (interrupted) -> 87 ms; an idle login page 20 s after the load: 1.5 % of a core (0.9 % for an empty page).

Fixed dialogs over a scrolled page (the wheel, the clicks, the rectangles)

§37 lays `position: fixed` boxes out at the scroll offset 0 and paints them moved by the scroll; everything else still used where they are laid out: once the page was scrolled, the **hit test** missed a fixed dialog (the wheel scrolled the page under it, clicks and fields went through it), the **redraws** of a scroller or a text field in it went to the wrong place (an inner scroller scrolled but was not painted again), `getBoundingClientRect` / `elementFromPoint` were off by the scroll. Now (`html_box_fixed_shift`, `html.c`: the viewport's scroll for a box in a fixed one): the hit test (`onyx_hit_box`) finds fixed boxes where they are painted, the pointer's boxes (`get_mouse_action_node`), the scrollbar and text-field drags, the caret, `html__redraw_a_box` (a scroller's scroll, a control's change), the text fields' redraws, the wheel into an iframe / object, and the scripts' rectangles (`n_rect`) add it. The wheel then takes the innermost scroller under the pointer that can still scroll, in a fixed dialog too, composited or not (§5's order, unchanged).

Facebook's desktop dialog (<pages/js-modal-doc.html>): when a Comet modal opens, the page under it is "detached" -- a fixed, viewport-sized `overflow: auto` view, its scroll bar hidden (`scrollbar-width: none`), scrolled to where the document was -- and the modal's content becomes the document (a flex column, `min-height: 100vh`, the dialog centred by an anchor with `pointer-events: none`): a dialog taller than the window scrolls **the document**. That works in Jet Browser (the wheel over the dialog scrolls the document, the page under it keeps its place). Jet Browser draws the detached view's scroll bar (`scrollbar-width` is not computed by libcss: §14's grammars keep it without effect) -- the "scroll bar on it" seen on the Pi.

The network: what a page does, held back when it loops

What Facebook does once logged in could not be watched here (no account on the bench; no credentials used). What was added, for every page:

- **ONYX-PERF net:minute** (the perf log: the file `SD:/apps/jet.app/perf`, or `NS_PERF=1`; on the Pi it goes to the kernel log, `kmsg`): a line a minute while anything happened, and one when Jet Browser closes -- the requests ended (by scripts, failed, retried), KB in (and KB/s) and out, the connects (failed, held back), the sockets open now (downloads, kept alive, HTTP/2 -- each with its host and streams), the WebSockets and EventSources open (each with its host and KB in / out), their connects and throttled ones, and the five hosts most requests went to. Facebook's login page, PC: 56 requests (4 by scripts), 2563 KB in the first minute, 2 requests the second, nothing after (no line).
- **Reconnections held back** (`onyx_ws.c`, as Chrome delays an endpoint that keeps failing): a WebSocket or EventSource that failed, or ended within 10 s without a message, counts against its host; from the third in a row each new connection waits 1, 2, 4 ... 60 s (`net:ws-throttle` in the log); one that lasted or brought something clears the count. A page re-opening a refused socket at once made a DNS + TCP + TLS handshake a turn for ever.

- **Failing hosts held back** (`onyx_connect`, every download and socket): from the third failed connect in a row to a host, its connects within 1, 2, 4, 8 s of the last failure fail at once, without the network (`net:tcp ... held back`); a connect that works clears it.
- **A hidden window's scripts**: the window minimised or covered (§32: its timers already slowed to a second), the scripts' requests (fetch, XHR) start **one a second** (polls, prefetches, beacons; `navigator.sendBeacon` sends nothing anyway).
- **Idle kept-alive connections closed after 10 s** (a 10 s timer): they were closed only when the next request came -- a page gone quiet held its sockets, of the kernel's 16 for every app (the remote desktop's and telnet's included).

To see what a page does on the Pi: create `SD:/apps/jet.app/perf`, browse, read `kmsg's net:minute` lines (`ONYX-PERF js:call` lines show the scripts' long turns).

Tests: `jstest.sh's js-fixed-scrolled.html` (a fixed dialog over a page scrolled 500 px: its rectangle, `elementFromPoint`, a click into its field and typing, the wheel scrolling its panel and not the page, its button) and `js-modal-doc.html` (the detached page and the document-scrolling dialog, the wheel, Accept after the scroll, the `offsetParent` chain ending, `offsetTop's` sum = the page position, null for the body / a fixed element: Chromium's answers); before the fix the first failed all but its `offsetTop`, the second looped. `nettest.sh's net-retry.html` (a refused WebSocket re-opened at once: 6 connections in 6 s, not hundreds; a fetch to a closed port retried at once: 6 connects, ~170 held back; the `net:minute` line) and `net-hidden.html` (a poll: paced when the window is hidden, ~16 a second shown).

Not done: `scrollbar-width / ::-webkit-scrollbar { display: none }` (a libcss property); `pointer-events: none` (not computed then: the hit test took such overlays; Comet's anchors and scroll thumbs are inside what they cover, so it did not matter here -- computed since, §42); images loaded lazily (`loading="lazy"`, off-screen images: Jet Browser fetches every image of the page -- a feed's pictures are probably what loads the network once logged in); a bandwidth cap.

42. Wikipedia's search box: the stacking order, forms a script made, pointer-events (2026-10-01)

The user, on `fr.wikipedia.org's` main page in the Mobile version (Minerva) -- and the same on the Desktop one (Vector 2022): the suggestions list under the search field "has no background" (the page's text through it), and Enter in the field does nothing. Both skins' search is Codex's **TypeaheadSearch**, a Vue 3 app: its form, field and menu are made by the script. Reproduced on the PC bench with the real site (`site-modes: wikipedia.org mobile / desktop`; the suggestions come from `/w/rest.php/v1/search/title`, which answered 429 now and then -- Wikimedia's rate limit for the bench's address, not Jet's doing). Four causes:

1. The stacking order (CSS 2.1 appendix E): a z-index inside a z-index auto box. The menu (`.cdx-menu, position: absolute; z-index: 50`, its background white) is in a relative wrapper with `z-index: auto` (`.cdx-search-input__input-wrapper`), in the page's stacking context (Minerva's `position: relative; z-index: 0` page box); the page's content after it is in relative boxes with `z-index: auto`. Jet's layers (§21, `redraw.c's onyx_layer_*`) treated **every positioned box as a stacking context**: the wrapper's put-off boxes were sorted and painted with the wrapper, in the tree's order among the z-index 0 boxes -- so the later relative content was painted over the menu (its transparent background over the menu's white, its text over the rows). Now `html_redraw_layer_context(box)` says whether a put-off box is a stacking context (a z-index set, `position: fixed / sticky`, a compositing group: opacity, transform, filter...); a positioned box with z-index auto is not: `onyx_layer_paint_in` paints its put-off boxes with z-

index 0 / auto right after it (the tree's order), and **gives those above 0 to its stacking context** (`onyx_hoist_*`), which sorts them in with the boxes it still has to paint. The tree's order among equal z-indexes is kept as nested intervals (`struct onyx_layer_key`: a box put off inside another gets a sub-interval of it, between it and the next one). The hit test (`interaction.c's onyx_hit_layer`) does the same, so the clicks go to the menu's rows, not to the page under them. No cost for a page without such boxes (the hoist list stays empty; an insertion is a short move).

2. Enter in a field of a form a script made. NetSurf's forms (`html->forms`) are the parser's, read once when the document is made, and libdom's form owner of a control (`dom_html_input_element _get_form`, the form's `elements` collection) was only the parser's association: a form and a field created by `createElement` (Vue, React) had no `struct form`, the field's control was an orphan, and Enter (`box_textarea.c`) had no form to submit. Now:

- **libdom** (`src/html/html_form_element.c _dom_html_form_owner`): a control without the parser's association has its **nearest ancestor form** as its owner -- for `get_form` (input, button, select, textarea) and the form's `elements` (so `form_dom_to_data` sends the script's hidden fields too, `title=Spécial:Recherche`). (Not done: the `form="id"` attribute.)
- **NetSurf** (`html/forms.c, html_forms_control_for_node`): a control in a form the parser did not see gets one made then (`onyx_form_for`: its action made absolute, the document's encoding, added to `html->forms`, the node referenced: `struct form's new node_ref`, unreferenced by `form_free`); an orphan control put in a form since it was made joins it at the next rebox.
- **The implicit submission** (`box_textarea.c`, HTML 4.10.21.2): Enter in a text field clicks the form's **default button** (its first submit button: a `click` event the page may prevent, then the button's `name=value` sent with the form), else sends the form; the `submit` event first, as before. On Wikipedia: `/w/index.php?title=Spécial:Recherche&search=paris`, which redirects to the article.

3. Object.prototype.toString of the DOM's objects. Each interface's prototype now has its `Symbol.toStringTag` (`dom.js's tagInterfaces`, `html5.js` for its own: every constructor the two files add to the global object): `[object HTMLInputElement]`, `[object Event]`... It was `[object Object]`, so **Vue 3's reactive() wrapped the elements** it keeps in its state in Proxies (it leaves alone what `toRawType` does not call `Object / Array / Map / Set`): the menu's code then handed a Proxy to the natives -- `TypeError: not a node` (its `scrollHeight`, `getComputedStyle`) at each keystroke.

4. pointer-events (Desktop): MediaWiki's notification area (`.mw-notification-area-overlay`: absolute, the whole page, z-index: 9999, `pointer-events: none`) took every click on the page's top -- the search field could not be focused. libcss now computes **pointer-events** (inherited; `auto | none`, SVG's values read as `auto`: `parse/properties/onyx_svg.c`, `select/properties/onyx_svg.c`, `select_config.py` and the autogenerated headers made again, `onyx_propbits.py`: a paint-only change; `CSS_PROP_POINTER_EVENTS`, `css_computed_pointer_events()` in `computed.h`, `enum css_pointer_events_e` in `properties.h`). The hit test passes over a box with `pointer-events: none` (`onyx_hit_visible`, as `visibility: hidden`): its descendants with `auto` still take the clicks. `getComputedStyle` answers it.

And the menu's footer ("Rechercher les pages contenant ...": absolute in the menu, under the menu's list, a static box with `overflow-y: auto`) was cut away by the list's overflow, and gave the list a scroll bar. An absolute box is clipped by the overflow of its containing block and of the containing block's ancestors, not by a static scroller between them: the redraw keeps the clipping boxes being painted with the clip they were given (`onyx_oclip`), and an absolute box put off takes the clip from outside those between it and its containing block (`onyx_oclip_escape`); the hit test looks, under a static clipping box the point is outside of, for its absolute boxes only (`onyx_hit_escapes`); the layout marks the boxes

between an absolute box and its containing block (`HAS_ABS_OUT`, `box.h`) so their scroll extent leaves it out (`layout_update_scroll_extent`) and their descendant bounds take it in.

GPU compositing. The same with `NS_GPU=0` and `NS_GPU=1` (the compositor composites what the redraw painted: the order was the redraw's).

Tests. `tools/tests/netsurf/typeaheadtest.sh` (new; local pages, no network): `pages/stack-typeahead.html` -- the menu's pixels over the later content, the footer's, no scroll bar, the content beside, a z-index 999 box inside a z-index 1 context under a later z-index 2 one, the clicks (the menu's, the footer's, through a `pointer-events: none` overlay, on its `auto` part), `getComputedStyle(...).pointerEvents`, with and without the GPU compositor; `pages/form-scripted.html` -- a form and its field made by the script: Enter fires `submit` and sends the GET with the hidden field; with a default button, Enter clicks it and sends its value; `Object.prototype.toString` of an input, the document, an event. Each check fails on the build before.

Not done. An absolute box escaping a scroller does not stay put when the scroller is scrolled (it moves with the scroller's content); the `form` attribute (`<input form="id">`); a form with two text fields and no submit button is still submitted by Enter (HTML: only a lone field); the menu's list shows a scroll bar of its own when its ten rows are taller than it (Chrome's overlay scroll bars take no room). Checked on the live site: the list opaque (Mobile and Desktop), Enter (both), the footer shown and a row clicked opening its article (Mobile).

43. XML documents: XHTML, XML with CSS or XSLT, SVG documents, XPath (2026-10-01)

Frames on a resource Jet does not show: an empty document, their load event

Acid3's test 65 makes seven frames (`svg.xml` in an `<iframe>` and an `<object>`, `empty.xml`, `empty.html`, `xhtml.1-3`) and test 69 waits -- up to 5 s, retried every 10 ms -- for their seven `load` events. Before, the `<object>` on `svg.xml` fired `error` (PlutoSVG refused the file: a `<?xml-stYLESHEET?>` before its root), so Acid3 sat on test 69 for its whole timeout (on the Pi the score stayed at 69 for seconds). Now:

- **A frame whose resource has a type no handler takes** ("UnacceptableType": an unknown binary, before §43's XML handler `text/xml...`) is loaded with **about:blank** instead of the fetch error page (`onyx_frame_error_blank`, `desktop/browser_window.c`): its `contentDocument` is an empty HTML document of its parent's origin, its element's `load` event comes once that is loaded (a handler reading `contentDocument` finds it) -- as in the other browsers, where the frame stays empty (or the file is downloaded) and `load` fires. The status bar no longer shows the frame's error.
- **A frame showing an image or a text** (no document of its own in Jet; Chrome makes an image or a text document) gives a same-origin script an empty document as its `contentDocument` (`N.frameShown`, `qjs_frames.c`; kept per element while the frame shows that resource).
- **PlutoSVG skips any processing instruction** before the root, not only `<?xml ...?>`.
- `getElementById` on a document other than the window's (`createHTMLDocument`, `DOMParser`, a frame's) searches that document (it searched the window's).

Acid3's own time (its log's "Total elapsed time", first run on the PC bench): **5.56 s -> 0.52 s** (`acidtest.sh` prints it). `xmltest.sh`'s `frameload.html`: the six frames' `load` events (an SVG in an `<object>` and an `<iframe>`, an unknown type, a 404, a malformed XML, XHTML as `text/xml`).

XML documents: expat into libdom (html/onyx_xml.c)

Expat 2.7.1 (MIT, `third_party/expat-2.7.1: lib/` only, unchanged, its `expat_config.h` written by hand -- no entropy source on the Pi: the caller sets the hash salt; built by the three builds, `libexpat.a` on the Pi) parses the XML documents into **libdom**, as the HTML parser's binding does for HTML: an element is made with its attributes, then appended to its parent (the mutation events NetSurf listens to see a `<link>`, a `<style>`, an `<img>` as in HTML), the text gathered and appended once a markup event comes. Expat runs in namespace mode; the namespace declarations become the `xmlns` attributes of the element that declares them, as in the browsers' DOM; CDATA sections, comments, processing instructions and the doctype are nodes. The entities: the five, the internal subset's (expat, with its billion-laugh protection), and -- in a document with an external DTD it does not read (XHTML's doctypes) -- **HTML's named character references** (` `; `©`; libhubbub's table, as Chrome reads them). An encoding expat does not know (windows-1252, ISO-8859-x...) is read through `libparserutils'` single-byte codecs. The parse is **incremental** (the bytes as they arrive) and **suspended while a script is fetched** (`XML_StopParser` resumable: the bytes that come meanwhile are queued, `onyx_xml_parser_resume` from `script.c` when the script ran).

The document is an **HTML document underneath** (`dom_html_document`: NetSurf lays it out as one) **of an XML kind** -- libdom's `dom_html_document_set_xml_kind` (1 XML, 2 XHTML, 3 SVG; a new field, libdom's header changed): the names keep their case (an XHTML element is still of its HTML type -- `<p>`'s `tagName` is `p`), an element in no namespace is none of HTML's types (an XML document's `<title>` is not HTML's), an XHTML document's `createElement` is in the HTML namespace. Elements of the XHTML namespace are HTML elements: layout, CSS, forms, scripts work as in HTML.

The types (`html.c`'s handler): `application/xhtml+xml` (parsed as HTML before), `text/xml`, `application/xml`; for a window (the top one or a frame: `HLCACHE_RETRIEVE_ONYX_DOCUMENT`, set by `browser_window.c`) `image/svg+xml` becomes `application/x-onyx-svg-document` (an XML document; an `<img>`, a CSS image or an `<object>` still get the SVG image of §12) and an `+xml` type no handler takes (`application/rss+xml`, `application/atom+xml`...) or `text/xml` an XML document.

What an XML document becomes (html_onyx_xml_complete)

Once parsed, by its root element's namespace and its prolog's `<?xml-stylesheet?>`:

- **XHTML** (the root in `http://www.w3.org/1999/xhtml`) and **SVG** (in `http://www.w3.org/2000/svg`): the document itself, laid out; their `<script>`s ran at their end tag (XHTML's and SVG's: Acid3's `xhtml.1` notifies its parent; `xhtml.2`, malformed, stops before its script; `xhtml.3`, in another namespace, runs none). A standalone SVG is its root's replaced box (§12's inline SVG, drawn at its own size from 0,0: the root's "fill the viewport" stretch is not for a replaced root).
- **An XSLT style sheet** (`type="text/xml"`, `application/xslt+xml`, `text/xml`, `application/xml`; same origin or `data:`): fetched (llcache), the transform run (`xslt.js`, below, in the document's realm, which goes after it), and **its result becomes the document**: parsed by the HTML parser (output method `html`, or text in a `<pre>`) or by expat again (`xml`: an XHTML result is XHTML, its scripts run). The page's address stays the XML file's.
- **CSS style sheets** (`type="text/css"` or none): fetched as a `<link>`'s (`html_css_process_pi`, the processing instruction as its node) and the XML elements laid out **with them only** -- HTML's UA sheet left out (`html_css_onyx_xml_mode`: elements are inline unless the sheet says otherwise; the root is a block), and the root's children's backgrounds are their own (the `<body>` propagation of the root background is for a `<body>` only: `redraw.c`).

- **Otherwise: the tree view**, as Chrome's -- "This XML file does not appear to have any style information associated with it. The document tree is shown below." and the document as indented coloured markup (elements, attributes with their prefixes, namespace declarations, text, CDATA, comments, processing instructions; an element with one short text on one line), made as an HTML page ([onyx_xml_tree_view](#), iterative: no recursion on a deep document) and parsed in place of the XML document.
- **A well-formedness error**: the parse stops there and **Chrome's error box** comes first -- "This page contains the following errors: error on line 6 at column 18: mismatched tag / Below is a rendering of the page up to the first error." ([onyx_xml_error_banner](#), a `<parsererror>` in the XHTML namespace with Chrome's style) -- above XHTML's page so far, above the tree view of the rest; DOMParser's malformed document is the DOM Parsing standard's (as Firefox): emptied, its root a `<parsererror>` with the message, line and column.

A frame's `contentDocument` is the document shown: the XHTML / SVG / XML + CSS document itself, the XSLT's result, the tree view's page (Chrome gives the XML document there: a script reading a frame's XML through the tree view sees its HTML).

Scripts and XML ([quickjs/qjs_xml.c](#), [html5.js](#))

- **DOMParser** with `text/xml`, `application/xml`, `application/xhtml+xml`, `image/svg+xml`: `expat`'s `parse` (`N.parseXML`; the small JavaScript parser of before is gone); **XMLHttpRequest**'s `responseXML` through it.
- **XMLSerializer**: the XML serialization of the DOM Parsing standard -- namespaces declared where they are needed (an SVG made by script gets its `xmlns`), `<br />` for HTML's void elements, `<e/>` for XML's empty ones, CDATA, processing instructions, doctypes.
- **document.implementation.createDocument**: an XML document of its root's kind (`contentType` `application/xml`, `application/xhtml+xml` for XHTML, `image/svg+xml` for SVG); `document.contentType` by the kind; `createElement` keeps the name's case in an XML document; `createCDATASection`; `write` / `writeln` / `open` / `close` throw `InvalidStateError` there.
- `getElementsByTagNameNS`; an attribute keeps its prefix (libdom: `Attr.name` is `x:a`, `getAttribute('xlink:href')` finds it); `lookupNamespaceURI`.
- **<object> / <embed>'s contentDocument and getSVGDocument()**: the object is drawn as an image (§12); its document is its resource parsed (same origin), kept while the resource is the same; an `<iframe>`'s `getSVGDocument()` is its SVG document.
- **The SVG DOM** ([html5.js](#)): `SVGGraphicsElement`, `SVGGeometryElement`, `SVGRectElement`, `SVGCircleElement`, `SVGEllipseElement`, `SVGLineElement`, `SVGPathElement`, `SVGPolylineElement`, `SVGPolygonElement`, `SVGGElement`, `SVGUseElement`, `SVGImageElement`, `SVGTextElement` / `SVGSpanElement` (`SVGTextContentElement`: `getNumberOfChars`, `getComputedTextLength`, `getSubStringLength`, `get{Start,End}PositionOfChar`, `getExtentOfChar` from the element's box), the gradients, `<stop>`, `<clipPath>`, `<mask>`, `<pattern>`, `<marker>`, `<symbol>`...; their geometry attributes as **SVGAnimatedLength** (`rect.width.baseVal.value`, the units: number, %, em, ex, cm, mm, in, pt, pc; `newValueSpecifiedUnits`, `convertToSpecifiedUnits`); `SVGSVGElement`'s factories (`createSVGPoint`, `createSVGMatrix`, `createSVGLength`...) and its animations' clock (`getCurrentTime`, `setCurrentTime`, `pauseAnimations`). An element in no namespace of an XML document is an `Element` (`TAGS['*xml']`).

XPath 1.0 and XSLT 1.0 (quickjs/xslt.js)

A library of its own, in JavaScript, **loaded on demand** (`N.loadXslt`: the first `document.evaluate`, `XSLTProcessor` or `<?xml-stylesheet type="text/xsl"?>` -- not in every realm's prelude; its bytecode in the code cache as the preludes):

- **The data model**: the tree built **in one call** from the DOM (`N.xmlFlat`: the subtree in document order, 7 slots a node, attributes after their element) -- each node its document order as a number, every tree a range of its own (the source, the style sheet, the result tree fragments sort together); the namespace declarations apart from the attributes. In an HTML document the HTML elements' names match in any case (`//DIV` finds the `<div>s`).
- **XPath**: the tokenizer with the 1.0 disambiguation rules (`*` and `and` / `or` / `div` / `mod` as operators or names), a recursive-descent parser, the 13 axes (the namespace axis empty), name / node-type / processing-instruction tests, positional and boolean predicates (a number literal taken at once), unions; node-sets unique in document order (a single context node's step needs no sort; `//x` is one descendant step, not descendant-or-self then child); the 27 core functions; numbers printed without exponent -- `document.evaluate`'s as Blink's (the shortest that reads back), XSLT's as libxslt's (15 significant digits: `sum` of 10.90 + 9.90 + 9.90 is 30.7).
- **document.evaluate**, **createExpression**, **createNSResolver**, **XPathEvaluator**, **XPathExpression**, **XPathResult** (the ten result types; iterators invalidated by a change of the document; `SyntaxError` / `NamespaceError` / `TypeError` as the browsers); attribute results are their `Attr`.
- **XSLT**: templates and their match patterns (default priorities, modes, import precedence, the last of equal ones; the templates indexed by the name their pattern's last step tests), the built-in rules; `apply-templates` / `call-template` / `apply-imports` with `with-param`, `for-each`, `if`, `choose`, `sort` (text / number, order, case-order, several keys, stable), `variable` / `param` (global ones lazily, cycles caught; result tree fragments usable as node-sets, `exsl:node-set` / `msxsl:node-set`), `value-of`, text and `disable-output-escaping`, `copy`, `copy-of`, `element`, `attribute`, `attribute-set` / `use-attribute-sets`, `comment`, `processing-instruction`, `number` (single / multiple / any, count, from, value, the formats 1 01 a A i l, grouping), `message` (terminate), `fallback`; `key()`, `format-number()` with `decimal-format`, `generate-id()`, `current()`, `document('')`, `system-property()`, `element-available()`, `function-available()`; `strip-space` / `preserve-space`; output (methods `xml`, `html`, `text` -- `html` by default for an `<html>` result --, the doctype, `omit-xml-declaration`); literal result elements with their attribute value templates and namespaces (`exclude-result-prefixes`); a literal result element as the whole style sheet. The result serialized (XML with its namespaces declared, HTML with its void elements) or built into DOM nodes.
- **XSLTProcessor**: `importStylesheet`, `transformToFragment` (nodes made by the owner document: HTML elements in an HTML document), `transformToDocument` (an HTML document for the `html` method, an XML one for `xml`), `setParameter` / `getParameter` / `removeParameter` / `clearParameters` / `reset`; an error of the transform is reported (console) and gives `null`, as Chrome.

Acid3: 100 / 100

The six subtests of §37 pass: 69 (the seven frames' loads; the SVG frame's `<text>` removed), 74 (`getSVGDocument()` of the `<iframe>` and the `<object>`), 75 (`SVGRectElement`'s `width`), 77 (`getNumberOfChars`), 79 (the `<object>`'s document emptied and given an `svg:svg`), 80 (`xhtml1.1`'s script ran, not `xhtml1.2`'s nor `xhtml1.3`'s). (The 2011 Acid3 commented SVG fonts, SMIL and `textPath` out of 75-79: only those DOM parts are tested.) `acidtest.sh`'s `ACID3_MIN` is 100; Acid2 is still identical to its reference.

The tests: `xmltest.sh`

`OUT=/tmp/nsbench PORT=8170 sh tools/tests/net surf/xmltest.sh` -- the pages of `pages/xml/` over `xmlsrv.py` (the Content-Type of the extension, or `?type=`): `frameload.html` (the frames' load events, above), `xmlframes.html` (23 checks through the frames' documents: the tree view, an `+xml` type, the error box's line and column, XML + CSS -- its computed colour, its elements no `HTMLElement` --, an SVG document and its DOM, an XSLT's result, XHTML, an `<object>`'s SVG document), `jspxml.html` (39 checks: `DOMParser`, `XMLSerializer`, `createDocument`, `document.evaluate` with every result type, namespaces, errors, `XSLTProcessor` -- sort, AVT, format-number, call-template, parameters, keys, number, element / attribute, copy-of, priorities), and the pages at the top, by their scripts' lines and their colours in the screenshot: `page.xhtml`, `bad.xhtml`, `data.xml`, `styled.xml`, `pic.svg`, `catalog.xml` (XSLT, html output), `list.xml` (XSLT, an XHTML result with its script).

The cost (PC bench, `NS_PERF`)

A 2.2 MB XML file (20 000 records, ~200 000 nodes): expat's parse is not what counts; its tree view takes 1.1 s to make and parse (`xml:tree` view) and 1.3 s to lay out (NetSurf's layout of ~100 000 lines); its XSLT (a `count(//book[price > 50])`, a sorted `for-each` of 2000 rows) 0.93 s (`xml:xslt`: the data model 0.47 s -- `N.xmlFlat` wraps no DOM node, a result's nodes are wrapped in one more walk, `N.xmlWrap` --, the transform 0.44 s), its page laid out in 14 ms; a `document.evaluate("count(//td)")` on the 4000-cell result 28 ms. Count 5-10x on the Pi. No step is quadratic: a node-set from one context node needs no sort, `//x[...]` (a predicate that is no number and asks no `position()`) is one descendant step, the templates are indexed by name, `key()` by value once a tree.

The Pi (not built here)

Expat is a new library: `make -C user/net surf builds third_party/expat-2.7.1/libexpat.a` (`net surf-app.mk` links `-lexpat`, compiles `qjs_xml.c` and makes `qjsngen/qjs_xslt_js.h` from `xslt.js`). **libdom changed** (struct `dom_html_document`'s new field in `src/html/html_document.h`, two functions in `include/dom/html/html_document.h`, `element.c`'s attribute prefix): rebuild `libdom.a` clean (delete its `.o`). PlutoSVG changed (the processing instructions): rebuild `libplutosvg.a`. Then `net surf-app.mk link` stage. Windows (`pc/Jet/jet.mk`) compiles expat with the rest.

Not done

- `xsl:include` / `xsl:import` (no synchronous fetch for `XSLTProcessor`; the top-level transform does not fetch them either: they are skipped), `document()` of another URI (empty), `xsl:namespace-alias`, `xsl:output`'s `indent` and `cdata-section-elements`, the namespace axis (empty); EXSLT beyond `node-set` / `object-type`.
- The tree view is static (no collapsing arrows) and is what a script sees of that frame.
- An SVG document without `width` / `height` is drawn at 300 x 150 (Chrome: the window's size); the SVG DOM's geometry is the attributes' (no `getBBox` beyond the box, no transforms in `getCTM`, no SMIL, no SVG fonts); an `<object>`'s SVG document is a copy (a script's change to it is not drawn).
- No DTD validation, no external entities (by design: as the browsers).

Windows: the certificates valid past 2038 (2026-10-01). Jet for Windows said Wikipedia's root, ISRG Root X2 (valid until 2040), had expired -- and so loaded none of `upload.wikimedia.org`'s images (the same chain). The certificates' dates are checked by `onyx_tls.hpp`'s own callback in seconds since 1970 (`x509_secs`, `clock_secs`), in a `long`: 64 bits on the Pi, **32 on Windows** (LLP64) -- a date past January 2038 wrapped negative, "in the past". They are `long long` now (checked with MinGW under Wine: 2040-09-17

was -2063514496, now 2231452800). The Pi's checks were right (its `long` is 64-bit): replayed there with the Pi's own mbedTLS libraries under qemu-aarch64 -- E6 -> ISRG Root X2, the bundle's root chosen, not the expired X1-signed copy.

44. Video and audio: `<video>`, `<audio>`, Media Source Extensions, the media library (2026-10-01)

The goal: YouTube in Jet Browser on the Pi 4 -- its pages, and a video that plays with its sound at a resolution the Pi decodes in software. **What is done:** the whole media path -- a media library shared with the Media Player (`user/av`), HTMLMediaElement, Media Source Extensions, MediaCapabilities, the Fullscreen API, native controls, the sound, A/V sync -- and the decoders YouTube serves except H.264 / AAC: **VP9 and VP8** (libvpx 1.15.2), **AV1** (dav1d 1.5.1), **Opus** (libopus 1.5.2), vendored and built for the Pi (libvpx's NEON, dav1d's AArch64 assembly), the PC bench and Windows (*The codecs* below). VP9 / AV1 + Opus play in `<video>` and through MSE, every frame bit-exact against the reference decoders. YouTube's pages and its player run, pick VP9 / AV1 + Opus from our answers and start streaming; **the video itself does not come yet:** Google's video servers answer the player's second streaming request with its head and no bytes (*YouTube* below: the transport is checked, the cause is on the server's side -- most likely the attestation Google asks of the player, or the bench's network, which Google already refuses the watch pages).

The media library (`user/av`, docs/03 "The media library")

Plain C, five layers, each usable alone (`user/av/av.h` is the reference):

- **Containers** (push parsers over a byte window; fed in stream order -- a network stream, MSE's appends -- or from where they ask, `av_demux_want`: a file, Range requests): **WebM / Matroska** (`av_mkv.c`: EBML, unknown-size Segment / Cluster, SimpleBlock and BlockGroup, the three lacings, Cues read where the SeekHead says -- a seek may read them first --, ContentEncryption = encrypted), **ISO BMFF** (`av_mp4.c`: plain files with the moov before or after the mdat -- its samples merged by offset, an mdat before the moov jumped over then come back to --, fragmented files and MSE's byte stream: moov / traf / tfhd / tfdt / trun, trex defaults, the first edit's media time, sidx for seeking; avcC, vpcC, av1C, esds, dOps, dfla, pcmC; encv / enca = encrypted), **WAV** (`av_riff.c`), **FLAC** (`av_flac.c`: frames cut where a header with a right CRC-8 follows a right CRC-16), **MPEG audio** (`av_mp3.c`: ID3v2 skipped, frames by their headers checked against the next, Xing / Info / VBRI for the duration and the seek table).
- **Codecs** (`av_codec.c`): the table of what is compiled in -- **built in:** FLAC (a decoder written from RFC 9639: fixed and LPC predictors, escaped Rice partitions, the stereo decorrelations, wasted bits), PCM (8 / 16 / 24 / 32-bit, float, A-law, mu-law), uncompressed I420 (the tests' video codec); **MP3** on minimp3 (the Media Player's `third_party/minimp3`, its public names renamed so that both link together); **on vendored libraries:** VP8 / VP9 (libvpx, `av_vpx.c`), AV1 (dav1d, `av_dav1d.c`), Opus (libopus, `av_opus.c`) with `AV_WITH_VPX`, `AV_WITH_DAV1D`, `AV_WITH_OPUS` (`user/av/codecs.mk`). `av_type_supported` answers `canPlayType` / `isTypeSupported` / MediaCapabilities from that table only (RFC 6381 strings: `vp09.PP.LL.DD` profile 0 / 8 bits, `av01.0.* 8 bits`, `avc1.*`, `mp4a.40.*`, `opus`, `flac`, `pcm...`), with "smooth" = within what the codec decodes in real time on the Pi (VP9 / VP8: 854 x 480 at 30 fps; AV1: 640 x 360 at 30 fps -- *Speed* below); a size past one and a half times that, an HDR `eotf`, `tunnelmode=true` or encrypted blocks: not supported (YouTube probes the largest size it may ask for with `width=` / `height=`).
- **Conversion:** `av_yuv.c` -- I420 to 32-bit RGB (BT.601 / BT.709, limited / full range), 16-bit fixed point; on AArch64 a **NEON** path (16 pixels a step: `vmull_u8`, saturating adds, `vqsrshrn`, `vst4q_u8`)

that is **bit-exact** with the C one (checked under qemu-aarch64); `av_resample.c` -- any rate and channel count to s16 stereo at the output's rate (Catmull-Rom; the playback rate; the volume; 5.1 / 7.1 folded down).

- **The store** (`av_store.c`): MSE's coded frames by source and track -- the coded frame processing (timestampOffset, sequence mode, the append window, a discontinuity waits for a random access point, frames of another coded frame group that new ones overlap removed with those depending on them), `remove()`, buffered ranges (a track's union, a source's intersection, the last range stretched at the end of the stream), a quota per source with eviction behind the playback position (else QuotaExceededError), the file mode (bytes fed at offsets, the demuxer's seek).
- **The player** (`av_player.c`): an **audio thread** (decodes, resamples, keeps the output's queue ~150 ms deep, corrects the clock from what is heard: the audio is the master clock), a **video thread** (decodes and converts four frames ahead, drops a frame already late without converting it, frames before a seek's target decoded and not shown), the **host's poll** (the frame due now, the clock, readyState, waiting / ended / seeked), the file mode (`av_player_open_file`: a reader thread ~30 s ahead, the file read through the kapi -- the Media Player's videos, done 2026-10-02).
FFmpeg (2026-10-02, `-DAV_WITH_FFmpeg`: `av_ffmpeg.c`, `av_lavf.c`, docs/03): every decoder and container -- built into the Media Player only (GPL-2.0+); Jet is GPL-2.0 too and could link it (H.264 / AAC for the sites that serve only those: `canPlayType` would say so) -- not done: Jet's size, its Windows build. The sound output is the kapi's (`kapi_sound_acquire / write / status / config`; released while paused -- what was queued stops at once); another app holding it: the clock is the wall's, the audio not decoded. Threads poll with naps (2..10 ms): nothing needs the kapi's events.

In Jet Browser

- `media.js` (compiled in, as `canvas.js`; run after `net.js`) and `qjs_media.c` (the natives `N.md*`: a player per element, its poll every 10 ms while playing / seeking / waiting, 40 ms until the first frame, 250 ms else). **HTMLMediaElement**: the load algorithm and the resource selection (`src`, `<source type>` children, `srcObject`; `setAttribute('src')`); an element the parser made is set up when its box is made -- `onyx_media_box_made` -- or at `DOMContentLoaded`), `play()` / `pause()` and their promises, `currentTime` (seeking), `duration`, `paused`, `ended`, `seeking`, `readyState`, `networkState`, `buffered`, `seekable`, `played`, `volume`, `muted` / `defaultMuted`, `playbackRate`, `loop`, `autoplay`, `preload` (none: nothing fetched before `play()`), `error` (`MediaError` 2 / 3 / 4), `canPlayType`, the events and their order (`loadstart`, `durationchange`, `resize`, `loadedmetadata`, `loadeddata`, `canplay`, `canplaythrough`, `play`, `playing`, `waiting`, `timeupdate` every 250 ms, `seeking`, `seeked`, `pause`, `ended`, `progress`, `suspend`, `emptied`, `abort`, `volumechange`, `ratechange`, `error`) and their `on...` properties; `videoWidth` / `videoHeight`, `getVideoPlaybackQuality()`, `requestVideoFrameCallback()`, `webkitDecodedFrameCount` / `webkitDroppedFrameCount`, `emptyTextTracks` / `audioTracks` / `videoTracks`, `addTextTrack`; EME refused (`setMediaKeys`, `navigator.requestMediaKeySystemAccess`: `NotSupportedError`); `new Audio(url)`.
- **A src that is not a MediaSource** is fetched with `fetch()` by **ranges of 1 MB** (`Range: bytes=`) from where the demuxer wants bytes, ~30 s ahead of the playback; a seek to where nothing came yet sends the demuxer there (WebM's Cues -- read first if needed --, MP4's sample tables) and drops the piece on its way; a server that ignores ranges (200): the whole body; `data:` and `blob:` URLs read whole.
- **Media Source Extensions**: `MediaSource` (`isTypeSupported`, `readyState`, `duration` -- the first initialization segment's, else `+Infinity`; `endOfStream`: the highest end buffered --, `addSourceBuffer` / `removeSourceBuffer`, `sourceBuffers` / `activeSourceBuffers`, `endOfStream(error)`, `setLiveSeekableRange`, `sourceopen` / `sourceended` / `sourceclose`),

SourceBuffer (appendBuffer -- parsed at once, its events in the next tasks: updatestart, update, updateend; QuotaExceededError thrown; a parse error or codecs not supported: error + endOfStream('decode') --, abort, remove, changeType, mode, timestampOffset, appendWindowStart / End, buffered, updating), SourceBufferList; URL.createObjectURL(mediaSource) (a blob: URL the element attaches to).

- **navigator.mediaCapabilities.decodingInfo** (file / media-source): supported, smooth (the Pi's limit above), powerEfficient: false (no hardware decoder) -- YouTube's adaptive bit rate stays at what decodes smoothly.
- **The Fullscreen API:** element.requestFullscreen() (and the webkit names), document.fullscreenElement / exitFullscreen() / fullscreenEnabled, fullscreenchange; the element is given a fixed, full-viewport style (its own restored after), the page scrolled to its top; Escape leaves. (The window is not maximised: a later step.)
- **Layout and painting:** <video> and <audio controls> are replaced boxes (box_special.c: their children not shown; without scripts the fallback content); their size from presentational hints (css/hints.c): width / height, else the video's natural size once known (a restyle when it comes), else 300 x 150; <audio controls> 300 x 54; audio:not([controls]) is display: none (default.css). The frame is the node's bitmap (as a canvas'), copied at each new frame and its box redrawn (html__redraw_a_box: the compositor uploads that rectangle); onyx_media_redraw (redraw.c) fits it in the content box (object-fit: contain) and draws the **native controls** when controls is set (the bar: play / pause, the time, the progress in red, the volume -- crossed when muted --, full screen; shown while paused, and for 3 s after the pointer moves over a playing video); their clicks and double clicks are handled in media.js.
- **YouTube** (tried on the PC bench, the real site): window.Window and ProcessingInstruction (dom.js: ShadyDOM patches their prototypes) and **a page that became "Error: OK"** (html.c: a style sheet finished while a script ran getComputedStyle -- the parser paused under it was taken for a failed parse) fixed: **the home page, the watch page and its player load and run** (no script error). The embed player (/embed/<id> in an iframe) probes isTypeSupported (vp9, av01, avc1 with width=, height=, framerate=, eotf=, tunnelmode=, decode-to-texture=...; opus, mp4a) and shows "This video can't be played with your browser". The bench's network was later refused the watch pages by Google ("unusual traffic": the 429 of google.com/sorry), not the embed pages. NS_MEDIASTUB=1 (the bench only: user/av/av_stub.c) decodes the codecs not built in into grey frames and silence, so that a site's media path runs on its real streams. With it the embed player, its play button clicked, went further and showed **a bug of every script request: a binary body** (an ArrayBuffer / typed array: YouTube gzips its player API's JSON, content-encoding: gzip; SABR's requests are protobuf) was sent as UTF-8 text cut at its first NUL -- the API answered 400 and the player "An error occurred". Fixed: dom.js' encodeBody hands the bytes as base64 behind a marker, onyx_fetch.c (onyx_body_decode) sends them as they are (the body's length kept: body_len). Then: the player API answers, **YouTube picks its formats from our answers** (Opus in WebM, AV1 at 480p -- the stand-ins claim AV1; with libvpx alone it would be VP9), addSourceBuffer twice, and starts **SABR** streaming (Server ABR: a POST to googlevideo.com/videoplayback?sabr=1 answered in UMP, application/vnd.yt-ump): the first answer is a redirect part (UMP part 43, to another googlevideo host), the second request gets its HTTP head and then **no bytes** within 80 s on the bench, and the player shows "Video unavailable".
- **YouTube with the real codecs** (the stand-ins off): the same path -- the player picks Opus in WebM and AV1 in MP4 (the embed's 480p is past AV1's smooth size but YouTube asks without width= / height= there; VP9 at 360p it asks with them), addSourceBuffer twice, SABR -- and the

same end: the first POST answered (1.1 KB: the redirect part), the second one's head (HTTP/1.1 200, `application/vnd.yt-ump`, chunked) and then nothing; "Vidéo non disponible". What was ruled out: **the transport** -- a POST of a binary body arrives byte for byte (a local server: length, content, sum), a chunked or HTTP/2 answer that comes in pieces seconds apart is handed to the script piece by piece over TLS (httpbin's `/drip`, HTTP/1.1 and HTTP/2) --; one bug was found and fixed on the way: the fetcher's HTTP/1.1 threads woke the page only every 32 KB of body, so a streamed answer that pauses under 32 KB (SABR's parts, server events) waited for its end (`onyx_fetch.c`, `in_flush_post`: what came is handed over before the thread waits for more). **The same request replayed with curl** (its URL, headers and body taken from the bench's log, even Jet's User-Agent and headers) is answered at once (151 bytes: UMP part 46, the server asking the player to reload its player response). So Google's server holds Jet's own stream open without data -- the cause is on its side of the wire: the player's attestation (BotGuard's PO token, which our run produces but YouTube may judge), the TLS fingerprint, or the bench's network (Google already refuses it the watch pages). The next step is a run from another network (the Pi at home) and a capture of a desktop Chrome's SABR exchange for the same video to compare the requests field by field. Debugging aids (the PC bench): `NS_MEDIADDEBUG=1` (media.js' steps on the console), `NS_NETBODY=<part of a URL>` (a script request's headers, its whole body, its response or its parts on stderr), `NS_NETDEBUG=1` (each response's head, each HTTP/2 frame), `NS_MEDIASUB=1` (the stand-ins; 0: off).

Speed on the Pi 4 (measured / estimated)

`tools/tests/av/bench.sh <clips>` (avbench: demux, decode, convert; the PC's C build, then the AArch64 build -- libvpx's NEON, dav1d's assembly -- under `qemu-aarch64 -cpu cortex-a72`; it prints a checksum of every frame's pixels: **the AArch64 build's frames are identical to the C build's** for every clip). Clips made with FFmpeg (PyAV) from its `mandelbrot` source (fine detail, constant motion: harder than most videos), 5 s at 30 fps, at YouTube's bit rates:

clip	PC, C code	AArch64 under qemu	YUV to RGB (qemu)
VP9 640x360, 500 kb/s	3.4 ms a frame	11.2 ms (key frame 72)	3.8 ms
VP9 854x480, 860 kb/s	5.7 ms	20.0 ms (95)	7.1 ms
AV1 640x360, 320 kb/s	5.0 ms	17.7 ms (148)	4.2 ms
AV1 854x480, 590 kb/s	7.9 ms	28.0 ms (211)	6.8 ms
Opus 48 kHz stereo	0.2 % of a core	3-4 %	--

qemu runs NEON code several times slower than the Cortex-A72 does (each vector instruction becomes a dozen of the host's); the YUV conversion, our own NEON loop of ~20 instructions for 16 pixels, takes ~1 ms on the A72 at 1.5 GHz for 480p against 7 ms under qemu. Scaled the same way (5-7x for these decoders, whose time is part scalar entropy decoding), **estimated on the Pi 4**: VP9 480p ~3-5 ms a frame, AV1 360p ~3-4 ms, AV1 480p ~5-7 ms with key frames of ~30-40 ms (the four decoded frames queued ahead absorb them). Onyx runs an app's threads on core 0 with the page's scripts and painting,

so "smooth" (MediaCapabilities, `isTypeSupported` with a size) is set with a margin: **VP9 / VP8 up to 854 x 480 at 30 fps, AV1 up to 640 x 360 at 30 fps** (`av_vpx.c`, `av_dav1d.c`). To be confirmed on a Pi: if AV1 480p proves comfortable, raise `av_av1_codec`'s size.

- **The page:** YouTube's watch page runs more than 14 MB of scripts (kevlar_base alone 10.8 MB, the player's base.js 3.0 MB); on the PC bench the watch page and its player settle in under a minute with the code cache cold (a bench run, not a timing). On the Pi expect a long first visit (the code cache helps the next ones); the embed player (`youtube.com/embed/<id>`: its base.js is 1.9 MB) is much lighter -- the better way to watch on the Pi.

The codecs (vendored, 2026-10-02)

- `third_party/libvpx-1.15.2` (BSD + Google's patent grant), `third_party/dav1d-1.5.1` (BSD), `third_party/opus-1.5.2` (BSD): the decoders' sources only (no encoder but libopus', no tests, no build systems), each with a `README.onyx` (what is kept, how it is configured). libvpx's configure was run once per target and its generated headers committed (`onyx/generic: C`; `onyx/arm64: NEON`, no run-time CPU detection, ARMv8.0); dav1d's and libopus' `config.h` are written by hand (`onyx/`). dav1d asks for pthreads, which newlib lacks: on the Pi a header of inline stand-ins (`onyx/pthread/pthread.h`; `user/av` opens it with one thread and a frame delay of 1, so it never starts a thread nor waits) and `sysconf` renamed (`onyx_sysconf.c`).
- **One source list for every build:** `user/av/codecs.mk` (the files, the flags, the `AV_WITH_*` and include paths of `user/av`'s glue), included by `user/net surf/Makefile` (the Pi: `libvpx.a`, `libdav1d.a`, `libopus.a`, `make -C user/net surf codecs`; dav1d's `.S` files assembled by GCC 13.3), `user/net surf/net surf-app.mk` (links them), `tools/tests/net surf/host.mk` (the PC bench: the C code), `pc/Jet/jet.mk` (Windows: the C code, dav1d's Win32 threads) and `tools/tests/av/bench.mk` (avbench). Jet on the Pi grows by ~1.1 MB (9.9 to 11.0 MB).
- The decoding threads get **1 MB stacks** (`av_os.h`; 256 KB before): dav1d's assembly keeps blocks on the stack (it gives its own threads 1 MB), libopus uses variable-length arrays.
- `av01` with 10 bits is no longer claimed (dav1d is built for 8 bits only).

Out of reach for now: the Pi 4's **H.264 hardware decoder** (the VideoCore's, behind the firmware's MMAL / V4L2 interfaces: a kernel driver Circle does not have -- a kernel project of its own); **HEVC** (hardware only); **EME / DRM** (Widevine: never). The decoder on an **app core** (`kapi_core_run`: no kapi and no malloc there -- the decoders allocate at each frame) would take the decoding off core 0: the next optimisation once a decoder runs.

Tests

- `tools/tests/av/run.sh` (no NetSurf): the clips of `tools/tests/av/mkmedia.py` (synthetic, made at test time, nothing committed: 1420 video with a moving bar and a flash every 0.5 s, a 440 Hz tone with a 1 kHz burst at the same times, WebM / MP4 / MP4 with its moov at the end / fragmented MP4 / MSE's WebM and fMP4 byte ranges, WAV, a FLAC made by mkmedia's own encoder -- every predictor, stereo mode, escapes, wasted bits --, an MP3 of silent frames with an Info frame); `avtest`: every container whole and in pieces of up to 7 bytes, the frames' bytes against the reference, seeks, FLAC bit-exact, the store's MSE cases (out of order, remove, appended twice, timestampOffset), YUV fast path = C path (also built for AArch64 and run under qemu-aarch64), the type answers, the player (first frame before play, playback to the end, every frame in order, A/V sync ~3 ms, a seek): **79 passed**, on the PC and AArch64.
- `tools/tests/net surf/mediatest.sh` (the bench, `mediasrv.py` with Range requests, the stand-in sound output `SIM_SOUND=1 / SIM_SOUNDOUT`): `<video src>` for each container (the events and their

order, 50 frames presented in order, the last frame's and a seek's frame's **pixels = the reference** -- the RGB sums mkmedia computes with the same conversion --, buffered, the window's picture), MSE with WebM and fMP4 (appends out of order, appendBuffer while updating, abort, remove, endOfStream, playback, **A/V sync < 6 ms**), `<audio>` FLAC then `new Audio()` WAV (the sound heard **bit-exact**), MP3, a 10 s file by 64 KB ranges and a seek to 8 s before those bytes came (17 requests, not the whole file), the type answers: **all passed**. `jstest.sh`, `acidtest.sh` and the others pass as before.

- **The codecs in mediatest** (2026-10-02): `tools/tests/av/clips/` (committed, 110 KB, made by `tools/tests/av/mkcodec.py` with PyAV from mkmedia's picture and sound: 96 x 64, 25 fps, 2 s, a key frame each 0.5 s, Opus at 64 kb/s): `vp9.webm` (VP9 + Opus) and `av1.mp4` (AV1 + Opus) in `<video src>` -- the events, 50 frames in order, **the last frame's and a seek's frame's pixels equal to the reference decoders' frames** (libvpx's VP9 via FFmpeg, libdav1d; their RGB sums in `<clip>.txt`), the 440 Hz tone heard --, `vp9-mse.webm` and `av1-frag.mp4` through MSE (the same steps as the raw clips; the buffered ranges within 20 ms -- an Opus packet -- of theirs), and the type answers (`vp9`, `av01`, `opus`: "probably"; `avc1`: ""): **106 checks, all passed**. The runs now end when the page logs `done` (the desktop simulator's new `waitlog N TEXT` step, `SIM_LOG`) instead of after a fixed number of turns, and the seek's frame is read once it is shown: the seek check failed now and then on a loaded machine.

45. Netflix's sign-in code: the boxes no digit could be typed in (2026-10-01)

The user, on `netflix.com`: the home page and the sign-in page fine, the e-mail address typed, Continue, then the page asking for the one-time code sent by e-mail -- "impossible to type the digits into the boxes". An audit on the PC bench, without an account: no address was entered on `netflix.com` and no code was asked for (the code step needs a real address); the code step's component was read in the sign-in page's script (`assets.nflxext.com/.../clcs/login/client.*.js`, 4.6 MB, read-only) and rebuilt as a local page with the real React.

What Netflix does. The sign-in page is server-driven ("CLCS": the server sends a tree of components, the client renders them with React). The code step is a `CLCSInputPinCode`, drawn by the design system's (Hawkins) `<InputPinCodeV2>`: not six fields but **one** -- `<input type="text" autocomplete="one-time-code" inputmode="numeric" pattern="[0-9]*">`, its text color: transparent (caret-color: black, font-family: monospace, a letter-spacing and a negative text-indent computed from CSS custom properties so that the caret falls on the next box), with no label. The six boxes are its **"chrome"**: Hawkins' `FormControl` puts after the field a `<div aria-hidden data-hcw-form-control-chrome>` that is `position: absolute; inset: 0; z-index: -1; background: white; border: ...` -- **behind** the field, in the field wrapper's stacking context (`position: relative; z-index: 0`) -- and the code component fills it with a span per box (the digits typed, then "O" placeholders). The value is React-controlled: `onChange` keeps the digits (`/\D/g` removed), six at most, and sets the state; on focus the caret is put at the end in an animation frame (`setSelectionRange`). The e-mail field is a `FormControl` too (its chrome behind it as well), but it has a floating label over most of it: a click on the label focuses the field -- the e-mail could be typed.

What was wrong in Jet.

1. **A negative z-index was painted, and hit, in place** (`redraw.c` §21: "a negative z-index is painted in place"): in the tree's order, so the chrome -- after the field in the tree -- was painted over the field and **took its clicks**: the mousedown went to the chrome's `<div class=chars>`, which is not

focusable; the field never got the focus, the keys went to the page. That is the bug the user saw (reproduced: `mousedown` on `DIV.chars`, no focus event, nothing typed). CSS 2.1 appendix E: a stacking context paints its children with a negative z-index **after its own background and borders, before its in-flow content**.

2. **The caret vanished at each key:** React sets the value attribute back at each render (`node.defaultValue = value`), and Jet rebuilt the field's boxes for a `value` attribute change; and any rebox (React re-rendering the boxes' spans) reformats the page -- the browser window hides the caret then (`CONTENT_MSG_REFORMAT`) and nothing placed it again: the field was typed in with no caret.
3. **Less visible, but what other code widgets rely on:** the `input` event came after `keyup` (a widget moving on at `keyup` read the old value) and was a bare `InputEvent` (no `inputType`, no `data`); no `beforeinput`; no `paste` event (no `ClipboardEvent`, no `DataTransfer`) -- a six-box widget sharing a pasted code out got the whole code in its first box; `maxLength` was not kept (a `maxLength="1"` box took "12"); `select()`, `setSelectionRange()` were no-ops and `selectionStart` always the text's end; `focus()` on the next box waited for the rebox (the next key could still go to the box before); a key with no text field focused went to the root, not to `document.activeElement` (a `<div tabindex>` widget); `Ctrl+V` gave no `keydown`; `keypress`'s `keyCode` of a letter was the upper case's.

The fixes (no site-specific code).

- **Negative z-indexes** (`redraw.c onyx_negz_*`, `html_redraw_negz`; `interaction.c onyx_hit_negz`): `html_redraw_box_children` of a stacking context (the root; a layer with a z-index set, fixed / sticky, a compositing group: `html_redraw_stacking_context`) first paints the boxes with a negative z-index it holds -- found by a walk that does not go into the stacking contexts inside it (they paint their own), culled by the redraw's clip, sorted by z-index then the tree's order, each placed from the context's origin and clipped by the overflow of the boxes between them (an absolute one not by a static scroller between it and its containing block, as `onyx_oclip_escape`), each with what it puts off; the in-flow walk passes over them. The hit test looks at them first (under the rest): the field takes the click, the chrome only where nothing in-flow covers it. A z-index -1 box inside a positioned box with z-index auto belongs to the outer context: painted under that box's background, as in Chrome.
- **The caret kept:** a `value` attribute change on a text field updates its control (`dom_event.c onyx_attr_text_value -> html_textty_element_update`) instead of rebuilding its boxes; after a reformat the focused field's caret is placed again (`html.c html_caret_refresh`, scheduled by `html_reformat`; `textarea_onyx_caret_refresh`).
- **The key's events in browser order** (`interaction.c html_keypress`, `html_keypress_edit`): `keydown`, `keypress`, then for an edit in the focused field `paste` (a `ClipboardEvent` whose `clipboardData.getData('text')` is the clipboard's text; prevented: not pasted), `beforeinput` (prevented: no edit), the edit -- **maxLength kept** (what is typed or pasted beyond it dropped; read from the element at the time) --, the **input event at once** with its `inputType` (`insertText`, `insertFromPaste`, `deleteContentBackward`, `deleteContentForward`, `deleteByCut`, `insertLineBreak`, `historyUndo` / `Redo`) and `data` (`js_event_init`'s new `data` / `input_type`; `html_script_changed_flush`), then `keyup` (at the field focused by then). `Ctrl+A` / `C` / `V` / `X` / `Z` / `Y` are `keydown`s of "a", "c", "v"... with `ctrlKey` (no `keypress`).
- **The selection** (`textarea_onyx_get_selection` / `textarea_onyx_set_selection`, in characters; natives `controlSelection` / `controlSelect`): in the focused field `selectionStart` / `selectionEnd` are the browser's, `setSelectionRange()`, `select()`, `setRangeText()` and the setters move its

caret / selection (what is typed replaces a selection); a field without the caret keeps what a script set.

- `focus()` puts the caret in the field now when its box is there (`html_script_focus_control`), a rebox due or not (the rebox keeps the focus on its control).
- dom.js: `ClipboardEvent`, `DataTransfer` (text), `InputEvent.getTargetRanges`, keys to `document.activeElement` when no text field has the caret, keypress's `keyCode` / `charCode` the character's ("a": 97), code "Space". And `navigator.serviceWorker` is no longer an undefined key ('serviceWorker' in navigator was true: the sign-in page threw at `.getRegistrations`).

Tests. `tools/tests/netsurf/otptest.sh` (new; local pages, no network; React 18's production builds in `pages/react/`, MIT): `pages/otp-netflix.html` -- `<InputPinCodeV2>` rebuilt with React: a click on the boxes focuses the field, "1 2 a 3 4 5 6" gives 123456 (the letter dropped by `onChange`), the caret drawn while typing, a pasted "98-76 54" gives 987654; `pages/otp-boxes.html` -- six `maxlength=1` boxes moving on at `input` (focus + select of the next), at `keyup`, Backspace back, `keydown` filtering (`key` / `code` / `keyCode`), a paste shared out (`clipboardData`), `maxlength=4` kept (typed and pasted), `inputType` / `data`, `setSelectionRange` and typing over a selection, `beforeinput` prevented; `pages/otp-react.html` -- six React-controlled boxes (`ref` focus, `select`, `onPaste`) and an invisible field (`opacity: 0`) over drawn boxes in a `<label>`; `pages/stack-negz.html` -- the painting order (a z-index -1 box under the in-flow block after it, under a z-index auto parent's background, under a static block, -2 under -1, clipped by an `overflow: hidden` box) with and without the GPU compositor, and the clicks. On the build before, the Netflix page's click went to the chrome (no focus, nothing typed) and the caret was not drawn. jstest, typeaheadtest, findtest, dltest, acidtest (Acid2 identical, Acid3 100 / 100), uatest, iframetest, fxtest pass; gputest's 3a as before (§41). Checked on the live sign-in page: no script error, the e-mail field typed in (a placeholder text, nothing sent).

Not done. A text field's `text-align`, `text-indent` and `letter-spacing` are not drawn by its text area (NetSurf's `desktop/textarea.c`): Netflix's caret shows at the field's left, not on the next box (the boxes themselves are right), and a box's digit is at its left instead of centred; no `contenteditable` (a code widget built on it cannot be typed in); Tab moves the caret to the next field without telling the scripts (`document.activeElement`, focus events); no composition (IME) events. **To try on the Pi:** the code step itself (an account needed).

8. Known gaps

- JavaScript: synchronous XHR (runs async), multipart request bodies, binary request bodies (sent as text: a NUL ends them); no IndexedDB, editing APIs, Service Workers; CSS `:active` / `:focus`.
- The network APIs (§19): a worker runs on the UI thread (a long computation in a worker holds the window as a page's script does; `script_timeout` stops it); an `importScripts` whose URL is not a literal in the script, over http(s), blocks the UI thread while it is read; a SharedWorker is shared within its document only; no `MessagePort` transferred to a worker, no `OffscreenCanvas`, `SharedArrayBuffer` / `Atomics` between workers; the WebSocket client sends its messages uncompressed (permessage-deflate inflates only) and never pings; CORS is not checked for EventSource either.
- The network (§24): no revocation checks, no CSP, no SameSite cookies, no CORS for the core's loads (EventSource, fonts, `<img crossorigin>`, module scripts), no HTTP/3, no back-forward cache.
- Frames (§29): a frame's initial about:blank is not there synchronously, no `document.domain`, a srcdoc's `location.href` is its data: URL, no focus moved between frames by script, `loading="lazy"` frames load at once, no frames in shadow trees.

- Shadow DOM (§20): the manual slot assignment's rendering, `exportparts`, a clonable root's cloning, `<link>` / `@import` / `@font-face` in shadow trees, `:host` in `matches()`; `::before` / `::after` of a `display: contents` element.
- The compositing layers' gaps (§21: a real perspective, `clip-path` / `mask` on a layer, the non-separable blend modes); animations: §22's "Not done"; GPU compositing: §25's (scrollers and groups as layers, fixed boxes that stay in view).
- WebAssembly and Web Crypto (§27): no Wasm SIMD, threads, `WebAssembly.Tag` / `Exception`, JSPI; no Ed25519 / X25519; RSA key generation blocks the window.
- JavaScript speed (§30): the JIT is a first version, off by default (a `catch` goes on interpreted, calls go through the interpreter's frame set-up); the code cache keys a script by its exact text (a script that changes at each load is compiled each time); a single long script still holds the window (its events are kept and handled after it).
- SVG: no `<mask>`, `<pattern>`, `<marker>`, filters, SMIL animations, `<textPath>`, per-glyph position lists, the page's web fonts in `<text>`; the page's CSS `fill` / `stroke` (`.icon path { fill: red }`) do not reach an inline `<svg>` -- libcss has no `fill` / `stroke` properties (only `fill-opacity` / `stroke-opacity`): with them (inherited, as text), `onx_svg_inline.c` would write the `<svg>`'s computed ones on its root; its `color`, its custom properties and the SVG's own attributes and `<style>` do. An `<svg>` / SVG `<img>` with only a `viewBox` and no CSS size takes the `viewBox`'s size (Chrome: the containing block's width); a `list-style-image` SVG without a size is drawn at its `viewBox`'s size (Chrome: small). The scripts' SVG DOM is `dom.js`' generic one (`namespaceURI`, `getBBox` from the box).
- Canvas: no shadows, filters, blend modes (source-over), conic gradients (a solid colour), per-corner `roundRect` radii; `drawImage` scales with the nearest pixel (PlutoVG's textures), and a canvas shown at another CSS size too (the framebuffer's bitmap plot); no `getContext('webgl')`, `captureStream`, video frames; the page's web fonts are not used (the card's fonts by family).
- A face split by `unicode-range` outside Latin-1 falls back to the card's fonts.
- Intl (§15): no `Intl.DurationFormat`, no Temporal; the Gregorian calendar only (another `calendar` falls back to it, no `relatedYear` / `yearName`); a date range's CLDR interval patterns are approximated (the shared fields, the locale's dash); no collation tailorings but Spanish and Nordic, no `co` types (phonebook, pinyin...); word and sentence segmentation without dictionaries; time zones with their current rules only (no history), four zones with irregular rules (Casablanca, El Aaiun, Gaza, Hebron) at a fixed offset; the lean languages (ja, zh, ko, ru, pl, sv, da, nb, fi, tr, cs) have English units, display names and zone names.